



BITS Pilani presentation

BITS Pilani
Pilani Campus

Prof. Narasimha Bolloju
CSIS Department, Hyderabad



SS ZG622

Software Project Management

Lecture No. 1

Activity - CS01#1



- What went wrong here, and how could better project management have helped?



About the SPM course

- Live lectures with recorded content on Taxila/elearn platform and MS Teams
- Interactive class activities individual or small groups (on MS-Teams)
- Audio/video participation expected and encouraged
- Expected to pause the video and work on the activities for the sessions you missed
- ACL case study will be used in all contact sessions

Activity - CS01#2



- a) What software project managers do or expected to do?
- b) What are the challenges associated with managing software development projects?

Topics for today

- Project vs. program vs. portfolio
- SDLC vs. project life cycle
- Why SPM is challenging?
- Evaluation scheme and assignment details
- Objectives for information system and project management

Activity - CS01#3



Please rank the possible reasons for software project management failure on Pollev in the descending of order of relevance.

*Portfolio
Management*

Portfolio

*Program
Management*

Program

*Project
Management*

Project

Project

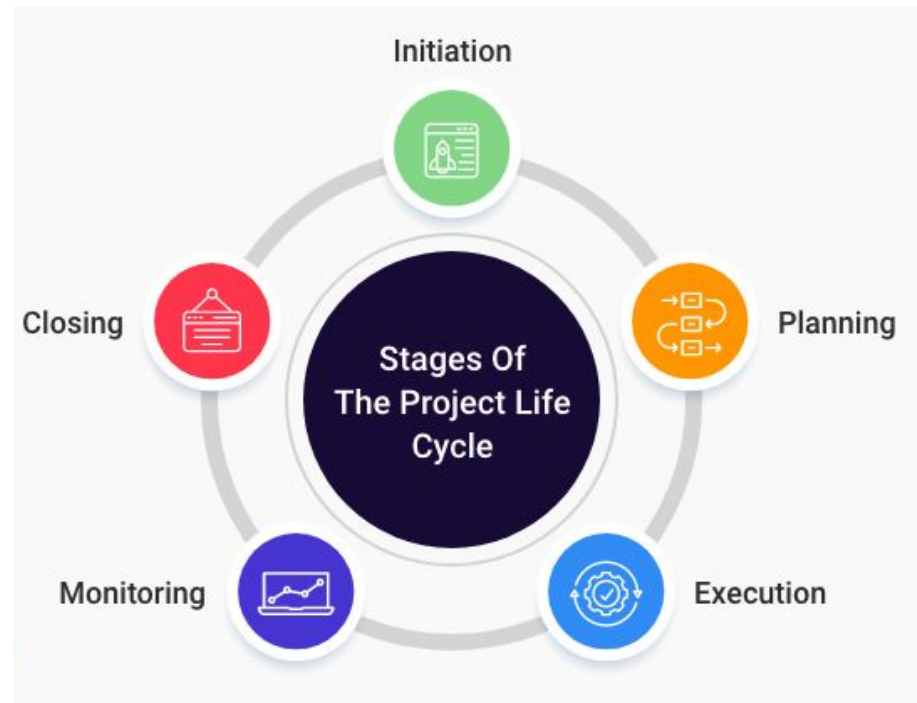
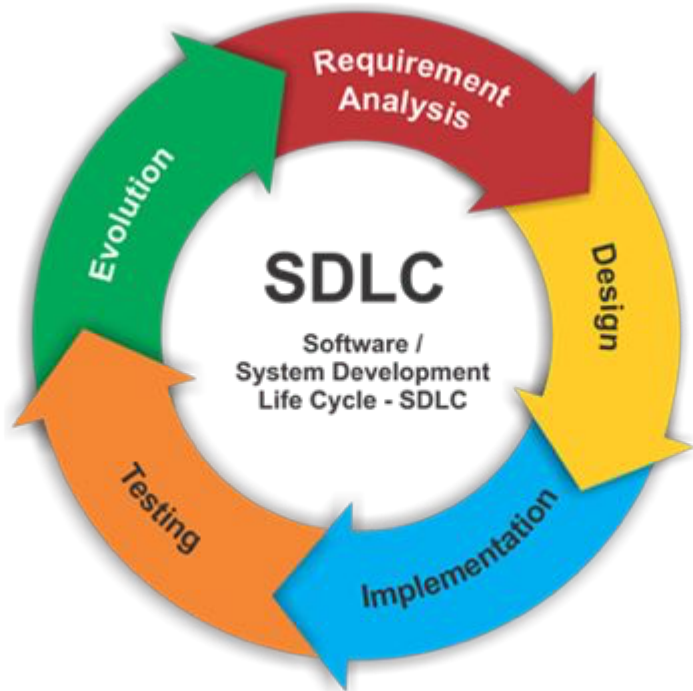
Project

Project

Project

Project

SDLC vs project life cycle





Software development projects – Success vs failure

MODERN RESOLUTION FOR ALL PROJECTS

	2011	2012	2013	2014	2015
SUCCESSFUL	29%	27%	31%	28%	29%
CHALLENGED	49%	56%	50%	55%	52%
FAILED	22%	17%	19%	17%	19%

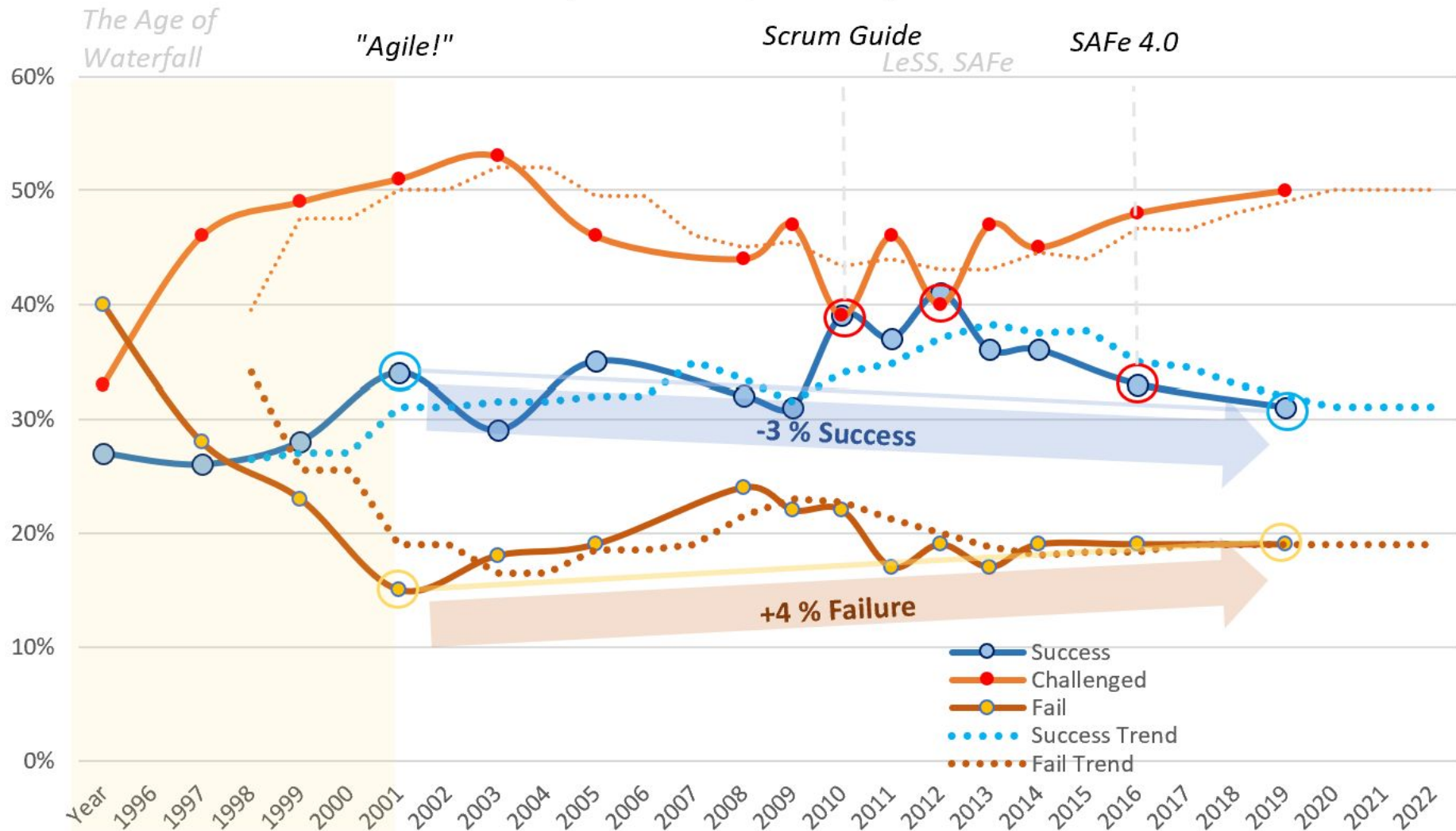
The Modern Resolution (OnTime, OnBudget, with a satisfactory result) of all software projects from FY2011–2015 within the new CHAOS database. Please note that for the rest of this report CHAOS Resolution will refer to the Modern Resolution definition not the Traditional Resolution definition.

<https://www.infoq.com/articles/standish-chaos-2015>

Software Project Outcomes (CHAOS Database, 2011-2015)



Standish Group Chaos Report: Project Outcomes



<https://failfastmoveon.blogspot.com/2023/01/is-agile-just-smoke-and-mirrors.html>

Activity - CS01#4



Why A2Z Cinemas Ltd (ACL) wanted to start Mux Core Project (especially when they have other systems providing the required functionality)?

Major topics to be covered in this course



- Project planning; project scheduling; AoN diagrams; resource scheduling; critical path
- Types of risks and their management alternatives
- Software product and process quality; quality attributes; quality planning
- Software change management; process and CCB

Major topics to be covered in this course (contd.)



- Project monitoring and control; managing teams/stakeholders; managing attrition and distributed teams
- Contracts and their management
- Project evaluation and program/IT-portfolio evaluation
- Project closure activities
- Migration to agile process model
- Leveraging AI trends in project management

Evaluation scheme

No	Name	Type	Duration	Weight	Day, Date, Session, Time
EC-1	Quiz 1	Online	1 week	5%	Before mid-sem test
	Quiz 2	Online	1 week	5%	After mid-sem test
	Assignment 1 and Assignment 2	Report	~ 2 weeks each	10% + 10%	Before the mid-sem test and Before the comprehensive exam
EC-2	Mid-Semester Test	Closed Book	2 hours	30%	Refer to the semester calendar
EC-3	Comprehensive Exam	Open Book	2 hours	40%	Refer to the semester calendar

Individual assignments

- A two-part report on a randomly assigned topic related to software project management
- Requires 10 to 12 hours of total effort in study, collecting relevant information from your current or past projects, and writing the report
- Submissions will be checked for plagiarism using Turnitin tool (no marks will be awarded for reports with similarity score above 25%)
- Further details will be provided after topic assignment by third week of the semester

Sample topics for the assignment



Software effort estimation

Project initiation or planning

Project management life cycle

Project risk management

Techniques for quality improvement

Project scheduling

Build or buy decision making

Project evaluation

Activity - CS01#5



Homework -

For the Mux-Core project of ACL, identify objectives for

- a) Mux-Core system and
- b) Mux-Core project management



BITS Pilani presentation

BITS Pilani
Pilani Campus

Prof. Narasimha Bolloju
CSIS Department, Hyderabad



SS ZG622

Software Project Management

Lecture No. 2

Objectives, buy vs build & process model selection



1. Project vs. project management objectives
2. Build versus buy decision
3. Process model selection
4. Plan driven process models
5. Agile methods

What is a Project?

- A complex, non-routine, one-time effort limited by time, budget, resources, and performance specifications designed to meet customer needs.
- A project is unique, has specific time frame (not a continuing or ongoing activity like automobile production, ...), requires resources, interdependent activities, and has a degree of uncertainty.

Project Stakeholders

These are people who have a stake or interest in the project

In general, they could be *users/clients* or *developers/implementers*

They could be:

- Within the project team
- Outside the project team, but within the same organization
- Outside both the project team and the organization (society,...)

Different stakeholders may have different objectives – need to define common project objectives

Key Stakeholders



Project characteristics



- Risks and uncertainties e.g.
 - are requirement well understood?
 - are technologies to be used well understood?
- Type of application being built e.g.
 - information system or embedded system?
 - criticality vs. differences between target and development environments?
- Clients' own requirements
 - need to use a particular method

Activity W02#1



Differentiate project objectives and project management objectives for ACL Mux Core Project.

Project objectives vs

Project management objectives



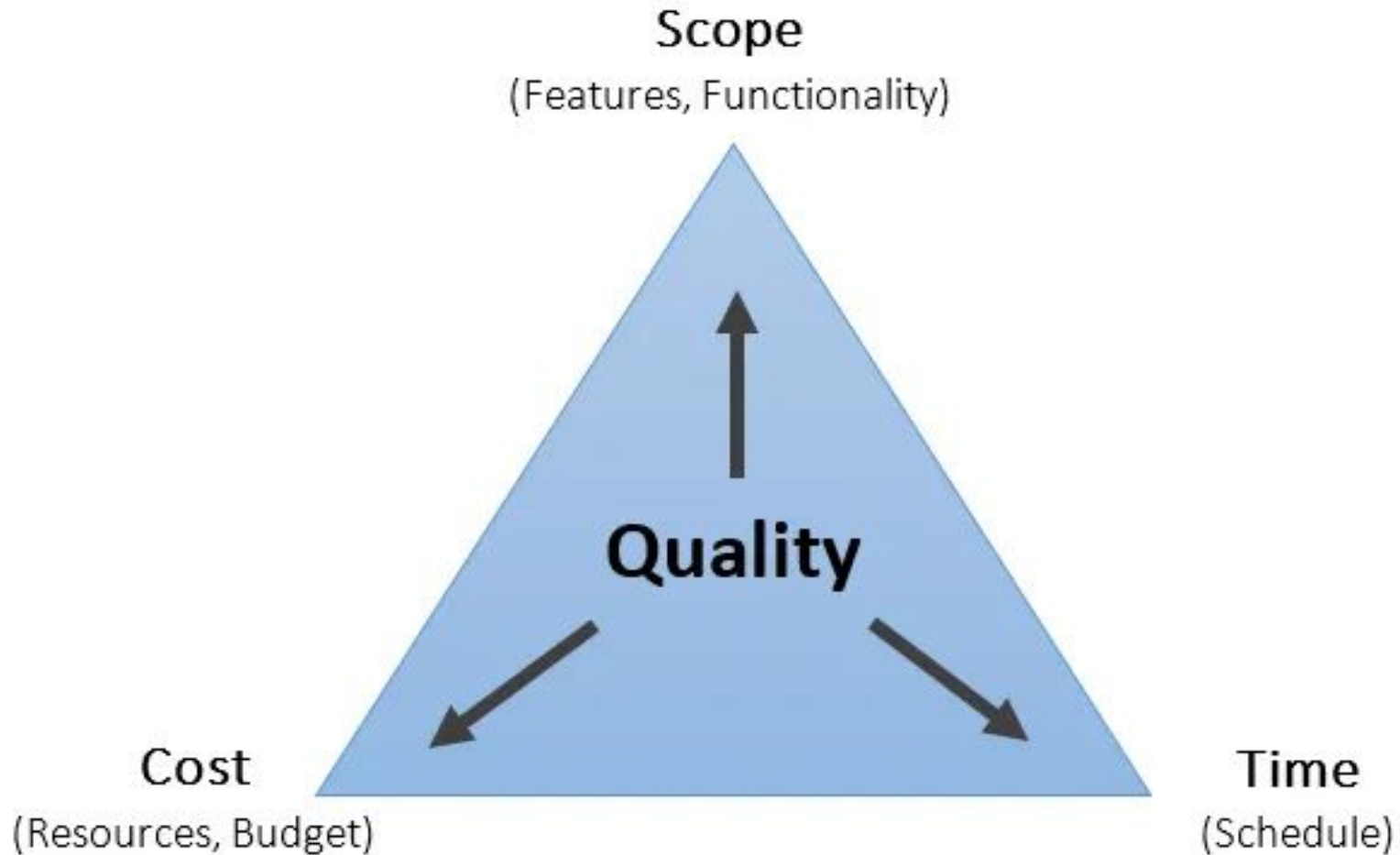
Project objectives

- define the **desired outcomes or deliverables**, focusing on the **end product or system** and its alignment with business goals.
- describe **what** the project aims to achieve in terms of functionality, performance, and value to the organization.
- centered on the **product** and its impact on stakeholders, such as customers, employees, or the business.

PM objectives

- focus on the **process of delivering the project**, emphasizing how the project is planned, executed, and controlled to ensure successful completion.
- describe **how** the project will be managed to achieve the project objectives.
- centered on the **project execution**, including timelines, budgets, resources, risks, and stakeholder coordination.

Project management triangle



<https://medium.com/@harpreet.dhillon/iron-triangle-triple-constraints-of-project-management-e818e631826c>

Build vs Buy Decision



- Option 1 - Buy a COTS solution
- Option 2 - Build in-house
- Option 3 - Build with a third party vendor

Option 1: Buy a commercial off the shelf (COTS) solution



- Purchase a pre-built software solution from a vendor that meets the project's functional requirements (e.g., CRM, ERP, or industry-specific systems). Implementation involves licensing, configuring the software to fit organizational needs, and integrating it with existing systems. Vendors typically provide cloud-based or on-premise options, along with ongoing support and updates.
- **Key Benefits:** Fast deployment (3-6 months), lower initial costs, and reduced development risk due to proven solutions. Ideal for standard requirements and organizations with limited technical resources.
- **Key Drawbacks:** Limited customization, potentially requiring workarounds for unique needs. Ongoing subscription or maintenance fees and dependency on vendor support.

Option 2: Build in-house



- Develop a custom software solution using the organization's internal IT team, tailored to specific requirements. The process involves defining requirements, designing, coding, testing, and deploying the system, following a software development life cycle (SDLC). The organization handles ongoing maintenance and updates.
- **Key Benefits:** High customization to meet unique needs, full control over the system, and no recurring licensing fees. Enables innovative features and aligns closely with organizational goals.
- **Key Drawbacks:** High costs, long timelines (12-24 months), and significant risk due to complexity and resource demands. Requires a skilled in-house team and robust project management.

Option 3: Build with a third-party vendor



- Partner with a software development vendor to build a custom solution tailored to the organization's needs. The vendor manages development, testing, and integration using methodologies like Agile, while the organization provides requirements and oversight. Maintenance can be vendor-managed or transitioned in-house.
- **Key Benefits:** Balances customization with professional expertise, moderate costs, and faster delivery (9-18 months) than in-house builds. Reduces risks through vendor experience and resources.
- **Key Drawbacks:** Higher costs than COTS, some vendor dependency, and requires strong coordination to align requirements. Potential for delays if scope changes occur.

Activity W02#2



Which one of the three options is best suited for the Mux Core System at ACL? And, why?

Development approaches



Plan-driven approaches

- Step-by-step methods where each step and intermediate product is carefully defined
- Emphasis on getting quality right first time
- Rapid application development is an alternative

Agile approaches

- Emphasis on speed of incremental delivery rather than documentation

Process model choices

- Waterfall model
- Evolutionary process model
- Incremental and iterative process models (e.g., Unified process)

Or

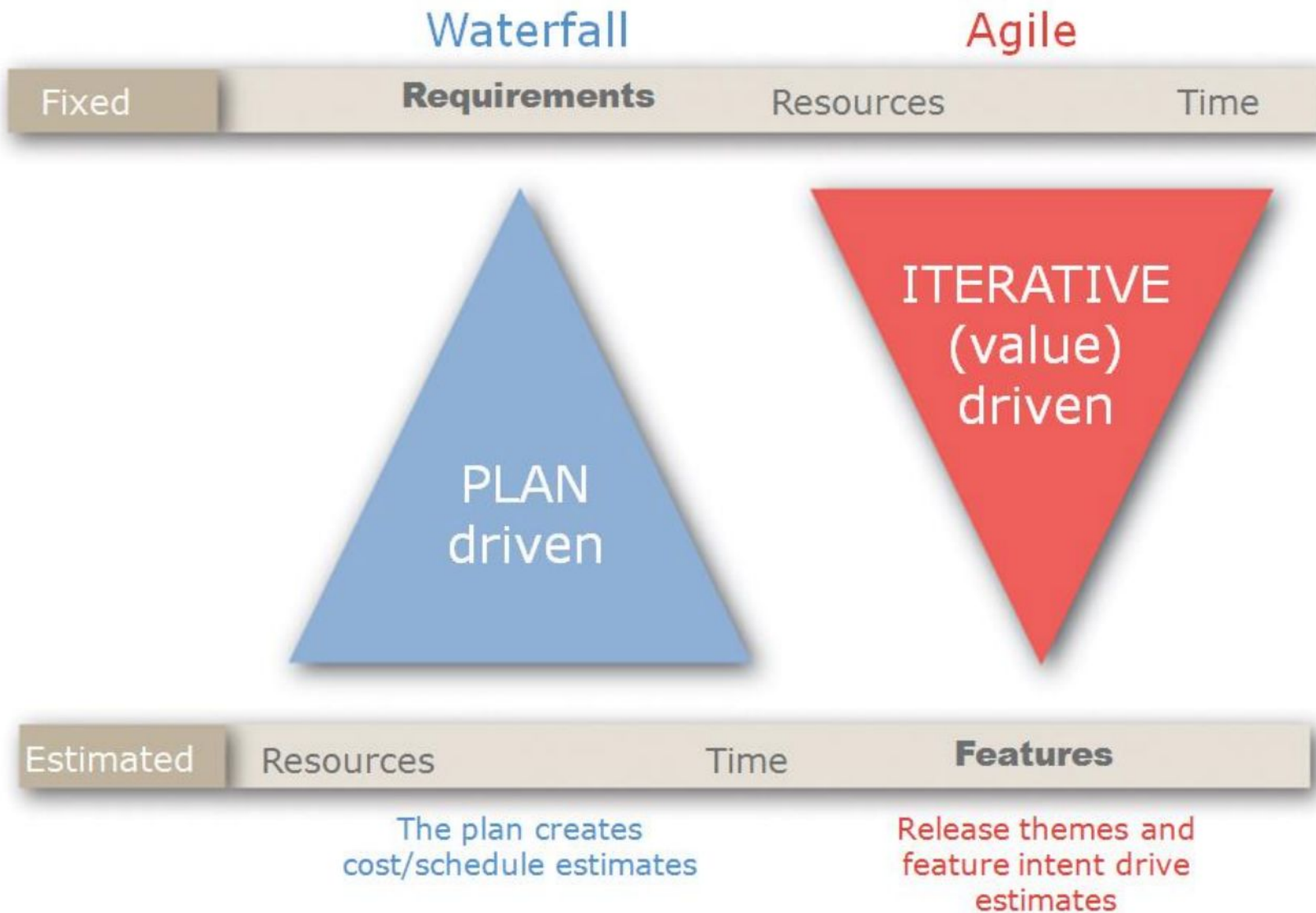
- Agile methods (e.g., Scrum, XP, ...)

Plan-driven process models vs. agile methods



- In plan-driven process models, activities are planned in advance and progress is measured against the plan
- In agile process models, planning is incremental and it is easier to change the process to reflect changing requirements
- There is no right or wrong process model; most organizations/teams include elements from both in their process models

The Agile Paradigm Shift



Activity W02#3



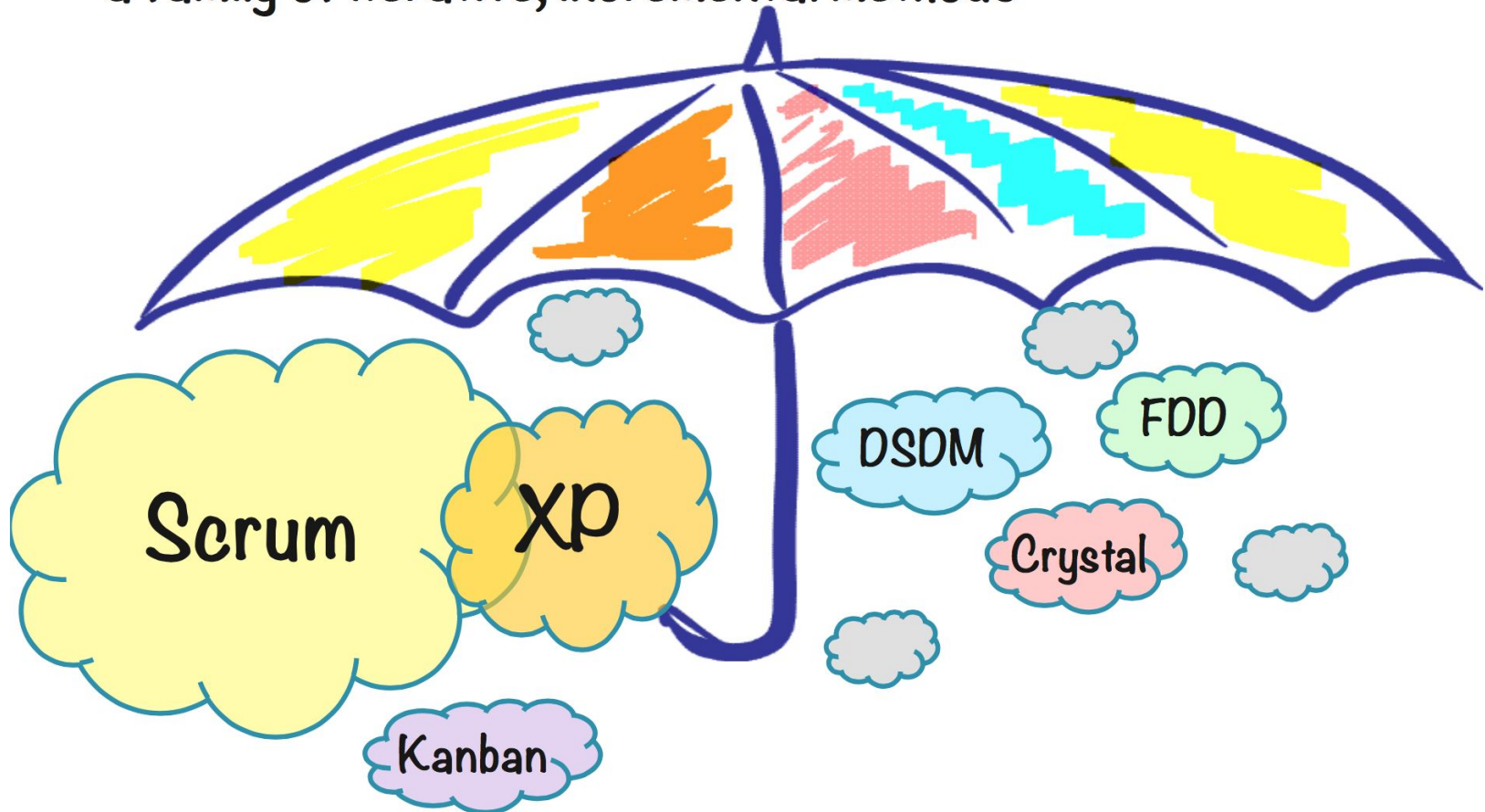
Identify pros and cons of using the following process models for the Mux-Core Project

- waterfall
- prototyping
- iterative/incremental

Agile methods



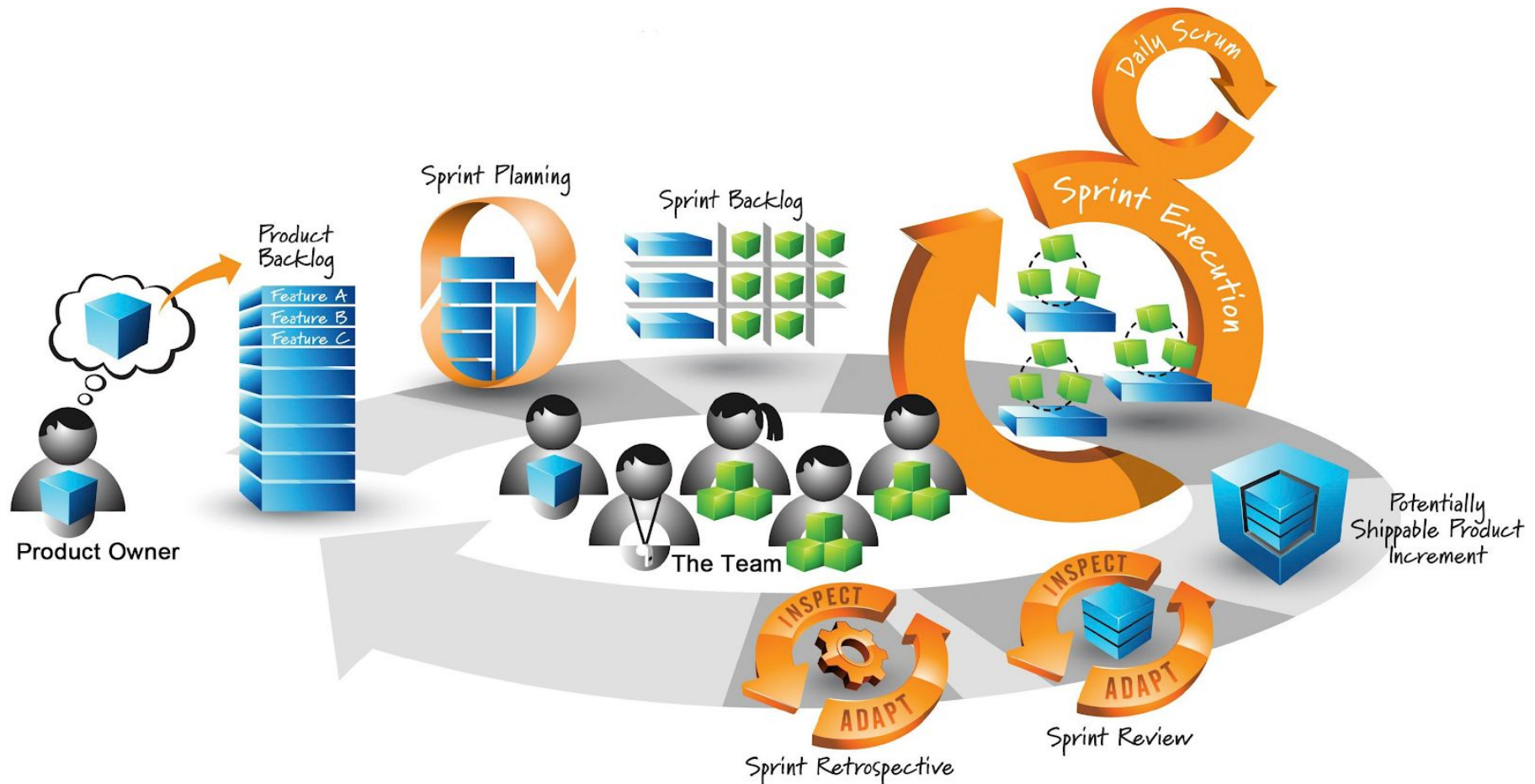
Agile "umbrella" –
a family of iterative, incremental methods



Scrum overview

- A framework within the agile paradigm that organizes work into time-boxed iterations called sprints (typically 2-4 weeks).
- Emphasizes roles (Product Owner, Scrum Master, Development Team), artifacts (Product Backlog, Sprint Backlog, Increment), and events (Sprint Planning, Daily Scrum, Sprint Review, Sprint Retrospective).
- Focuses on project management and team collaboration, providing a structure for delivering increments of working software while adapting to changing requirements.

Agile method Scrum



<http://www.programmeronrails.com/2016/03/20/scrum-overview/>



Extreme Programming Overview

- An agile methodology that focuses on engineering practices to improve software quality and responsiveness to customer needs.
- Emphasizes frequent releases, continuous feedback, and technical excellence through practices like pair programming, test-driven development (TDD), continuous integration, and refactoring.
- More prescriptive about how development work is performed compared to Scrum.
-

Activity W02#4



- a) Do you think Scrum is the right process model for the Mux-Core Project development?
- b) How XP can be used during the development?

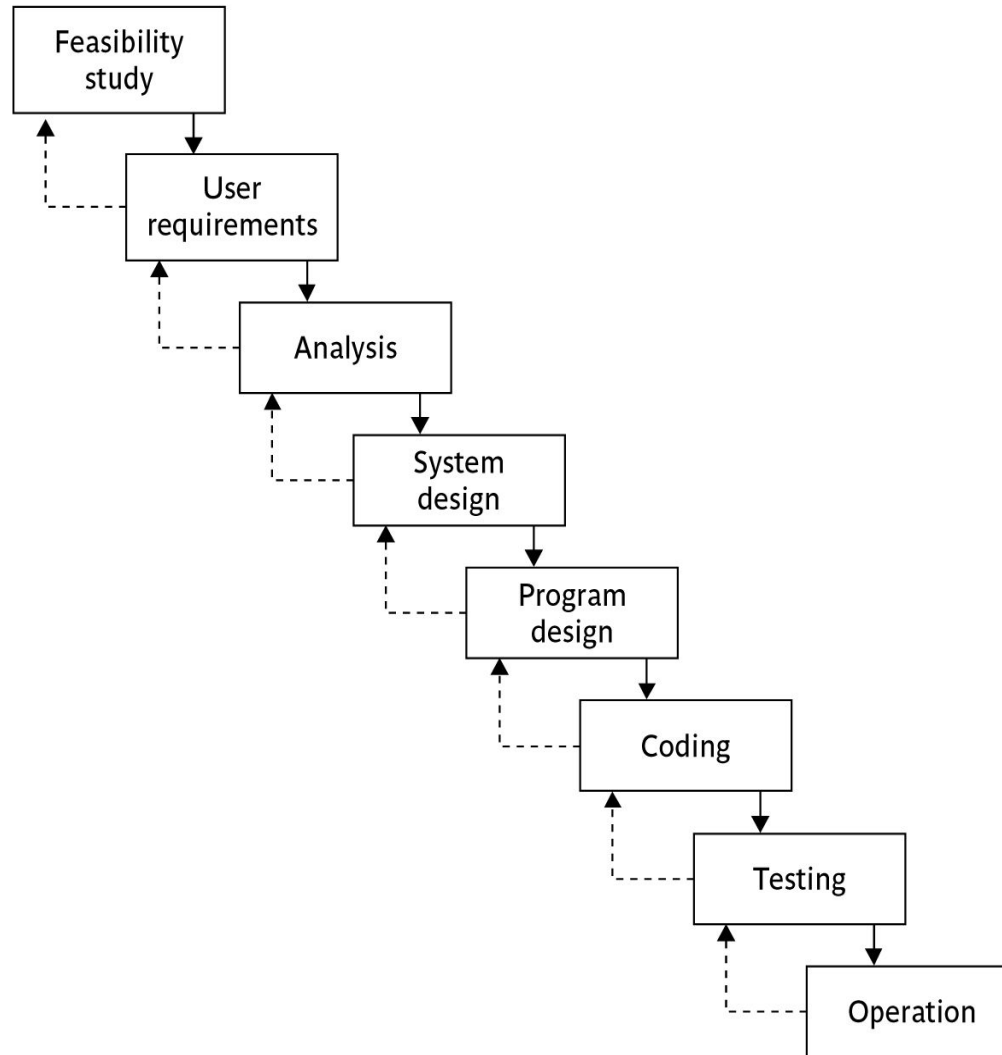
Activity W02#5



Explain how Scrum is different from the iterative and incremental process model.

Additional slides (on process models)

Waterfall Model



Waterfall

- the 'classical' model
- imposes structure on the project
- every stage needs to be checked and signed off
- BUT
 - limited scope for iteration
- V model approach is an extension of waterfall where different testing phases are identified which check the quality of different development phases

Evolutionary prototyping

An iterative process of creating quickly and inexpensively live and working models to test out requirements and assumptions'

- 'throw away' prototypes
- evolutionary prototypes

what is being prototyped?

- human-computer interface
- functionality

Reasons for prototyping

- learning by doing
- improved communication
- improved user involvement
- a feedback loop is established
- reduces the need for documentation
- reduces maintenance costs i.e. changes after the application goes live
- prototype can be used for producing expected results

Prototyping: some dangers

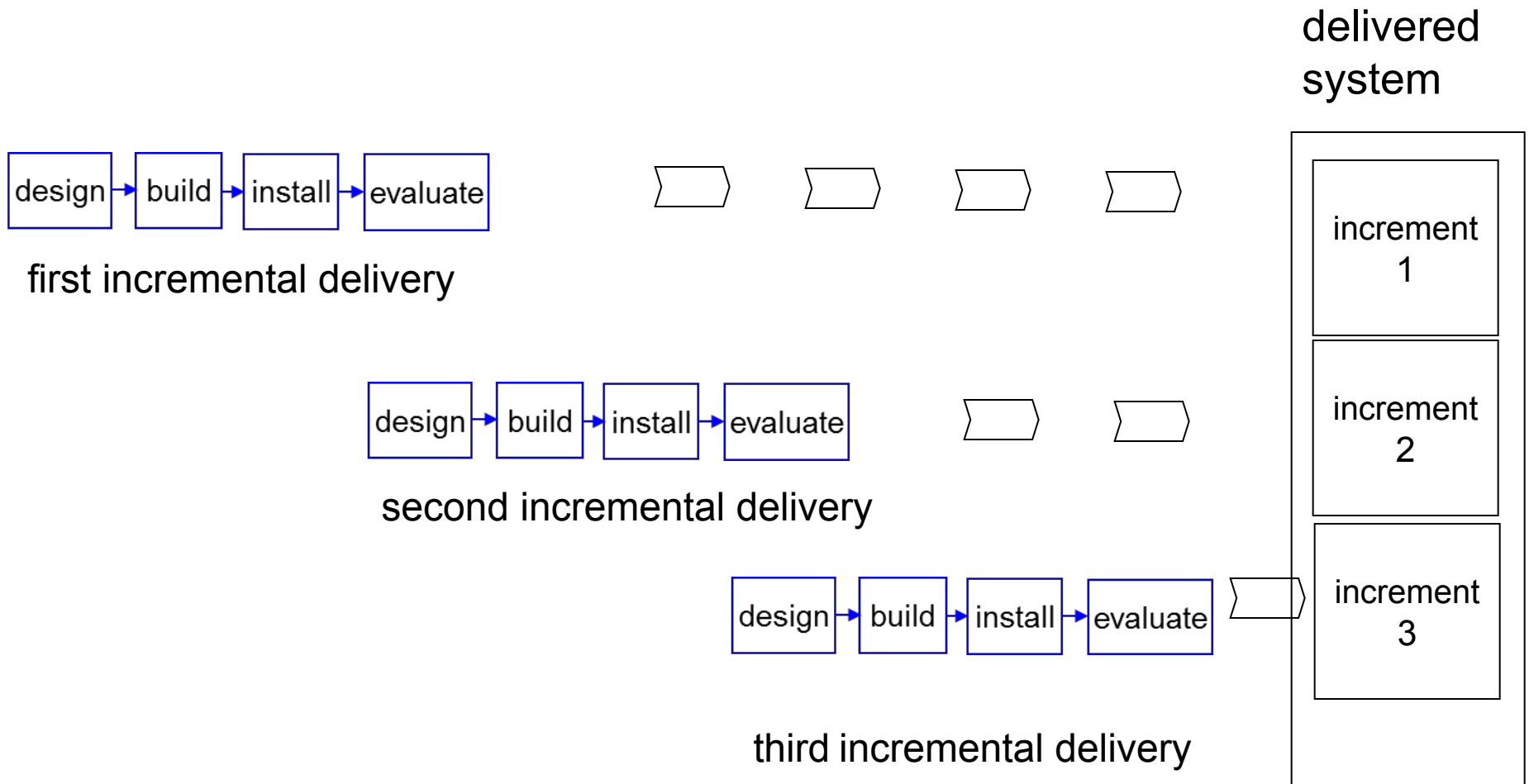
- users may misunderstand the role of the prototype
- lack of project control and standards possible
- additional expense of building prototype
- focus on user-friendly interface could be at expense of machine efficiency

Categories of Prototypes



- Throw-away Prototypes
 - ‘experimental’ (HW/SW prototype)
 - ‘exploratory’ (application prototype)
- Re-usable Prototype
 - ‘evolutionary’ build (for continuous feedback)
 - Final prototype = Deliverable!

Incremental and iterative models



Incremental and iterative models: benefits



- feedback from early stages used in developing latter stages
- shorter development thresholds
- user gets some benefits earlier
- project may be put aside temporarily
- reduces 'gold-plating':

BUT there are some possible disadvantages

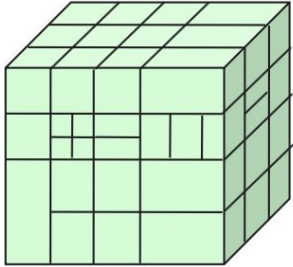
- loss of economy of scale
- 'software breakage'

Incremental planning

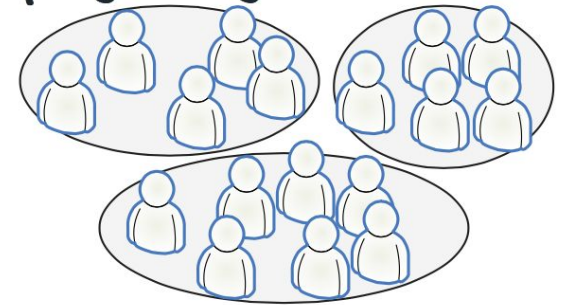
- steps ideally 1% to 5% of the total project
- ideal if a step takes one month or less:
 - not more than three months
- each step should deliver some benefit to the user
- some steps will be physically dependent on others

Scrum in a nutshell

Split your product

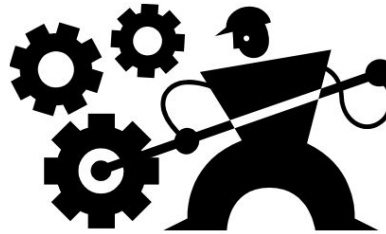


Split your organization

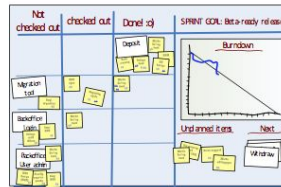
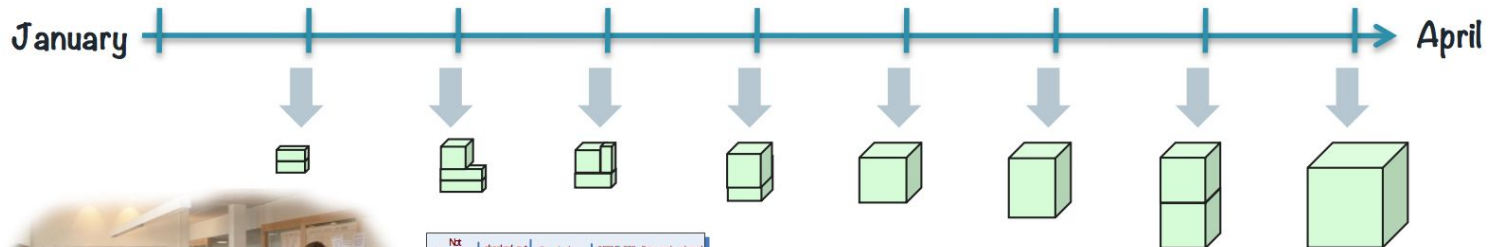


~~Large group~~ spending ~~a long time~~ building a ~~huge thing~~
Small team spending a little time building a small thing
... but integrating regularly to see the whole

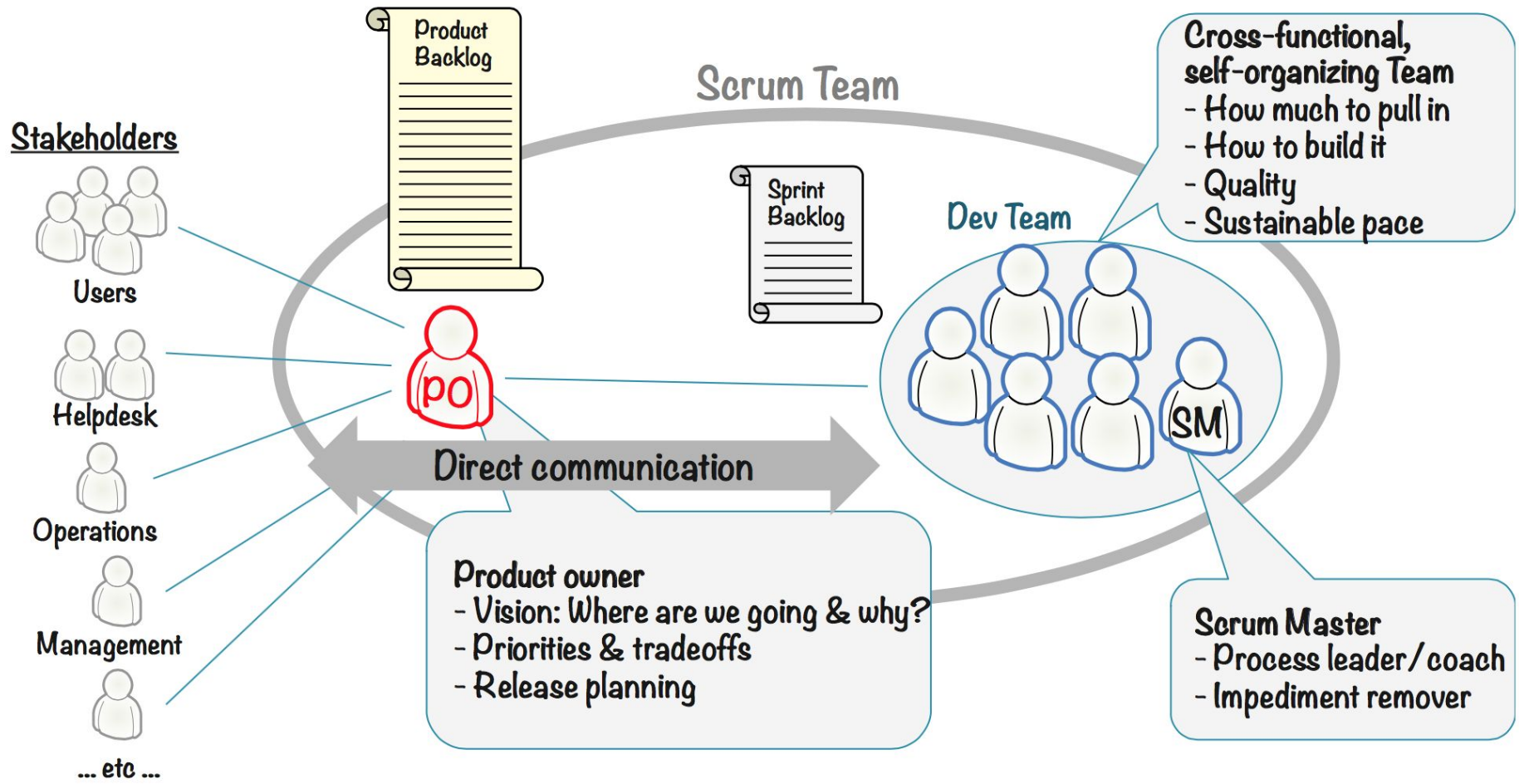
Optimize process



Split time



Scrum overview – structure





BITS Pilani presentation

BITS Pilani
Pilani Campus

Prof. Narasimha Bolloju
CSIS Department, Hyderabad



SS ZG622

Software Project Management

Lecture No. 3

Project goals & objectives and project charter



1. Goals vs objectives
2. Defining project goals and project management goals
3. Mapping goals to SMART objectives
4. Preparing project charter

1. Goals vs objectives



Goals vs. objectives

Aspect	Goals	Objectives
Definition	Broad, general intentions or outcomes to be achieved	Specific, measurable steps/actions to achieve the goals
Nature	Qualitative, high-level	Quantitative, detailed
Time Frame	Long-term (whole project or beyond)	Short- to medium-term
Measurability	Often hard to measure directly	Always measurable
Scope	Strategic — align with vision or business drivers	Tactical — tied to deliverables and milestones
Who Uses It	Sponsors, executives, customers	Project managers, team members
Example	“Enhance customer experience”	“Reduce average online ticket booking time by 30% in 3 months”

Goal and corresponding objectives - an example



Goal - Become fit by healthy eating and regular exercise

Objectives ?



Goal and corresponding objectives - an example



Goal - Become fit by healthy eating and regular exercise

Objectives:

- *Prepare and eat home-cooked meals at least 5 days a week for the next 3 months.*
- *Limit added sugar consumption to no more than 25 grams per day for the next 6 weeks.*
- *Eat at least 3 servings of vegetables daily for 30 consecutive days.*
- *Walk or jog for at least 30 minutes, 5 times per week, for the next 12 weeks.*
- *Do strength training exercises twice a week (Monday and Thursday) for 8 weeks.*

2. Defining project goals and project management goals

Project goals vs. Project management goals



	Project Goals	Project Management Goals
Examples	Improving customer experience, increasing revenue	Completing the project on time, staying within budget
Focus	High-level outcomes the project aims to achieve	Successful execution and delivery of the project
Purpose	Align with broader business objectives	Manage project constraints like time, budget, and scope
Scope	Broad and ongoing, often tied to business performance	Specific to the project lifecycle, time-bound to the project's duration
Metrics	Business performance metrics (e.g., customer satisfaction)	Project-specific metrics (e.g., timeline adherence, risk management)
Time Frame	Can be continuous or long-term	Limited to the duration of the project
Alignment	Tied to the end results or deliverables of the project	Focused on the process of achieving those results

Activity - W03#1



Define three goals

- a) for the Mux Core project and
- b) for its project management

3. Mapping goals to SMART objectives

Project objectives vs

Project management objectives



Project objectives

- define the **desired outcomes or deliverables**, focusing on the **end product or system** and its alignment with business goals.
- describe **what** the project aims to achieve in terms of functionality, performance, and value to the organization.
- centered on the **product** and its impact on stakeholders, such as customers, employees, or the business.

PM objectives

- focus on the **process of delivering the project**, emphasizing how the project is planned, executed, and controlled to ensure successful completion.
- describe **how** the project will be managed to achieve the project objectives.
- centered on the **project execution**, including timelines, budgets, resources, risks, and stakeholder coordination.

SMART is a framework to ensure objectives are clear and actionable



- **Specific:** Clearly defined, answering who, what, where, when, or why.
 - **Measurable:** Quantifiable to track progress and confirm success.
 - **Achievable:** Realistic given resources, constraints, and capabilities.
 - **Relevant:** Aligned with project and organizational priorities.
 - **Time-bound:** Tied to a deadline or timeframe for completion.
-

- ❑ *Prepare and eat home-cooked meals at least 5 days a week for the next 3 months.*
- ❑ *Eat at least 3 servings of vegetables daily for 30 consecutive days.*
- ❑ *Walk or jog for at least 30 minutes, 5 times per week, for the next 12 weeks.*



Example objective for improving customer satisfaction

Objective: "Increase the average customer satisfaction score for the online ticket purchasing process by 15% within the next six months."

- ***Specific:*** Focus on customer satisfaction during the online ticket purchasing process.
- ***Measurable:*** The 15% increase in the satisfaction score.
- ***Achievable:*** Through targeted improvements in user interface and customer support.
- ***Relevant:*** Directly tied to enhancing customer experience.
- ***Time-bound:*** Achieve within six months.

Activity - W03#2



Define SMART objectives corresponding to the given ACL Mux Core Project goals

Activity - W03#3



Define SMART objectives
corresponding to the given ACL Mux Core
project management goals

4. Preparing project charter

Project charter

A foundational document that officially authorizes the existence of a project and gives the project manager the authority to use organizational resources to execute the project.

Key Purpose

- Formally authorizes the project.
- Aligns all stakeholders on the project's objectives, scope, and purpose.
- Establishes the project manager's authority.
- Serves as a reference point throughout the project lifecycle.

Typical contents of project charter

- Project purpose
- Project objectives
- Project management objectives
- Scope (in-scope and out-of-scope)
- Execution approach
- Key deliverables
- Timeline
- Budget
- Stakeholders
- Risks and mitigation
- Assumptions
- Constraints
- Approval



Project charter comparison

Aspect	Plan-Driven (e.g., Waterfall)	Agile (e.g., Scrum)
Purpose	Define entire scope, timeline, and deliverables up front	Align on product vision, goals, and team expectations
Detail Level	High: includes full scope, WBS, budget, Gantt chart, etc.	Lean: high-level goals, constraints, product roadmap
Change Flexibility	Low: changes after charter require formal re-approval	High: changes expected; charter reflects flexible goals
Ownership	Sponsor + PM-driven	Product Owner + Team-driven with PM facilitation
Planning Horizon	Full project duration	Current release or increment (1–3 months)
Use in Execution	Reference for contract, fixed deliverables	Alignment tool for iterations/sprints

Summay



1. Differences between goals and objectives
2. How to define project goals and project management goals
3. Mapping goals to SMART objectives
4. Purpose of creating a project charter

Stepwise project planning (from Textbook #1)



Step 1 - Identify project scope and objectives



- Step 1.1 - Identify objectives and practical effectiveness measures to meet those objectives
- Step 1.2 - Establish project authority
- Step 1.3 - Identify all stakeholders and their interests
- Step 1.4 - Modify objectives in the light of stakeholder analysis
- Step 1.5 - Establish methods of communication with all parties

Step 2 - Identify project infrastructure



- Step 2.1 - Identify relationship between the project and strategic planning
- Step 2.2 - Identify installation standards and procedures
- Step 2.3 - Identify project team organization

Step 3 - Analyse project characteristics



- Step 3.1 - Distinguish the project as either objective-driven or product-driven
- Step 3.2 - Analyse other project characteristics
- Step 3.3 - Identify high-level project risks
- Step 3.4 - Take into account user requirements concerning the implementation
- Step 3.5 - Select development methodology and life-cycle approach
- Step 3.6 - Review overall resource estimates

Step 4 - Identify project products and activities



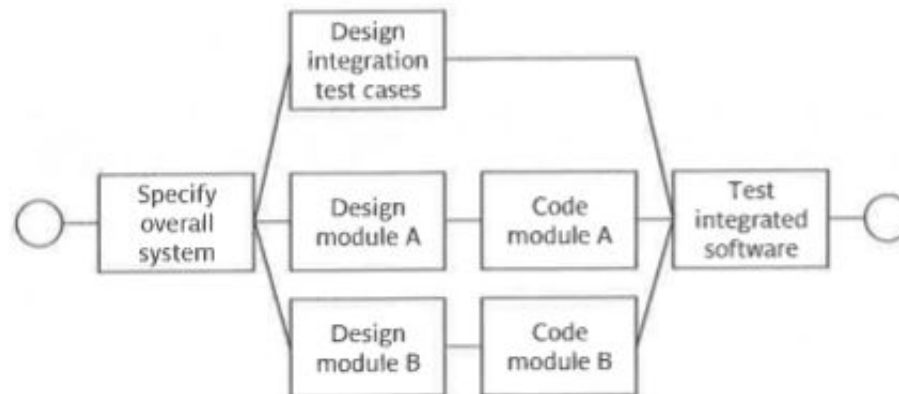
Step 4.1 - Identify and describe project products or deliverables

Step 4.2 - Document generic product flows

Step 4.3 - Recognize product instances

Step 4.4 - Produce ideal activity network

Step 4.5 - Modify the ideal to take into account need for stages and checkpoints



Software project management (6e) - introduction © The McGraw-Hill Companies, 2018

Step 5 - Estimate effort for each activity



Step 5.1 - Carry out bottom-up estimates

Step 5.2 - Revise plan to create controllable activities

Step 6 - Identify activity risks

Step 6.1 - Identify and quantify activity-based risks

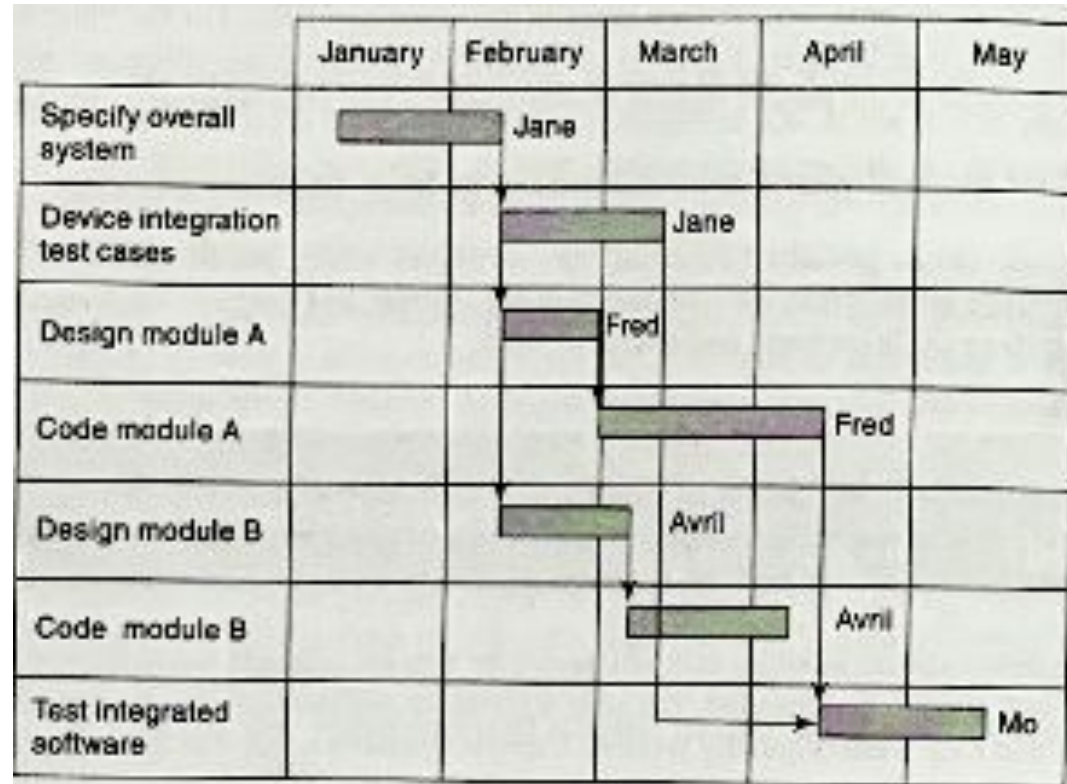
Step 6.2 - Plan risk reduction and contingency
measures where appropriate

Step 6.3 - Adjust overall plans and estimates to take
account of risks

Step 7 - Allocate resources

Step 7.1 - Identify and allocate resources

Step 7.2 - Revise plans and estimates to take into account resource constraints





Step 8 - Review/publicize plans

Step 8.1 - Review quality aspects of the project plan

Step 8.2 - Document plans and obtain agreements

Steps 9 and 10 - Execute plan and perform lower level planning



BITS Pilani presentation

BITS Pilani
Pilani Campus

Prof. Narasimha Bolloju
CSIS Department, Hyderabad



BITS Pilani
Pilani Campus



SS ZG622

Software Project Management

Lecture No. 4

Recap

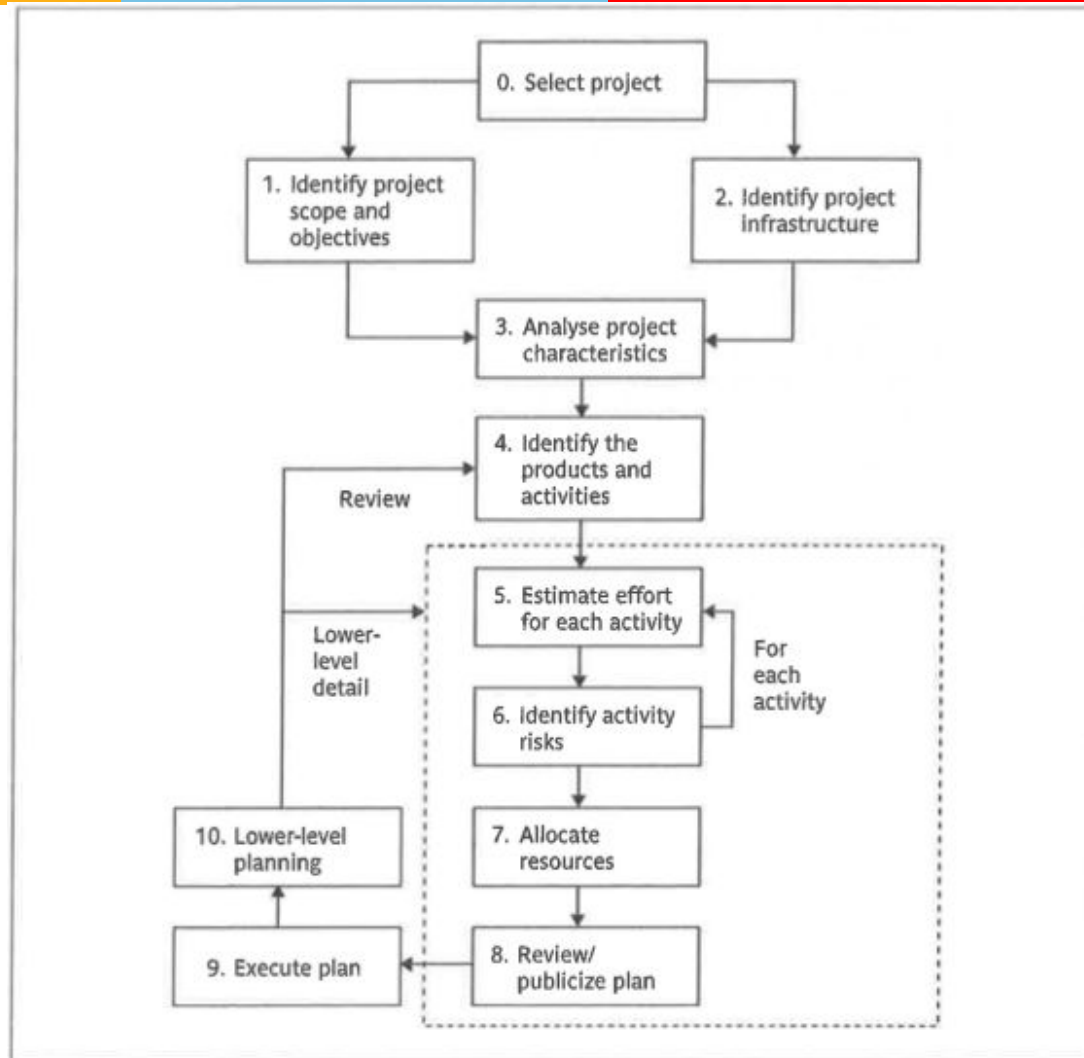


- Challenges in software project management (20% failure; 30% success and 50% challenged)
- Need for understanding project and project management goals and objectives before planning and execution
- Plan-driven vs value- driven projects
- Project charter and its importance

Recap



Challenge	Impact	Within PM's Control?
Unclear or Changing Requirements	Scope creep, delays, misaligned deliverables	Partially
Inaccurate Effort/Cost Estimation	Budget overruns, missed deadlines	Yes
Poor Stakeholder Engagement	Misaligned expectations, lack of buy-in	Yes
Scope Creep	Increased costs, delayed timelines	Partially
Inadequate Risk Management	Unforeseen issues, project failures	Yes
Team Collaboration Issues	Reduced productivity, conflicts	Yes
Resource Constraints	Delays, compromised quality	Partially
Schedule Delays	Missed deadlines, increased costs	Partially
Technical Debt Accumulation	Long-term maintenance issues, rework	Yes
Inadequate Quality Assurance	Defective deliverables, customer dissatisfaction	Yes



Product & Process Measures and Effort estimation



1. Product/process and direct/indirect measures
2. Size- and function-oriented metrics
3. Effort estimation approaches
4. Effort estimation using function points

Product and Process Measures



	Direct Measures	Indirect Measures
Product Metrics	Lines of Code (LOC or KLOC): A measure of the size of the software by counting the number of lines in the source code. Cyclomatic Complexity: A measure of the complexity of a program by calculating the number of linearly independent paths through the source code. Defect Density: Number of defects per unit of code (e.g., bugs reported per 1,000 lines of code).	Customer Satisfaction: Often measured through surveys or Net Promoter Score (NPS), reflecting user satisfaction with the product. Maintainability Index: A measure that combines several factors (e.g., LOC, complexity) to predict the ease of future maintenance.
Project Metrics	Earned value (EV) with schedule variance and cost variance. Cycle Time: The time taken to complete a specific process or task from start to finish.	Developer Productivity: Often inferred from metrics like LOC per day or features completed per sprint, indirectly measuring efficiency. Risk Exposure: A qualitative measure of potential risks that could impact the project, often assessed through expert judgment or risk assessment tools.

Direct vs Indirect measures



- Direct Measures are quantifiable and typically involve straightforward data collection. They focus on measurable aspects like code, time, or cost.
- Indirect Measures are qualitative/quantitative and involve interpretation or inference from other measures. They focus on outcomes or effects that are influenced by multiple factors.

Activity W04#1



Connecting goals, objectives, metrics, and
functional specifications
(enter your answers in [this document](#))

Size-oriented metrics



- Metrics are derived by normalizing quality and/or productivity measures by considering the size of the software produced in KLOC (thousand lines of code)
- Example metrics include
 - Errors per KLOC
 - Defects per KLOC
 - Cost per KLOC
 - Errors per person-month
 - KLOC per person-month
 - Cost per page of documentation
- KLOC measurements
 - Are dependent on the programming language
 - Penalize well-designed but short programs
 - Cannot easily accommodate nonprocedural languages
 - Require a level of detail that may be difficult to achieve



Function-oriented metrics

- Function-oriented metrics (e.g., function points FP) use a measure of the functionality delivered by the application as a normalized value
 - Errors per FP
 - Defects per FP
 - Cost per FP
 - Pages of documentation per FP
 - FP per person-month

Use of Metrics

- Product and project metrics are quantitative measures to gain insight into the efficiency of the software process and the projects
- Project metrics enable project manager to
 - Assess status of ongoing project
 - Track potential risks
 - Uncover problem are before they go critical
 - Adjust work flow or tasks
 - Evaluate the project team's ability to control quality of software work products

Activity W04#2



Read [these functional specification](#) and
give a rough estimate of effort [in person-months](#)
for developing the **Mux-Core sales system**

Effort estimation - approaches



- Expert judgment
- Analogous estimation
- Parametric estimation
- Bottom-up estimation
- Top-down estimation
- Wideband Delphi technique
- **Function points**
- Use-case points
- *COCOMO (Constructive Cost Model)*
- *Story points (Planning Poker)*



Need for multiple approaches

- In case of large projects, we cannot depend on one approach to estimation, because there could be some aspects which might have been overlooked
- Better to use multiple estimation techniques and compare the results
- If the difference is large then it is likely that some aspects have been overlooked
- Understand the reason for the difference and re-estimate until results converge (say within 10%)

Activity W04#3



Now, estimate the effort required for each function listed in [this Google Sheet](#) and then give your total effort estimate

Function point (FP) technique



- Each requirement (e.g., selling a ticket at counter) is mapped to several functions and logical data entities/files
- The calculation of size is based on functions supported by the software and the data files used by the software
- Functions are of 3 types
 - External inputs
 - External outputs
 - External queries
- Data files are of 2 types
 - Internal logical files
 - External logical files

Types of Functions

- **External Inputs (EI):** functions that modify the data of the system.
 - Examples: Create Purchase order, Withdraw money from ATM, Update price of a product
- **External Outputs (EO):** functions that send data to external systems or do some processing of data and provide to end users.
 - Examples: Exporting data to other systems via files or messages, Generating reports involving analysis such as average
- **External Queries (EQ):** inquiry functions that extract data from the system and present to user.
 - Examples: Show availability of seats in a flight, generating a list of all orders to be shipped tomorrow

Types of Data

- Internal Logical Files: files (database entities/tables or flat files) maintained by the system and that reside within the system.
 - Example: Employee file, Orders file, Payment file, Inventory issue file
- External Interface Files: data files (database tables or flat files) maintained outside the system but needed by the system under development.
 - Example: Schedule of flights of other airlines, Interest rate offered by different banks

Functions Points

Steps to calculate Function points

1. Assign FPs to each Function & each File based on its complexity
2. Total the FPs. This is called Unadjusted FP or Raw FPs
3. Multiply the Unadjusted FPs by an adjustment factor. The adjustment factor is determined / based on 14 characteristics of the system

FPs for Functions & Files

Different types of functions and files, and their complexity for a given requirement to be implemented

Program Characteristic	Low Complexity	Medium Complexity	High Complexity
External Inputs	___ × 3	___ × 4	___ × 6
External Outputs	___ × 4	___ × 5	___ × 7
External Queries	___ × 3	___ × 4	___ × 6
Internal Logical Files	___ × 4	___ × 10	___ × 15
External Interface Files	___ × 5	___ × 7	___ × 10
Unadjusted Function Point (UPFs) = Sum of EI FP, EO FP, EQ FP, ILF FP, EIF FP			

Function point (FP) estimation technique



- FP technique is used in the early stages of the project to determine the size of the project based on software requirements and refined in later stages based on the design details
- The size of software is measured in Function Points
- Once the size is estimated in FPs, we can convert FPs to Effort, based on historical data if available (otherwise, established productivity data can be used)

Function point estimation



Function Count					
Item	Item Description	Complexity	Count	Weight	Weighted
					Count
1	External Inputs	Simple	0	3	0
		Average	0	4	0
		Complex	0	6	0
2	External Outputs	Simple	0	4	0
		Average	0	5	0
		Complex	0	7	0
3	External Inquiries	Simple	0	3	0
		Average	0	4	0
		Complex	0	6	0
4	Internal Logical Files	Simple	0	7	0
		Average	0	10	0
		Complex	0	15	0
5	External Interface Files	Simple	0	5	0
		Average	0	7	0
		Complex	0	10	0
Total Weighted Function Count (FC)					0

Function point estimation

Complexity Factor		
Factor	Description	Rating
		0=Irrelevant
		5=Essential
F1	Reliability and backup recovery	
F2	Data communications	
F3	Distributed processing	
F4	Performance	
F5	Operate on existing system	
F6	On-line data entry	
F7	Data entry over multiple screens	
F8	Master files updated on-line	
F9	Complex inputs, outputs, files & inquiries	
F10	Complex internal processing	
F11	Code needs to be reusable	
F12	Need conversion and installation	
F13	Multiple installations of the system	
F14	Easy to change and use	
Complexity Factor (CF) = sum of ratings		0
Function Points		
Function Points (FP) = FC x (0.65 + 0.01 x CF)		0

Estimate the effort using function point method for these specifications of Mux-Core system

Instructions:

- Make a copy of this document “[estimating function points](#)”
- Estimate function points for F2 to F6 (for F1 it is already estimated)

FP productivity based on programming language (only an example)



Language	Hours per FP	FP per month
ASP*	06.1	28
Visual Basic	08.5	20
Java	10.6	16
SQL	10.8	16
C++	12.4	14
C	13.0	13
PL/1	14.2	12
C#	15.5	11
COBOL	16.8	10
ABAP	19.9	9

http://www.webratio.com/website/documentation/Case_Study_Productivity_with_WebRatio.pdf

Additional slides on effort estimation

Example of historical data



Table 7-1 Examples of Quantities That Can Be Counted for Estimation Purposes

Quantity to Count	Historical Data Needed to Convert the Count to an Estimate
Features	■ Average effort hours per feature for development and/or testing
Use cases	■ Average total effort hours per use case ■ Average number of use cases that can be delivered in a particular amount of calendar time
Stories	■ Average total effort hours per story ■ Average number of stories that can be delivered in a particular amount of calendar time
Engineering requirements	■ Average number of engineering requirements that can be formally inspected per hour ■ Average effort hours per requirement for development/test/documentation
Function Points	■ Average development/test/documentation effort per Function Point ■ Average lines of code in the target language per Function Point
Change requests	■ Average development/test/documentation effort per change request (depending on variability of the change requests, the data might be decomposed into average effort per small, medium, and large change request)
Web pages	■ Average effort per Web page for user interface work ■ Average whole-project effort per Web page (less reliable, but can be an interesting data point)

Data collection

- Collect effort for all activities and all items (screen, report, etc.) including
 - Requirement gathering
 - Architecture
 - Detailed design
 - Coding & Unit testing
 - Design of test cases
 - Integration testing
 - System testing
 - User acceptance testing
 - Project Management
 - User documentation
 - Reviews of requirement, design, code, test cases, user manual, etc.
- Collect data while project is underway. It is difficult to go back six months in the project to try to collect data
- If you do not have historical data, use data from current project, as a basis to estimate the remainder of the project

Calibration



- Based on data collected, we can calibrate our productivity, ie. Effort for unit size
- Calibration examples
 - Our team takes 8 hours for design & 24 hours to code one screen
 - Our testers take 6 hours to create a test case
 - Our maintenance team takes 4 hours to fix a defect

Factors influencing effort estimation



1. Project size:

- Project size has the biggest influence on effort estimation
- Effort increases exponentially (not linearly) as project size increases because there is much more coordination in larger projects (this is opposite of 'Economies of scale' concept)

2. Kind of software being developed

- Avionics, embedded systems and System software are much more complex than business systems and internet systems

3. Personal factors impact estimation

- Capability of requirements analyst, capability of programmer, change of team members, level of domain knowledge, platform experience, team cohesion

Why Opt for FP?

- Used readily countable characteristics that are determined early in the software process
- Does not “penalize” inventive (short) implementations that use fewer LOC than other more clumsy versions
- Makes it easier to measure the impact of reusable components



Function Point Controversy

- Proponents claim that
 - FP is programming language independent
 - FP is based on data that are more likely to be known in the early stages of a project, making it more attractive as an estimation approach
- Opponents claim that
 - FP requires some “sleight of hand” because the computation is based on subjective data
 - FP has no direct physical meaning...it’s just a number

Determining complexity of EI, EO & EQ



- Complexity of functions is determined based on number of Data Element Types referenced and File Types referenced, by the function
 - Example #1: The function “Create purchase order” (EI), updates the fields – PO#, Date, Description, PO amount, Customer ID, Customer name, etc. These are the DETs maintained by the function
 - Example #2: The report “Generate Sales analysis report” (EO) displays the following fields on the report – Product ID, Product name, City, Month, Total quantity sold, Total Sales revenue, Name of Salesman. These fields are the DETs of this function. In order to perform this function, it uses 3 files namely – Sales file, Product file and Salesman file. These are the File Types referenced

Determining complexity of EI, EO & EQ



- Here is the matrix to determine the complexity

Data Element Types (DETs) – user-recognizable data elements that are maintained as ILFs by the EIs, appear in the EOs, EQs (data fields, buttons, search).

File Types Referenced (FTRs) – all ILFs and EIFs referenced or maintained during the processing of the EIs/EOs/EQs (tables, flat files).

EI	1-4 DETs	5-15 DETs	16 + DETs
0-1 FTR	Low	Low	Average
2 FTRs	Low	Average	Average
3+ FTRs	Average	High	High

EO/EQ	1-5 DETs	6-19 DETs	19+ DETs
0-1 FTR	Low	Low	Average
2-3 FTRs	Low	Average	High
4+ FTRs	Average	High	High

Determining complexity of files



- Complexity of files is determined based on 2 characteristics of the file namely - Data Element Types (DET) and Record Element Types (RET)
- The meaning of DET & RET is explained below:
- Example:
 - Employee file can have the following logical groupings
 - Basic data of employee such as Name, Date of birth, Address, etc.
 - Past employment info such as Name of company, Start date, end date, Last designation, etc.
 - Past project info such as Name of project, From date, To date, Role in project, etc.
 - In the above example,
 - the DETs are Name, date of birth, Address, Name of company, From date, etc.
 - We have 3 types of logical grouping of data namely - Basic data group, Past employment info group, Past project info group. Hence we have 3 RETs

Determining complexity of LIF & EIF



- Here is the matrix to determine the complexity

Data Element Types (DETs) – Unique, user-recognizable, non-repeating fields or attributes, including foreign key attributes that enter the boundary of the subsystem or application (fields).

Record Element Types (RETs) – Logical sub-groupings based on the user's view of the data (0 RET = 1 RET).

ILF/EIF	1-19 DETs	20-50 DETs	51 + DETs
1 RET	Low	Low	Average
2-5 RETs	Low	Average	High
6+ RETs	Average	High	High

Adjustment factor

- Adjustment factor depends on the rating of 14 system characteristics
- We need to rate each characteristic on a scale of 0-5
- The adjustment factor is calculated using a formula shown below

F1	Reliable back-up and recovery	0 – 5
F2	Data communications	0 – 5
F3	Distributed functions	0 – 5
F4	Performance	0 – 5
F5	Heavily used configuration	0 – 5
F6	Online data entry	0 – 5
F7	Operational ease	0 – 5
F8	Online update	0 – 5
F9	Complex interface	0 – 5
F10	Complex processing	0 – 5
F11	Reusability	0 – 5
F12	Installation ease	0 – 5
F13	Multiple sites	0 – 5
F14	Facilitate change	0 – 5
Value adjustment factor (VAF) = $0.65 + 0.01 * \text{Sum}(F1, F14)$		
Adjusted function point (AFP) = $\text{UFPs} * \text{VAF}$		

Object based estimation

- This is a flexible estimation approach that can be adapted to different types of projects
- For example, some projects have screens & reports as objects. Others have configurations settings as objects (ex, ERP package implementation), others have change requests as objects
- This technique determines the size of project based on number of objects and their complexity
- Please note that this is not the same as Object Point technique



Object based estimation

Steps:

1. List all objects of different categories
2. Assign points to each object based on its complexity (Low, Medium, High)
3. Determine the total points
4. Finally convert the Points to effort based on Calibrated Historical data

Note: The classification of objects and the way to determine their complexity needs to be done by each organization depending on the nature of projects executed

Assigning points, based on complexity of objects: Example



Point based on complexity

Object type	Complexity ->	Low	Medium	High
Screen		2	4	8
Report		1	2	3
Interface		2	4	8

Points will differ from organization to organization. It has to be based on historical data of effort needed to develop & test these objects



Object based estimation: Example

Screens	Name of screen	Complexity	Points
	Account maintenance	High	8
	Funds transfer	Medium	4
	Pay bills	Low	2
Reports	Name of report	Complexity	Points
	Generate monthly statement	High	3
	Print last 10 transactions	Medium	2
Interfaces	Name of interface	Complexity	Points
	Send statement via email	Low	2
	Receive government rules	Medium	4
Total Object points			25

Parameters that determine complexity of objects: Example



- Given below are some parameters that can be used to determine complexity of objects:
 - Screen complexity: can be determined based on number of fields on the screen, number of command buttons,
 - Report complexity: can be determined based on number of fields in the report, number of data files needed
 - Interface complexity: can be determined by the number of data items sent or received, number of data files needed

Determining complexity of objects: Example



Screen complexity determination: Example

		Number of fields		
		<5	6 - 10	> 10
Number of command buttons	1	Simple	Simple	Medium
	2 - 3	Simple	Medium	Medium
	> 3	Simple	Complex	Complex

Summary



- Object based estimation is a flexible approach that can be adapted to different types of projects
- Projects need to identify suitable types of objects based on the type of project
- The size of projects are estimated based on number of objects and complexity of objects



Estimating effort for different phases

- Once the size of the project is estimated using FPs or Object points, we can convert the size into effort, based on calibrated historical data
- We can then distribute the total effort across different phases of the project, again based on historical data
- For example:
 - 7% for Requirement gathering
 - 8% for Requirement analysis
 - 20% for architecture & design
 - 45% for construction
 - 15% for System testing
 - 5% for Project management

Estimating effort: Other considerations

There may be other factors to consider:

- Configuration management: 2-5% of technical effort
- Requirement change: 1-4% effort per elapse month
- First time development with new language & tool: 20- 40% effort
- Contingency depending on risk severity & probability: 5-10%

Note: All percentages are illustrative only. Actual percentages should be based on historical data



Estimation negotiation techniques

Project managers encounter situations where stakeholders ask for changes to estimation. Here are some ways to handle them:

1. If Marketing team tells you to reduce the cost or effort
 - Use historical data to justify the effort calculations
2. If more features are requested without adding more effort
 - Move some functions to version 2, or
 - Cut features altogether, if the business value is low
3. When you are asked to complete the project faster
 - Ask for more developers & testers & team leads, or
 - Add higher skilled technical staff, or
 - Ask what features are a 'must have' & what are not required



BITS Pilani presentation

BITS Pilani
Pilani Campus

Prof. Narasimha Bolloju
CSIS Department, Hyderabad



SS ZG622

Software Project Management

Lecture No. 5

Recap - Function point analysis



- [Suggested solution](#)
- A realistic estimate that includes all activities
- Possible repetition of certain points
- Detailed FPA estimation once the design is completed

Planning in agile projects



- Overview of agile project planning
- Product backlog (evolving system scope)
- Prioritizing and sprint backlog
- Estimating story points
- Burndown charts for project management
- Decomposition of complex user stories

Overview of Agile Projects

Planning



- Iterative, adaptive and collaborative process
- Spread throughout the project life cycle
- Product vision and roadmap
- Release planning and sprint planning
- Planning during sprint review and retrospective

Activity W05#1



Write user stories for the Mux Core Sales system
(you may refer to [these functional specifications](#))

Please enter your set of user stories [here](#)
(link shared in Chat panel)

Activity W04#2



Prioritize the given user stories by
changing the order of the stories
(high value ones at the top)



<https://pollev.com/narsibolloju019>

Scrum - estimation



- Traditional software teams give estimates in a time format: days, weeks, months.
- Many agile teams, however, have transitioned to story points. Story points rate the relative effort of work using
 - Fibonacci-like format: 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...
 - Powers of 2: 1, 2, 4, 8, 16, 32, 64, ...
 - T-shirt sizes: XXS, XS, S, M, L, XL, XXL
 - ...



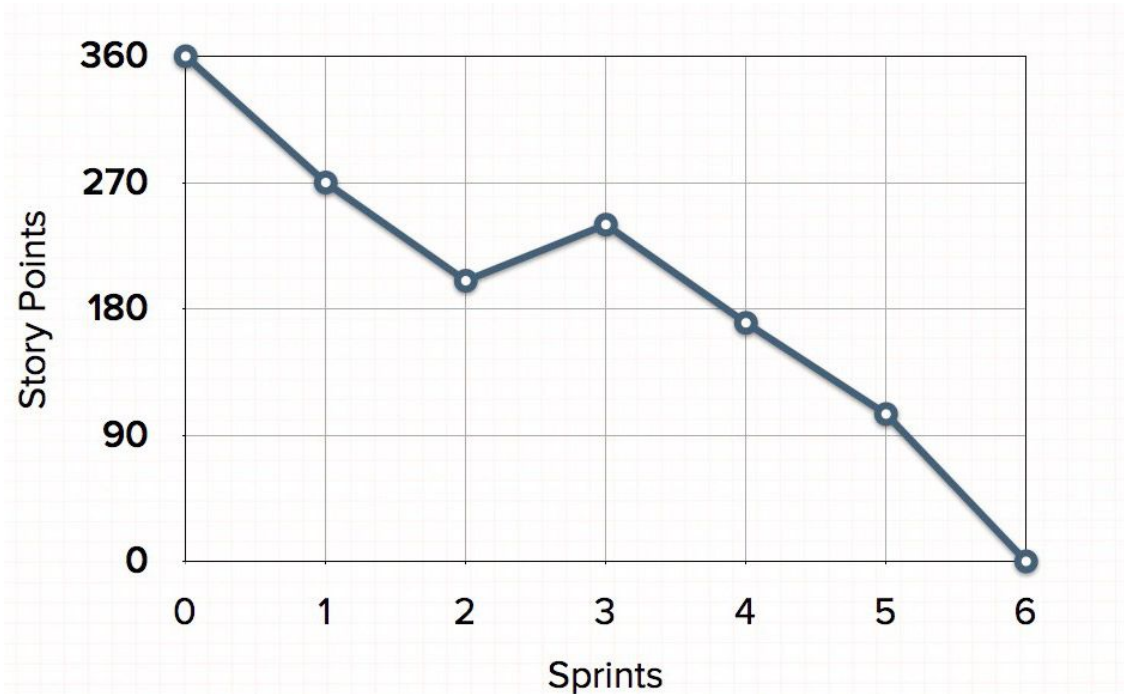
Planning Poker – estimation technique



- All participants use numbered playing cards and estimate the items
- Individual estimates are done and discussion is raised when there are large differences
- This process is repeated till the whole team reached consensus about the agreed estimation (which is not an average of estimates!)

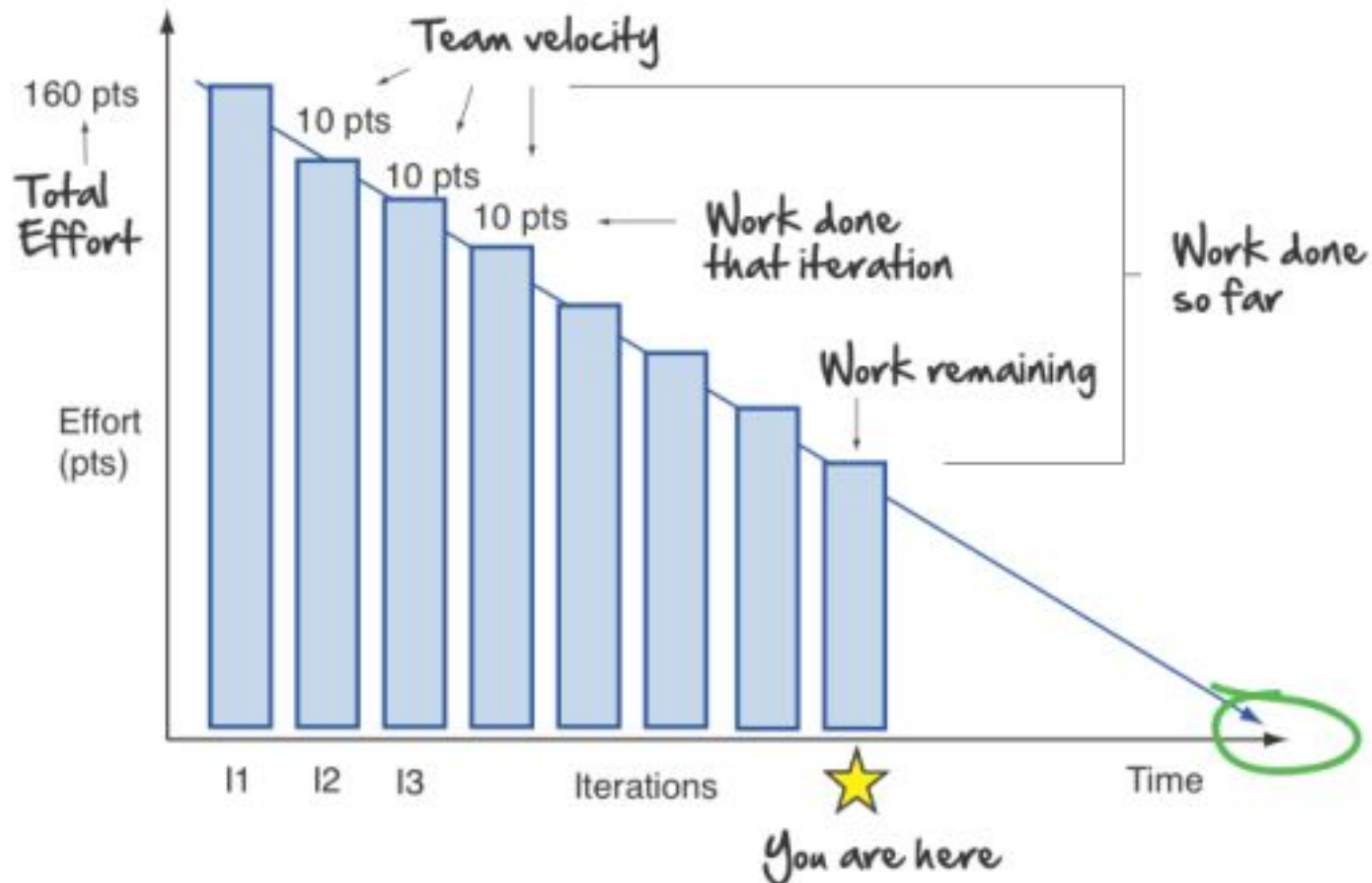
<https://technology.amis.nl/2016/03/23/8-agile-estimation-techniques-beyond-planning-poker/>

Burndown Chart - example



*

Burndown Charts



Activity W05#3



Decompose the user story
“sell movie tickets to customers”
into several simpler user stories

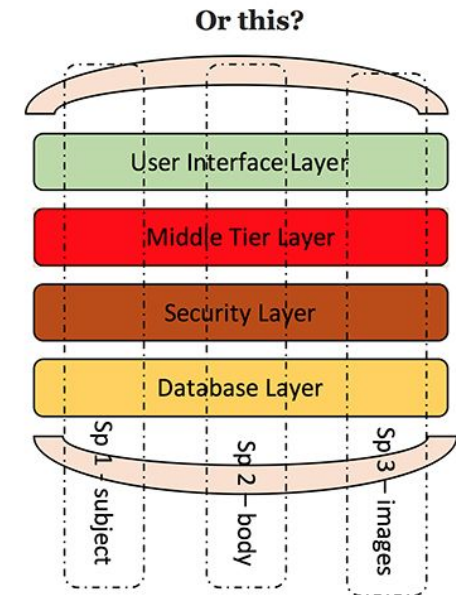
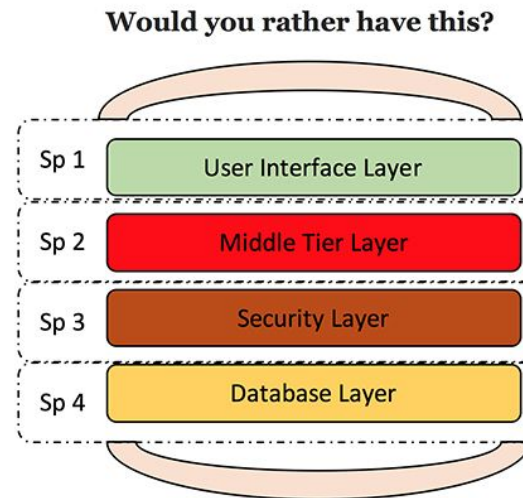
User story decomposition - horizontal vs. vertical slicing



<https://gojko.net/2012/01/23/splitting-user-stories-the-hamburger-method/>

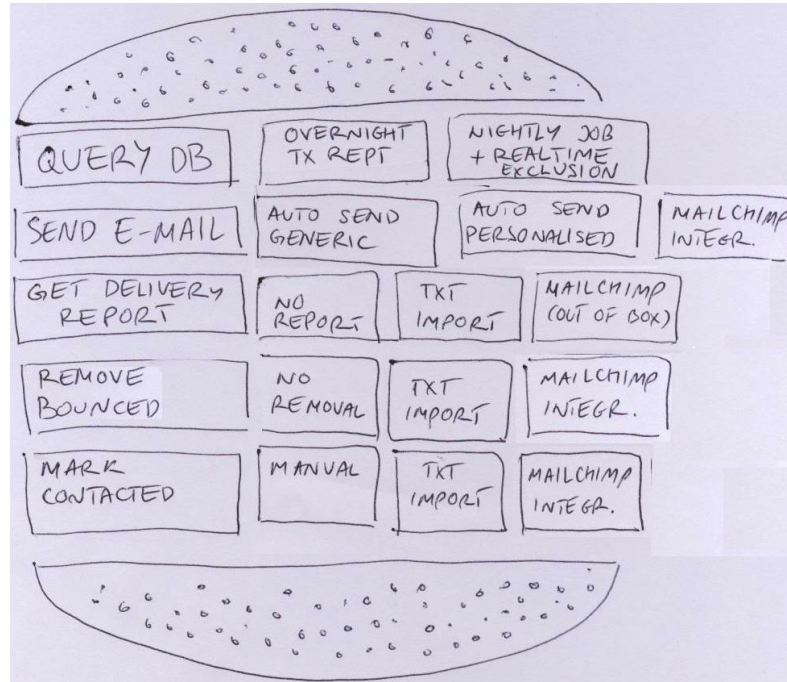
Splitting user stories - vertical vs. horizontal slicing

- Vertical slicing for delivering functional piece of the story that delivers value to the end user
- Horizontal slicing breaks a complex story into smaller parts (but do not deliver value to the end user until all the parts are completed)



<https://www.apple-brook.com/vertically-slicing-taking-a-bite-of-the-hamburger/>

User story decomposition - vertical slicing



<https://gojko.net/2012/01/23/splitting-user-stories-the-hamburger-method/>

Splitting user stories - The hamburger method



1. Identify tasks
2. Identify options for tasks
3. Combine results
4. Trim the hamburger
5. Take the first bite
6. Take another bite ... and continue

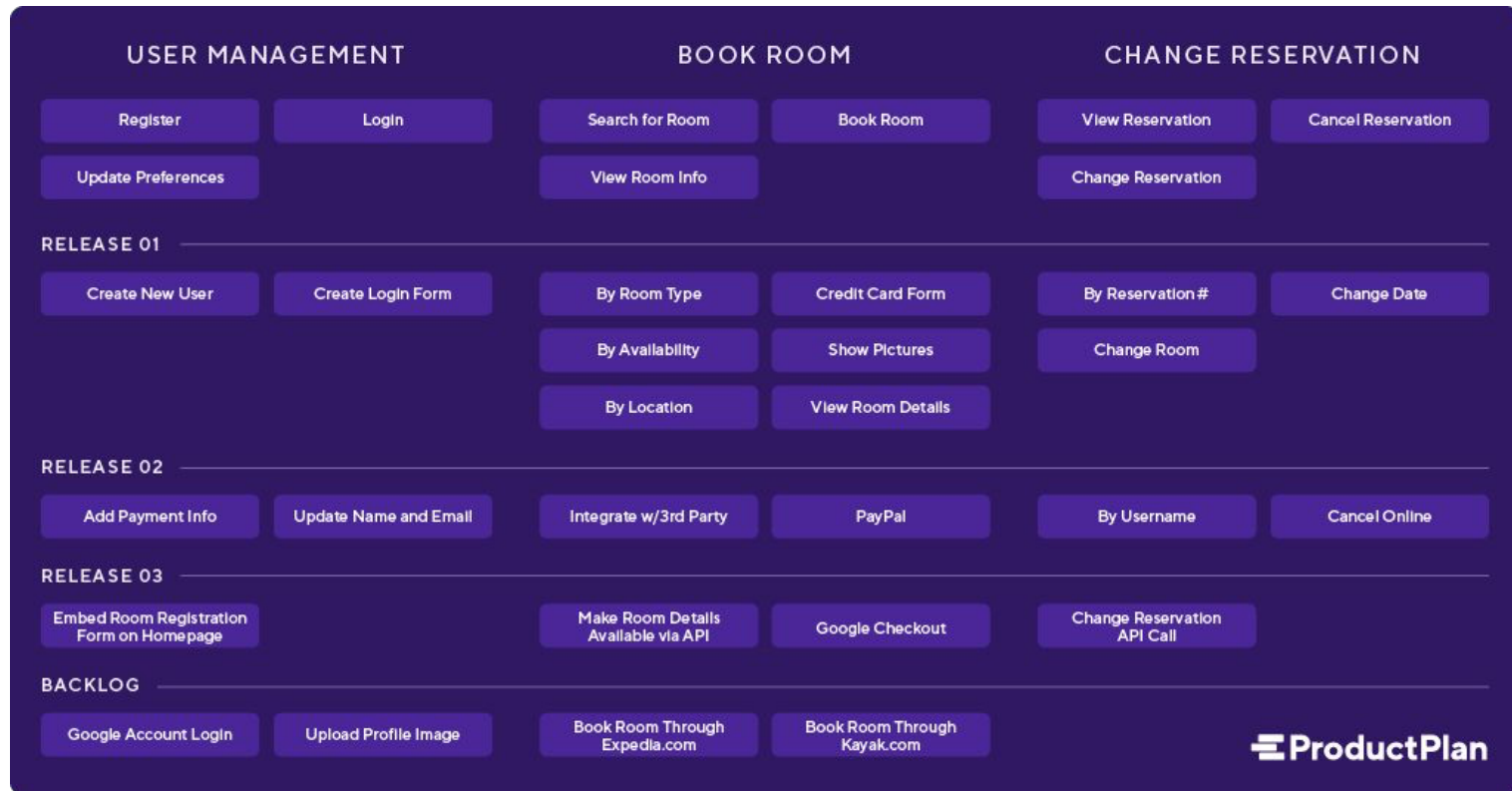
<https://gojko.net/2012/01/23/splitting-user-stories-the-hamburger-method/>

Activity W05#4



Decompose the user story
“sell movie tickets to customers”
into several simpler user stories
using the Hamburger method

Story mapping technique



Activity W05#5



Compare and contrast project planning between
plan-driven and agile models

Additional slides



Definition of Ready (for a user story)

- The story is defined
- The story dependencies are identified
- The story is sized by the team
- Team accepts user experience artefacts
- The story has a **well-defined acceptance criteria** (including performance criteria where appropriate)
- Team has a good idea what it will mean to demo the story
- Person who will accept the story is identified

<https://www.quickscrum.com/ScrumGuide/169/sg-Definition-Of-Ready>

A Good User Story Is

Independent (of all others)

Negotiable (not a contract for a specific feature)

Valuable (a vertical slice)

Estimable (to a decent approximation)

Small (enough to fit in a single iteration)

Testable (even if the test doesn't exist yet)

<https://www.knowledgehut.com/tutorials/scrum-tutorial/definition-of-ready>

Acceptance Criteria

- Acceptance criteria are a set of statements, each with a clear pass/fail result, that specify both functional and non-functional requirements.
- These requirements represent “conditions of satisfaction.”
- There is no partial acceptance: **either a criterion is met or it is not.**

<http://www.seguetech.com/what-characteristics-make-good-agile-acceptance-criteria/>

Scenario: Handling loyalty card during sale

Given a customer presents a loyalty card and the staff scans/verifies it in the system

When the staff applies any discounts or points redemption to the ticket price and completes the purchase

Then the system updates the loyalty points balance, reflects the discounted total on the ticket, and sends a confirmation SMS/email if opted in.

GWT specification of acceptance criteria (Format 1) example



Scenario	Given	When	Then
User wants to log in to his/her registered account	User is on the login page	User fills in correct username AND fills in correct password AND clicks the login button	System presents the dashboard page
	User is on the login page	User fills in correct username AND fills in wrong password AND clicks the login button	System informs the user that the password is wrong
	User is on the login page	User clicks on forgot password link	System displays password change request page

GWT specification of acceptance criteria (Format 2) example



Scenario	Given	When	Then
User wants to reset password of his/her registered account	User submits forgot password form with an email address	The email address is invalid	System informs that email address is not valid
	User submits forgot password form with an email address	The email address is valid AND the email address is not registered	System informs that email address is not registered
	User submits forgot password form with an email address	The email address is valid AND the email address is registered	System sends an email to that address with instructions for resetting the password or disregarding the email message



BITS Pilani presentation

BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Prof. Narasimha Bolloju
CSIS Department, Hyderabad



SS ZG622

Software Project Management

Lecture No. 6

Recap - Effort estimation



- Function point estimation in plan-driven process models
- Story points estimation in agile process models
- Horizontal vs vertical slicing of stories
- Differences between estimation approaches

Project Scheduling - Topics



- Plan-driven methods
 - Mapping effort to duration
 - Examples of effort and time distribution
 - Work breakdown structure (WBS)
 - Gantt charts
 - AoN Diagrams and critical path
- Agile methods
 - Example of effort distribution in agile methods
 - Example planning tool - Pivotal Tracker (user stories, bugs, chores and milestones)

Scheduling in plan-driven methods



- Defining project tasks and milestones
- Mapping estimated effort to duration
- Identifying dependencies
- Developing a work breakdown structure (WBS)
- Creating gantt charts or AoN diagrams
- Finding critical path(s)
- *Monitoring and updating schedule* (as part of execution phase)

Mapping estimated effort to duration



- Effort in function points or KLOC
- Effort to duration
 - Experience-based and heuristics-based methods
 - Parametric methods like COCOMO81 and COCOMO II

W06-#1 Suggested project duration



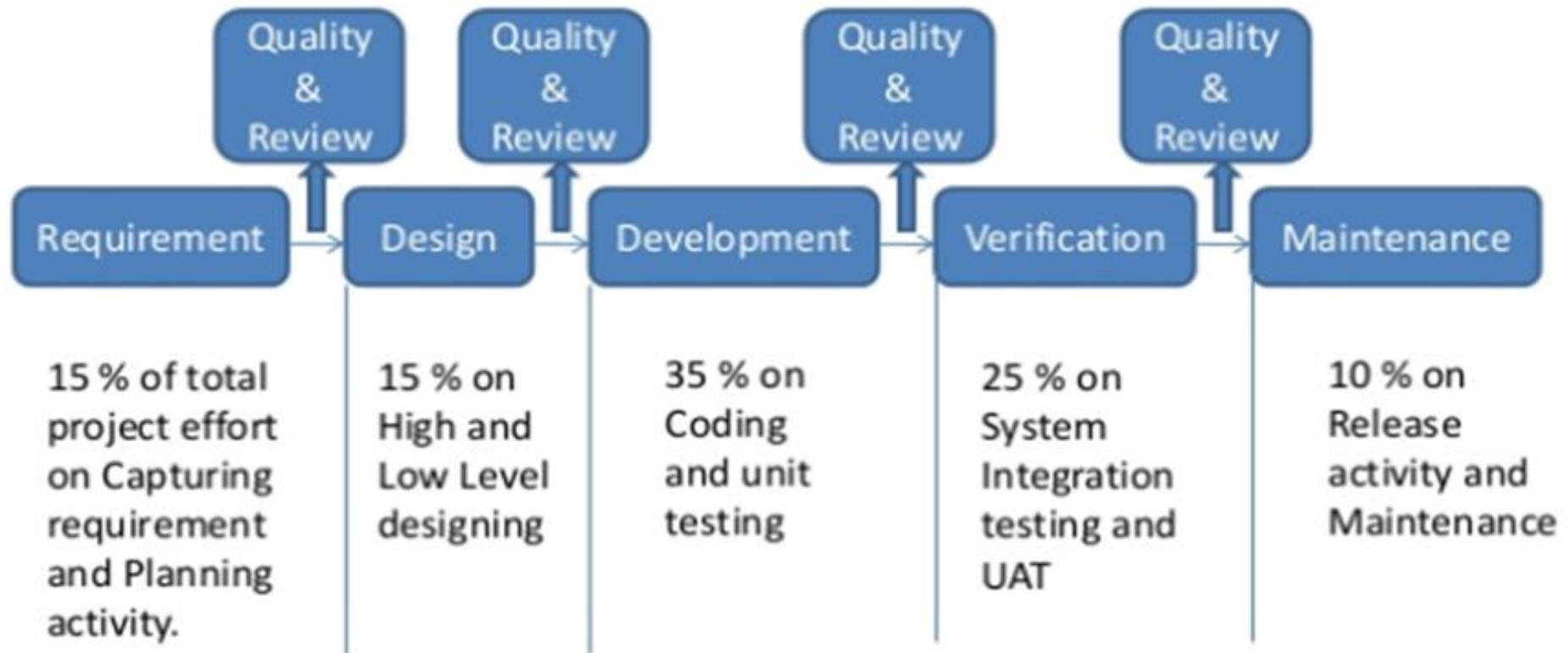
Assuming that the effort and productivity for the Mux-core project are as follows:

- Function points ~ 500
- Average productivity 10 function points per month per developer
- # of developers available is flexible (between 1 to 20)

What should be the ideal project duration?

<https://www.ifpug.org/wp-content/uploads/2017/04/IYSM.-Thirty-years-of-IFPUG.-Software-Economics-and-Function-Point-Metrics-Capers-Jones.pdf>

Example of effort distribution



<https://www.slideshare.net/brickedestimation/effort-distributiononwaterfallandagile>

Example of time distribution



Phase	Tasks	% of Total Time
Requirements Gathering & Analysis	Requirements documentation, stakeholder meetings	10%
System Design	High-level design, detailed design	15%
Implementation (Coding)	Module coding, unit testing	35%
Integration & Testing	System testing, bug fixing	25%
Deployment	Deployment setup, production roll-out	10%
Buffer/Contingency	Risk mitigation, buffer for delays	5%

COCOMO formula for duration



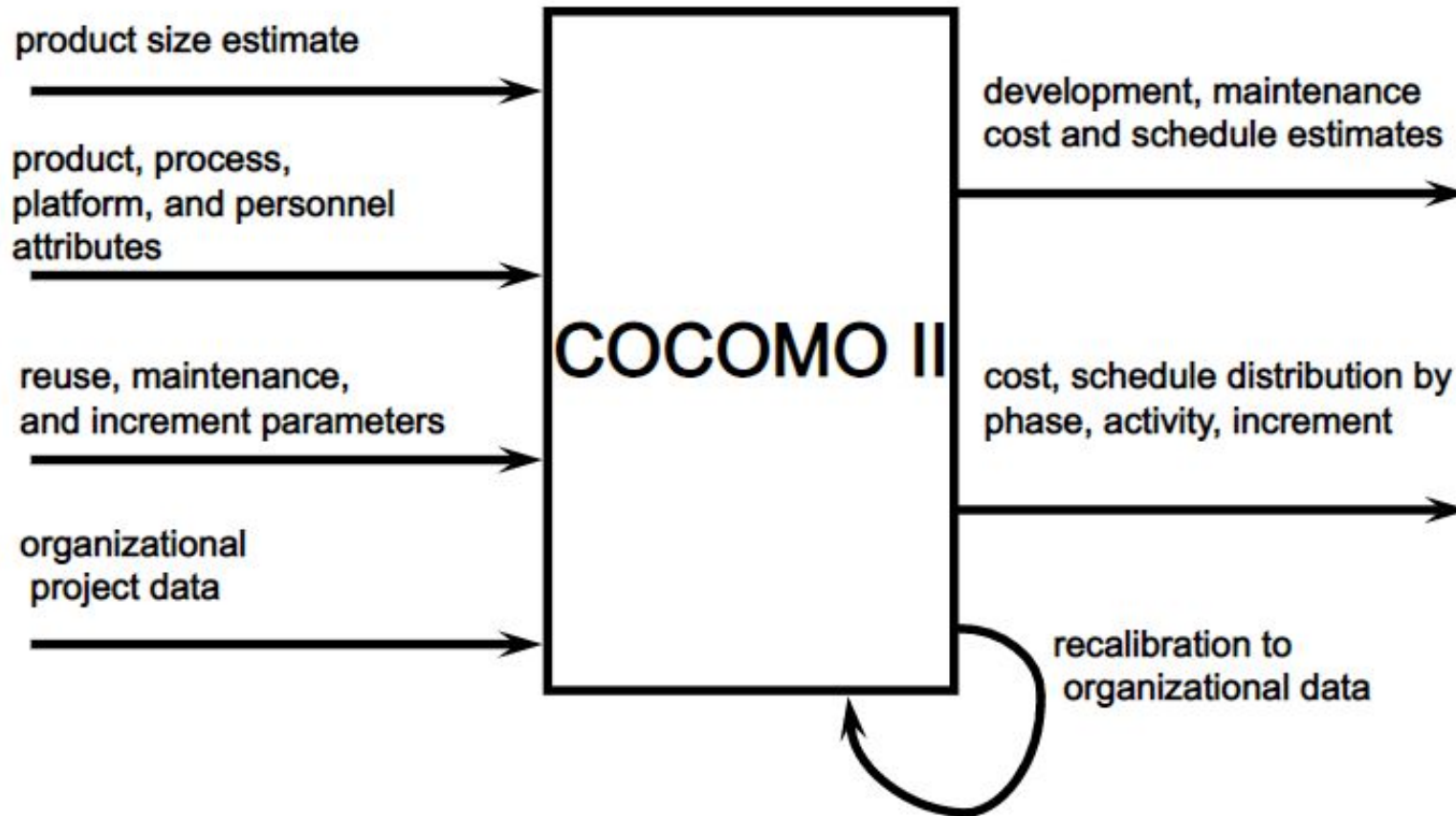
COCOMO formula for duration calculation: $M = a \times E^b$
where E is the estimated effort and

Project Type	a	b
Organic	2.5	0.38
Semi-detached	2.5	0.35
Embedded	2.5	0.32

Assuming that Mux-Core is a semi-detached project type with 500 fp, the project duration should be 22 months

See <https://www.geeksforgeeks.org/software-engineering/software-engineering-cocomo-model/> for details

COCOMO Black Box Model



©USC-CSSE

Brief introduction -

<https://www.geeksforgeeks.org/software-engineering-cocomo-ii-model/>

<https://slideplayer.com/slide/7947860/>

COCOMO II Calculator (read chap 5 of the text book for details)



Software Size Sizing Method **Function Points**

Unadjusted Function Points **440** Language **Java**

Software Scale Drivers

Precedentedness	Nominal	Architecture / Risk Resolution	Nominal	Process Maturity	Nominal
Development Flexibility	Nominal	Team Cohesion	Nominal		

Software Cost Drivers

Product		Personnel		Platform	
Required Software Reliability	Nominal	Analyst Capability	Nominal	Time Constraint	Nominal
Data Base Size	Nominal	Programmer Capability	Nominal	Storage Constraint	Nominal
Product Complexity	Nominal	Personnel Continuity	Nominal	Platform Volatility	Nominal
Developed for Reusability	Nominal	Application Experience	Nominal	Project	
Documentation Match to Lifecycle Needs	Nominal	Platform Experience	Nominal	Use of Software Tools	Nominal
		Language and Toolset Experience	Nominal	Multisite Development	Nominal
				Required Development Schedule	Nominal

Maintenance **Off**

<http://softwarecost.org/tools/COCOMO/>

COCOMO II Calculator (read chap 5 of the text book for more details)

innovate

achieve

lead

Results

Software Development (Elaboration and Construction)

Effort = 169.9 Person-months

Schedule = 20.0 Months

Cost = \$4246906

Total Equivalent Size = 40000 SLOC

Effort Adjustment Factor (EAF) = 1.00

12 169.9 20.0 4246906 40000 1.00

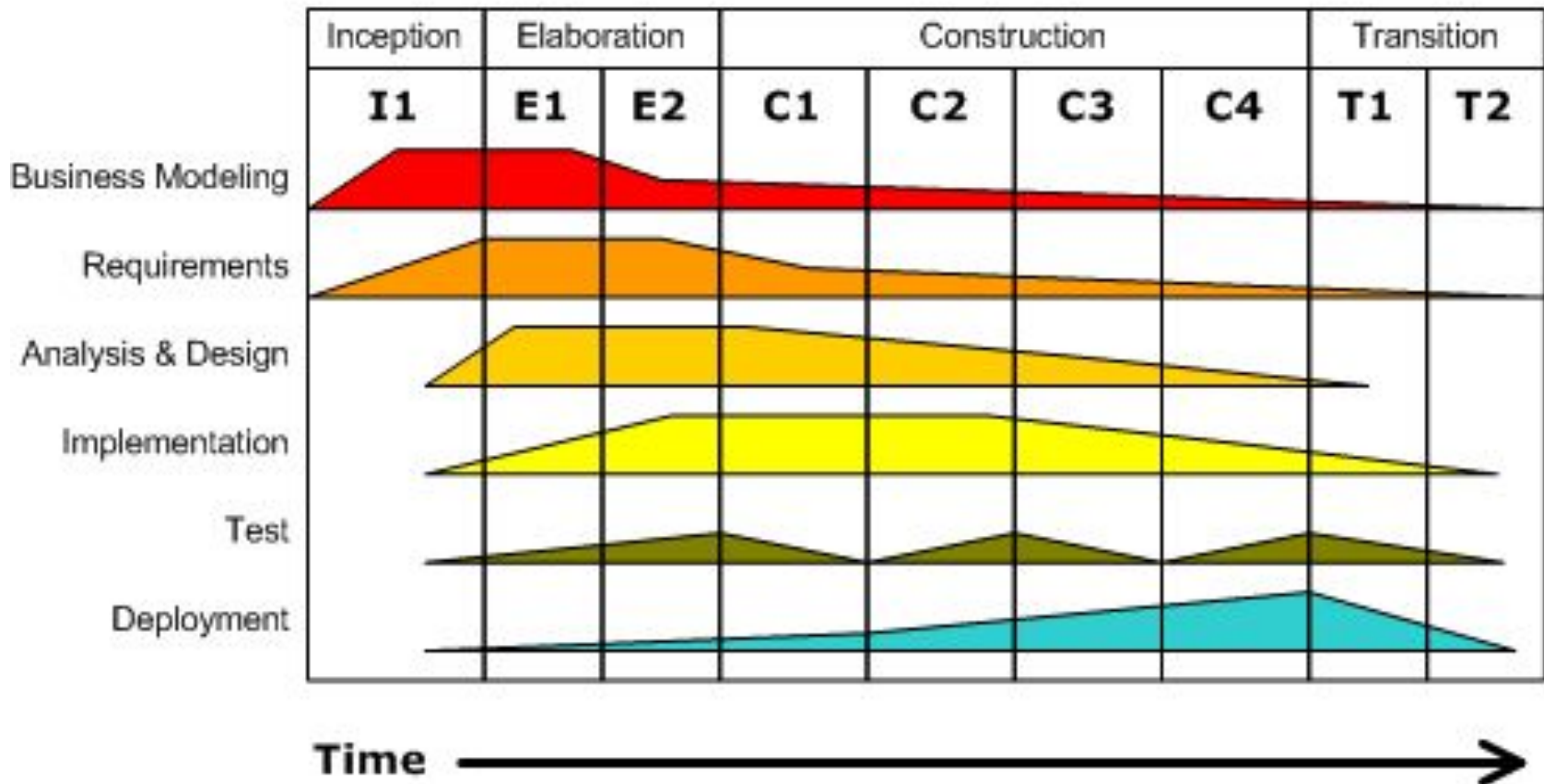
Note: Assume \$ = INR for interpretation of the inputs and results;; i.e., 25k rupees average salary per month

Unified Process (for explaining estimation results)



Iterative Development

Business value is delivered incrementally in time-boxed cross-discipline iterations.



COCOMO II Calculator



Results

Software Development (Elaboration and Construction)

Effort = 169.9 Person-months

Schedule = 20.0 Months

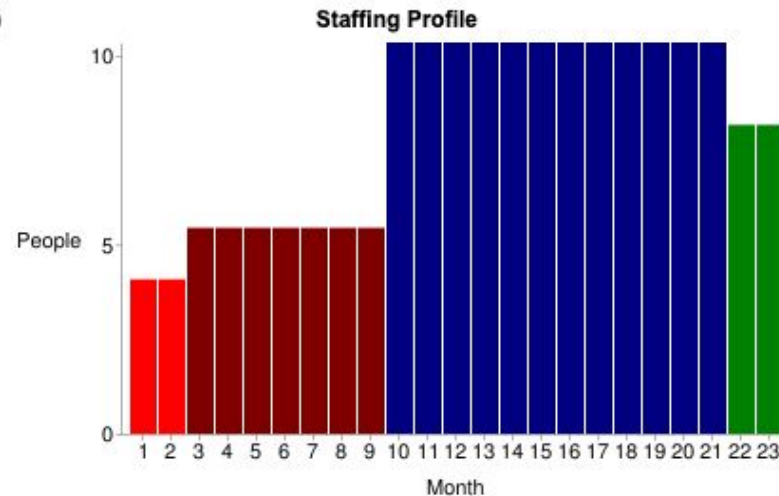
Cost = \$4246906

Total Equivalent Size = 40000 SLOC

Effort Adjustment Factor (EAF) = 1.00

Acquisition Phase Distribution

Phase	Effort (Person-months)	Schedule (Months)	Average Staff	Cost (Dollars)
Inception	10.2	2.5	4.1	\$254814
Elaboration	40.8	7.5	5.4	\$1019258
Construction	129.1	12.5	10.3	\$3227649
Transition	20.4	2.5	8.2	\$509629



Software Effort Distribution for RUP/MBASE (Person-Months)

Phase/Activity	Inception	Elaboration	Construction	Transition
Management	1.4	4.9	12.9	2.9
Environment/CM	1.0	3.3	6.5	1.0
Requirements	3.9	7.3	10.3	0.8
Design	1.9	14.7	20.7	0.8
Implementation	0.8	5.3	43.9	3.9
Assessment	0.8	4.1	31.0	4.9
Deployment	0.3	1.2	3.9	6.1

<http://softwarecost.org/tools/COCOMO/>

COCOMO II Calculator (read chap 5 of the text book for details)



Software Effort Distribution for RUP/MBASE (Person-Months)

Phase/Activity	Inception	Elaboration	Construction	Transition
Management	1.4	4.9	12.9	2.9
Environment/CM	1.0	3.3	6.5	1.0
Requirements	3.9	7.3	10.3	0.8
Design	1.9	14.7	20.7	0.8
Implementation	0.8	5.3	43.9	3.9
Assessment	0.8	4.1	31.0	4.9
Deployment	0.3	1.2	3.9	6.1

Note: Assume \$ = INR for interpretation of the inputs and results;; i.e., 25k rupees average salary per month

W06-#2 COCOMO II Calculator

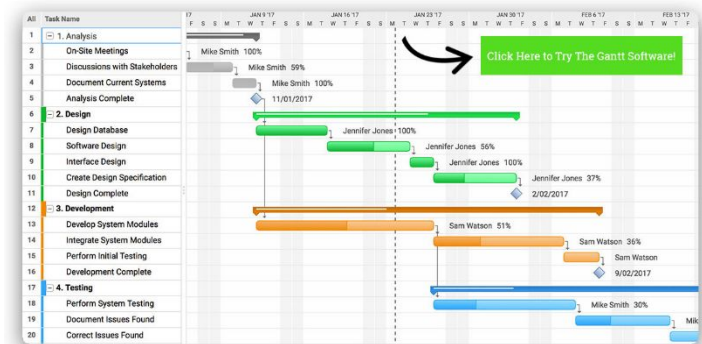


Keeping the function points as 500, observe the effect of software scale and cost drivers on the estimated effort, schedule and cost.

<http://softwarecost.org/tools/COCOMO/>

WBS and Project Plan

- **Work Breakdown Structure (WBS):** Breaking down the work into parts and assign these to project team members, anticipate problems that might arise and prepare tentative solutions to those problems.
- **Project Plan**, which is created at the start of a project, is used to communicate how the work will be done to the project team and customers, and to help assess progress on the project.



Example of a WBS fragment



3 Physical Design

3.01 Design or specify physical database

- 3.01.01 Review logical database design
- 3.01.02 Determine access methods to be used
- 3.01.03 Normalize database
- 3.01.04 Design database architecture
- 3.01.05 Identify reusable database structures
- 3.01.06 Develop detailed database layout
- 3.01.07 Develop database file, record, and schema descriptions
- 3.01.08 Develop module calling sequences
- 3.01.09 Update data dictionary entries
- 3.01.10 Validate physical database design

Example of a WBS fragment

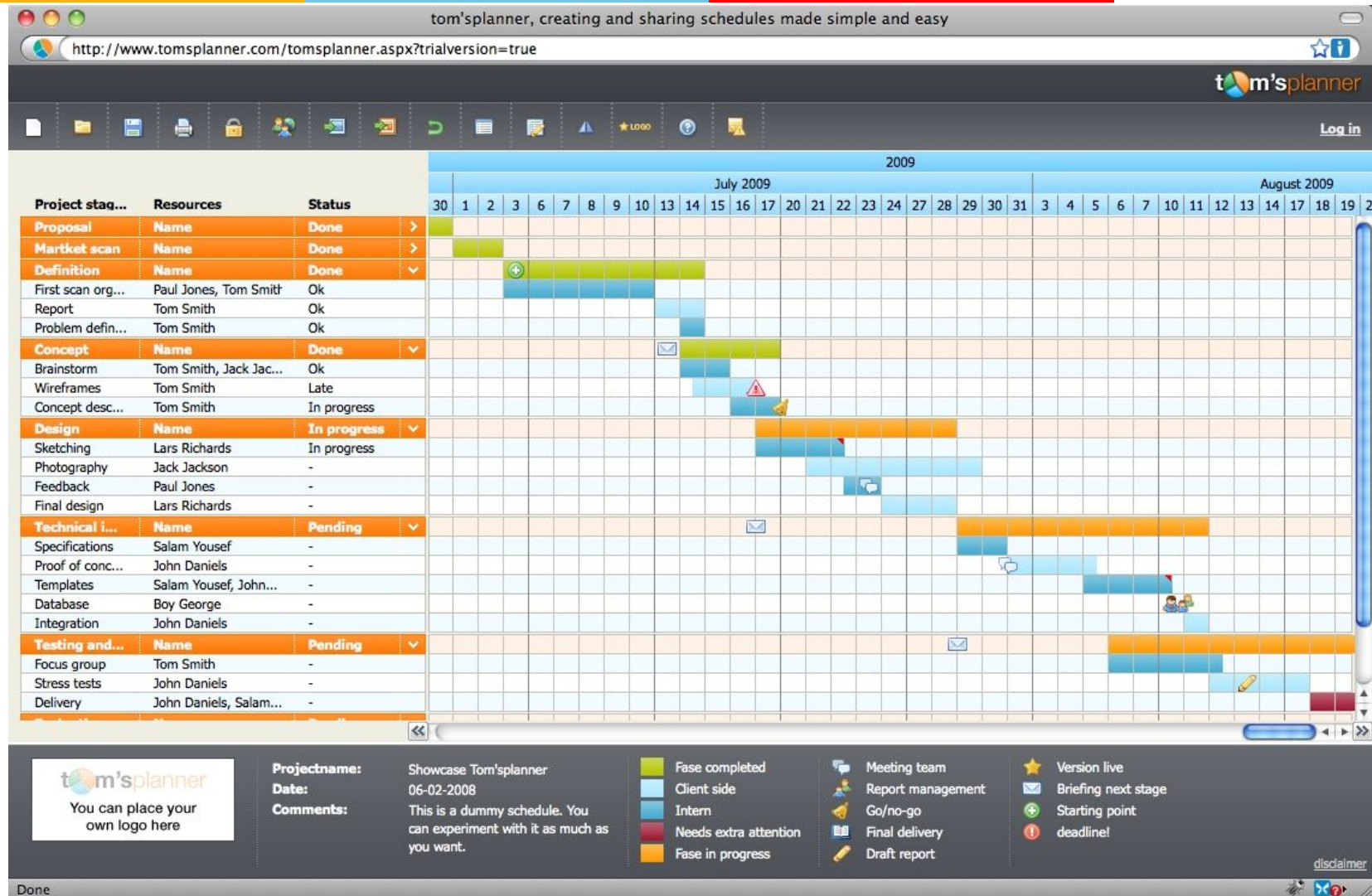


Level 3: Sub-Tasks within Each Phase

1. Requirements Gathering and Analysis

- **1.1. Requirement Elicitation**
 - Stakeholder interviews
 - Workshops
 - Surveys and questionnaires
 - Competitor analysis
- **1.2. Requirement Documentation**
 - Functional Requirements Specification (FRS)
 - Non-Functional Requirements Specification (NFRS)
- **1.3. Requirement Review and Validation**
 - Requirements review sessions
 - Sign-off by stakeholders
- **1.4. Feasibility Study**
 - Technical feasibility
 - Budget and resource feasibility

Activity planning – Gantt Charting tool



W06-#3 Gantt Chart

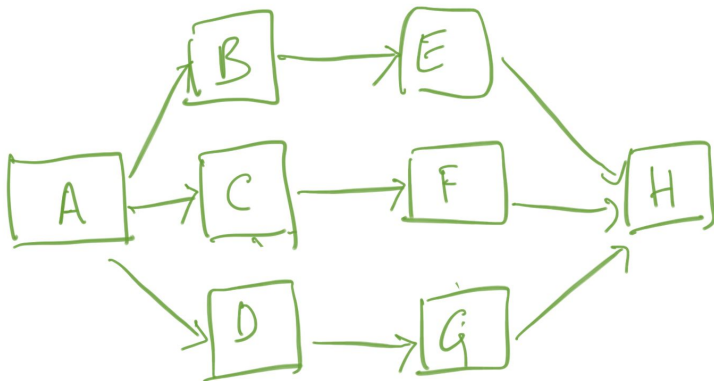


Activity	Duration (days)	Depends on	Resource
A	3		SA
B	1	A	SD1
C	2	A	SD2
D	4	A	SD2
E	3	B	CD1
F	3	C	CD1
G	6	D	CD2
H	3	E,F,G	SA

Prepare a Gantt Chart (on paper or any tool you can find) to find the critical path and project duration

Activity on Node Diagrams

Early Start	Duration	Early Finish
Task Name		
Late Start	Slack	Late Finish

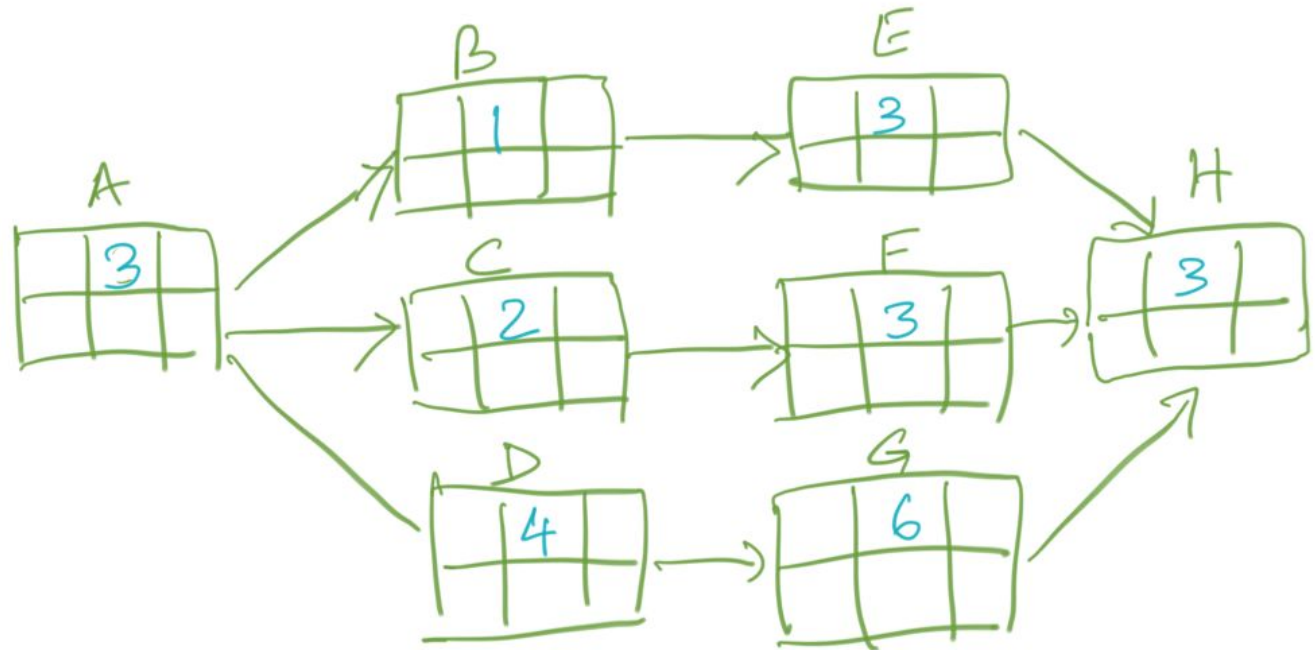


Activity	Duration (days)	Depends on
A	3	
B	1	A
C	2	A
D	4	A
E	3	B
F	3	C
G	6	D
H	3	E,F,G

Activity on Node Diagrams

Early Start	Duration	Early Finish
Task Name		
Late Start	Slack	Late Finish

Activity	Duration (days)	Depend s on
A	3	
B	1	A
C	2	A
D	4	A
E	3	B
F	3	C
G	6	D
H	3	E,F,G



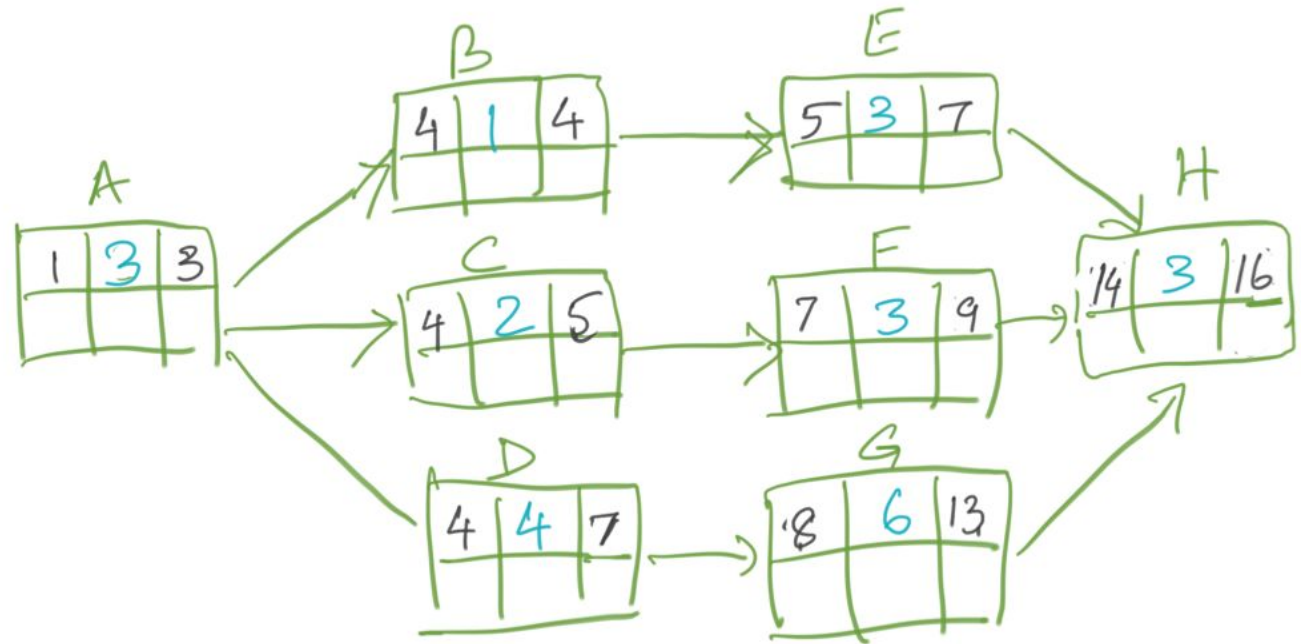
Forward pass in Activity on Node Diagrams



When an activity (e.g., H) is dependent on multiple activities then take the max of EFs

Early Start	Duration	Early Finish
Task Name		
Late Start	Slack	Late Finish

Activity	Duration (days)	Depend s on
A	3	
B	1	A
C	2	A
D	4	A
E	3	B
F	3	C
G	6	D
H	3	E,F,G



$$\text{Early Finish (EF)} = \text{Early Start (ES)} + \text{Duration} - 1$$

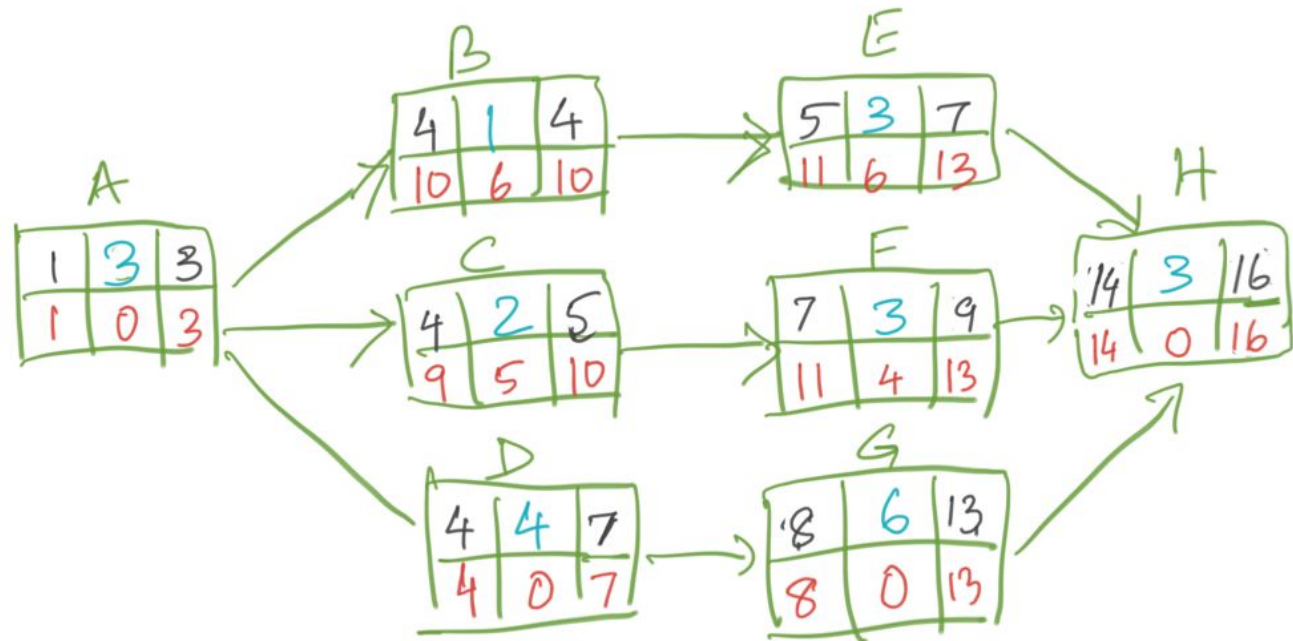
Backward pass in Activity on Node Diagrams



When an activity (e.g., A) is branching to multiple paths then take the min of LSs

Early Start	Duration	Early Finish
Task Name		
Late Start	Slack	Late Finish

Activity	Duration (days)	Depend s on
A	3	
B	1	A
C	2	A
D	4	A
E	3	B
F	3	C
G	6	D
H	3	E,F,G



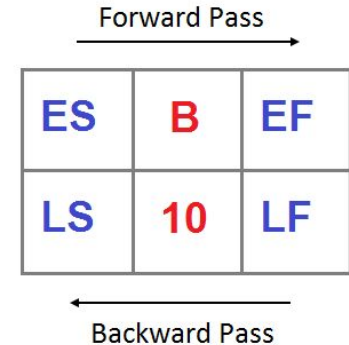
$$LS = LF - Duration + 1$$

$$Float/Slack = LS - ES \text{ or } LF - EF$$

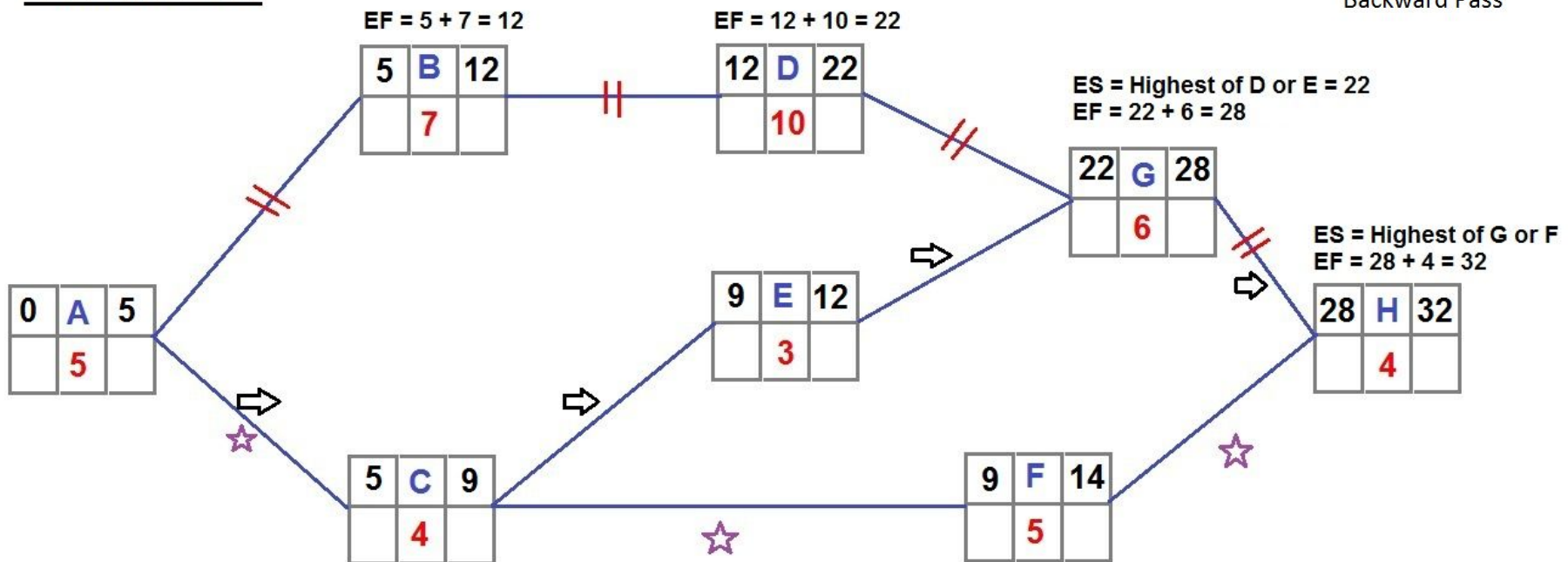
Activity on Node Diagrams (with 0 start instead of 1)



$$\text{Early Finish (EF)} = \text{Early Start (ES)} + \text{Duration}$$



FORWARD PASS



<https://tiemchart.com/>

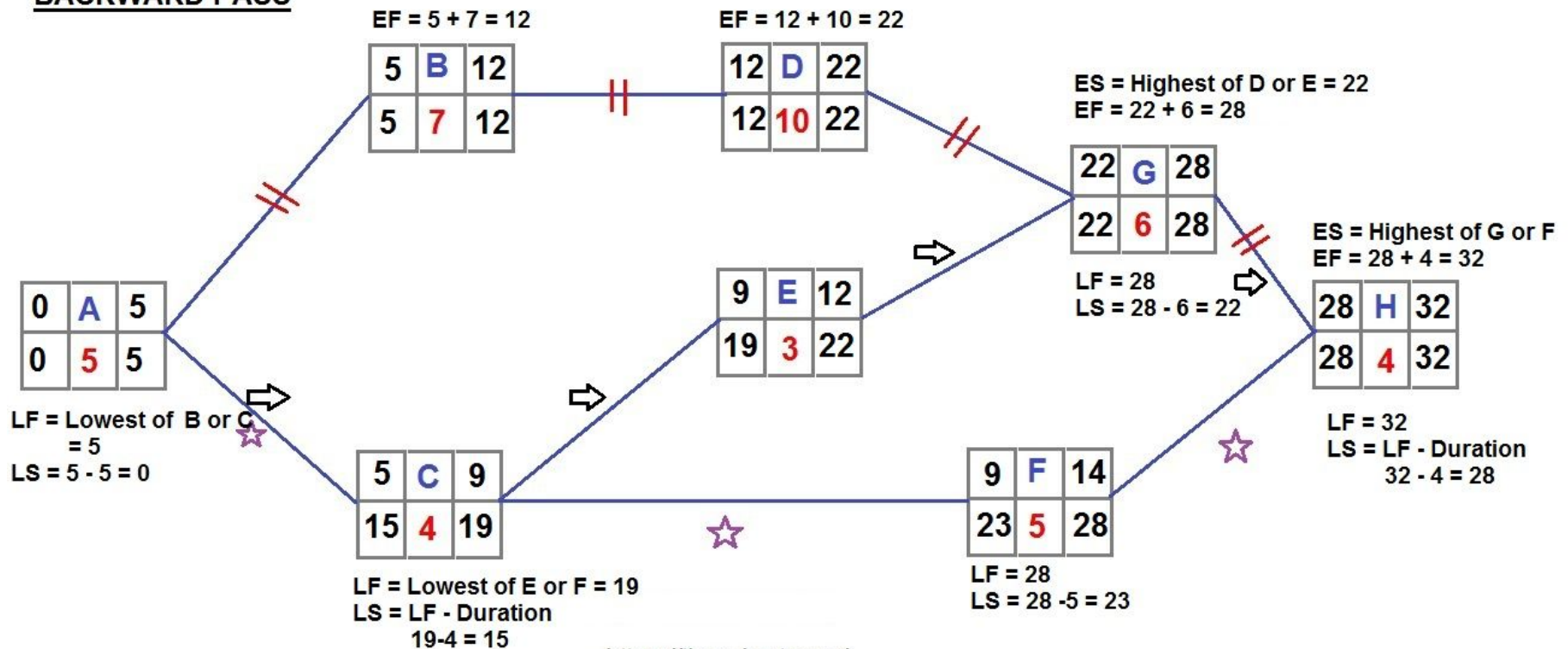
<https://tiemchart.com/blogs/training/forward-pass-and-backward-pass/>

Activity on Node Diagrams (with 0 start instead of 1)



$LS = LF - \text{Duration}$ (float/slack is not shown in the diagram)

BACKWARD PASS



<https://tiemchart.com/>

<https://tiemchart.com/blogs/training/forward-pass-and-backward-pass/>

AoN Summary



- Forward Pass gives us the earliest start (ES) and earliest finish (EF) of each activity.
- Backward Pass gives us the latest start (LS) and latest finish (LF).
- Float/slack is calculated by comparing the results of forward and backward passes, indicating how much flexibility there is for each activity.
- The critical path is the sequence of activities with zero float, representing the longest path through the project



Gantt chart vs AoN diagram

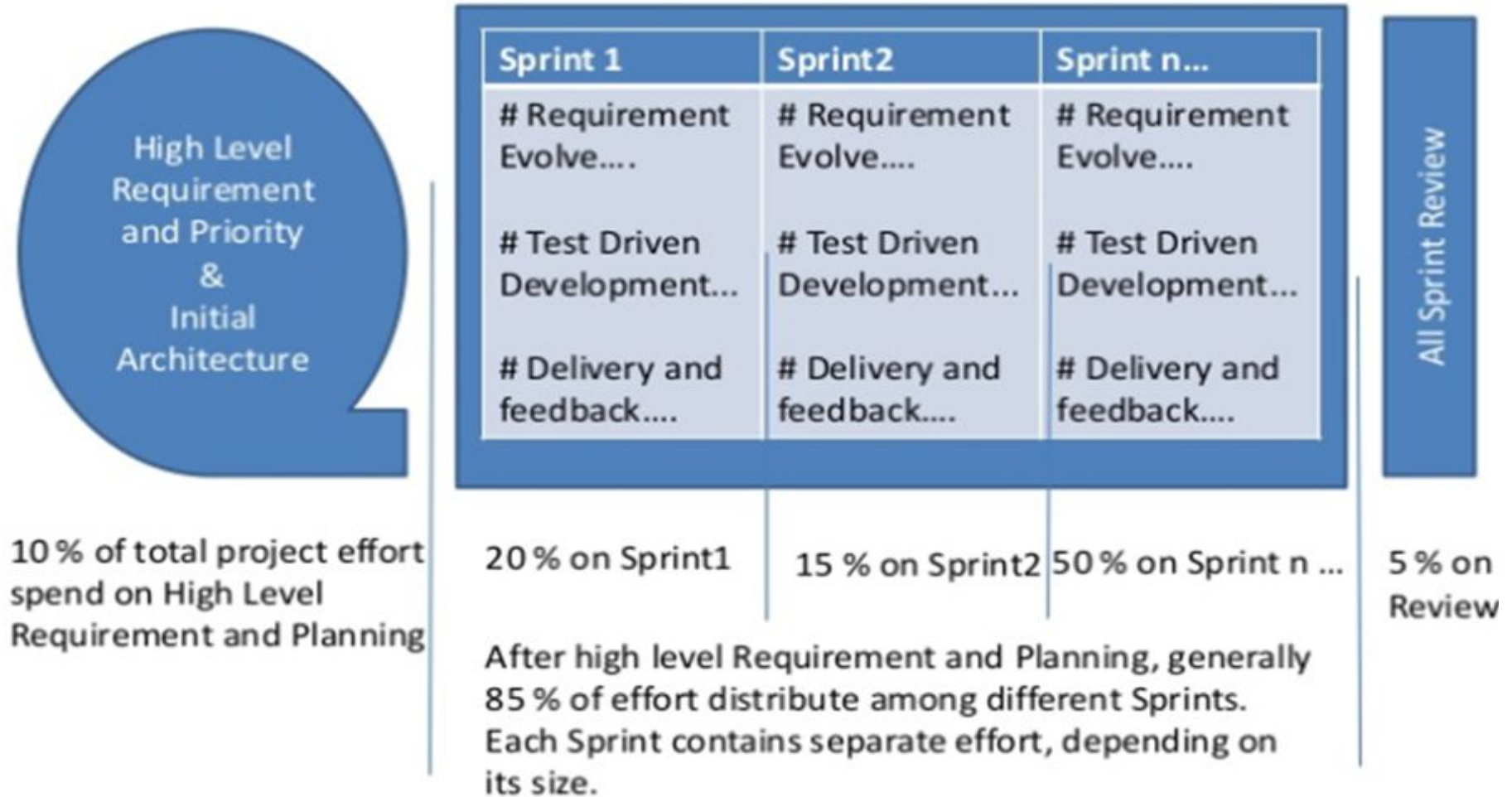
Compare suitability of Gantt charts and
AoN diagrams for ACL Mux Core project scheduling

Scheduling in agile methods



- Allocate time for **sprint 0** activities
- Distribution of effort and time (not uniform across sprints)
- Example agile project management tool

Example of effort distribution in agile process (Scrum)



<https://www.slideshare.net/brickedestimation/effort-distributiononwaterfallandagile>

WBS in agile projects

- High-level planning with WBS (Epics and release planning)
- Product -> Epics -> User stories -> Tasks
- Dynamic WBS
- Time-boxed WBS

Pivotal Tracker(example)

https://www.pivotaltracker.com/help/articles/creating_a_project/



Pivotal Tracker (example)



- Epics
- Icebox
- My work
- Story types
 - Features
 - Chores
 - Bugs
 - Releases

W06-#5 Gantt chart and AoN diagram in agile projects



Do you think Gantt charts and AoN diagrams are relevant for ACL Mux Core project to be executed using Scrum framework ? Explain.



BITS Pilani presentation

BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Prof. Narasimha Bolloju
CSIS Department, Hyderabad



SS ZG622

Software Project Management

Lecture No. 7

Recap - Planning and scheduling



Scheduling in Waterfall

- **Upfront Planning:** Scheduling is detailed and occurs at the project's start, with all tasks, milestones, and dependencies defined early using tools like WBS, Gantt charts, and critical path analysis with AoN diagrams..
- **Fixed Timeline:** Once set, the schedule is rigid, with phases (e.g., requirements, design, implementation) completed sequentially.
- **Dependency-Driven:** Tasks follow a strict sequence, with the critical path determining the minimum project duration.
- **Progress Monitoring:** Measured against the initial plan, with little flexibility for changes.
- **Challenges:** Suited for stable requirements but risks delays if unforeseen issues arise.

Recap - Planning and scheduling



Scheduling in Scrum

- **Iterative Planning:** Scheduling is incremental, with plans evolving through 2-4 week sprints.
- **Flexible Timeline:** Release dates are estimates, adjusted based on sprint velocity and feedback.
- **Sprint-Based:** Each sprint has a goal with schedules refined during sprint planning.
- **Progress Monitoring:** Tracked with burndown charts and sprint reviews allowing real-time adjustments.
- **Challenges:** Requires active team collaboration and may lack long-term visibility, necessitating a release plan.

Quality Planning

Quality Planning



- Quality management
- Quality planning
 - Identification relevant quality attributes
 - Defining associated measures
 - Setting up standards for the product and processes

Quality



- Quality, simplistically, means that a product should meet its specification (i.e., meets functional needs and adheres to standards)
- The software product should deliver the required functionality (*functional requirements*) with the required **quality attributes** (*non-functional requirements*)

Quality management



- **Quality planning**
 - Setting quality standards and developing a plan to achieve them
- **Quality assurance**
 - Ensuring that the development process is compliant with the quality plan
- **Quality control**
 - Monitoring and evaluating the product to ensure that it meets the quality requirements

Further discussion assurance and control is planned during the second half of this course

Product and process standards –

Some examples



Product standards

Design review form

Requirements document structure

Programming style

Project plan format

Change request form

Process standards

Design review conduct

Submission of new code for system building

Project plan approval process

Change control process

Test recording process

Quality planning



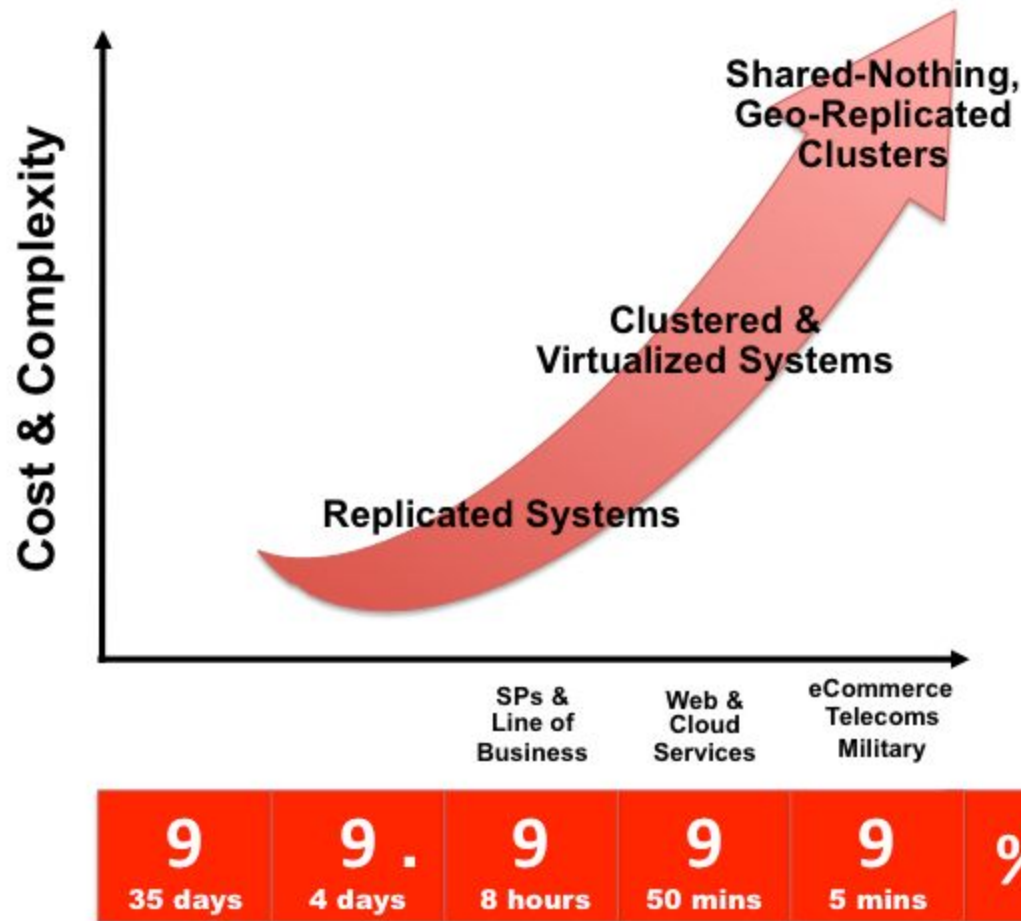
- Identifies the most significant quality attributes appropriate for the product
- Defines the assessment process in detail for each quality attribute (including goals or target values)
- Indicates which standards should be applied and defines new standards as necessary

Quality attribute example



- Reliability – probability of system working satisfactorily within a specific period of time
- Possible measures:
 - Availability: % of a time that a system is usable
 - Mean time between failures: total service time/number of failures
 - Failure on demand: probability that system will not available when needed
 - Support activity: number of fault reports generated and processed

Trade-offs: Cost & Complexity vs. Availability



https://docs.oracle.com/cd/E17952_01/mysql-5.5-en/ha-overview.html

Software Quality Attributes

- Safety
- Security
- Reliability
- Resilience
- Robustness
- Understandability
- Testability
- Adaptability
- Modularity
- Complexity
- Portability
- Usability
- Reusability
- Efficiency
- Learnability

ISO/IEC 25010 standard



<https://www.monterail.com/blog/software-qa-standards-iso-25010>

ISO/IEC 25010 - Product Quality model



Focuses on the static properties of a product, including its eight main quality characteristics (listed below) and their sub-characteristics:

- **Functional suitability:** Functions performed by the system fulfill user needs.
- **Performance efficiency:** How well the system performs its functions in terms of time and resources.
- **Compatibility:** The ability of the system to exchange information with other systems or operate in a shared environment.
- **Usability:** How easy the system is to use and understand by users.
- **Reliability:** The ability of the system to maintain its level of performance under specific conditions.
- **Security:** How well the system protects data and resources from unauthorized access and breaches.
- **Maintainability:** The ease with which the system can be modified.
- **Portability:** The ease with which the system can be transferred from one environment to another.

Quality attributes and some related measures/metrics



Quality Attribute	Description	Measure/metric Example
Reliability	System uptime and error handling	Mean Time Between Failures (MTBF)
Usability	Ease of use	User satisfaction surveys
Performance	Speed and efficiency	Response time metrics

Activity - W07#1

Suggest measures/metrics corresponding to the eight quality attributes listed for Mux-Core system

Some common practices for selecting attributes and metrics



- **Functional Suitability:** 1-2 metrics (e.g., Percentage of requirements fulfilled, Functional completeness).
- **Performance Efficiency:** 2-3 metrics (e.g., Response time, Throughput, Resource utilization).
- **Compatibility:** 1-2 metrics (e.g., Integration success rate, Error rate in data exchange).
- **Usability:** 2-3 metrics (e.g., Task completion time, User satisfaction score, Error frequency).
- **Reliability:** 2-3 metrics (e.g., Mean Time Between Failures (MTBF), Availability percentage, Failure rate).
- **Security:** 2-3 metrics (e.g., Vulnerability detection rate, Data breach incidents, Compliance rate).
- **Maintainability:** 1-2 metrics (e.g., Change request resolution time, Code complexity).
- **Portability:** 1-2 metrics (e.g., Adaptation effort, Platform compatibility rate).

Typical quality planning activities in waterfall



Waterfall's linear approach allows for comprehensive upfront quality planning, ensuring all quality aspects are defined early in the project. Key activities include:

- **Define Quality Attributes and Metrics:** Establish specific quality standards (e.g., ISO 25010 attributes) and metrics (e.g., defect rate < 5%, system uptime > 99.9%) for the system.
- **Develop Quality Management Plan:** Create a detailed plan outlining quality objectives, roles, and processes (e.g., reviews, audits) for each phase—requirements, design, implementation, testing, and deployment.
- **Conduct Requirements Validation:** Perform thorough reviews and walkthroughs of requirements with stakeholders to ensure quality is built into the system design, addressing issues like integration with other systems.
- **Plan Testing Phases:** Schedule distinct testing phases (unit, integration, system, acceptance) with predefined test cases, ensuring the system meets performance and security needs.
- **Risk-Based Quality Prioritization:** Identify potential quality risks (e.g., legacy system failures) and prioritize quality efforts accordingly, documenting mitigation strategies in the plan.

Typical quality planning activities in Scrum



Scrum's iterative nature integrates quality planning into each sprint, making it adaptable for the project's evolving requirements. Key activities include:

- **Define Sprint Quality Goals:** Set quality objectives for each sprint, aligning with user stories and the Definition of Done (DoD).
- **Incorporate Quality in Backlog Refinement:** Prioritize quality-related tasks (e.g., automated testing scripts) in the product backlog, ensuring continuous attention to quality during sprints.
- **Implement Continuous Testing:** Conduct unit tests, integration tests, and acceptance tests within each sprint using automated tools, validating features like integrating with external systems.
- **Conduct Sprint Reviews for Quality Feedback:** Engage stakeholders in sprint reviews to assess quality, gathering feedback on usability and performance to refine future sprints.
- **Use Retrospectives for Quality Improvement:** Analyze quality issues (e.g., bugs in systems integration) during retrospectives, planning improvements for the next sprint to enhance the system's reliability.

Risk Management

Risk management



- Software projects are inherently complex and uncertain due to changing requirements, evolving technology and dependence on multiple stakeholders
- Effective risk management helps to anticipate and address potential issues before they become major problems
- Ensures project success by minimizing the impact of negative risks and maximizing the potential of positive risks

Risk Management



- Risk identification – what are the risks to a project?
- Risk analysis – which ones are really serious?
- Risk planning – what shall we do?
- Risk monitoring – has the planning worked?

Risk Identification



- **Use of checklists** – usually based on the experience of past projects
- **Brainstorming** – getting knowledgeable stakeholders together to pool concerns
- **Causal mapping** – identifying possible chains of cause and effect

Activity W07#2: Identify possible risks for the Mux-Core project



Visit <http://www.projectengineer.net/project-risk-checklist/> and identify five or six possible risks relevant for the Mux-Core project

Post your suggestions with justification with the probability of its occurrence high/medium/low and severity or impact (e.g., high > 0.8 , medium > 0.4 , low ≤ 0.4)

E.g., scope creep (medium) – many of the requirements related to administration and counter ticket sales are fairly well understood; but, the mobile/web for customers and management support need more clarity

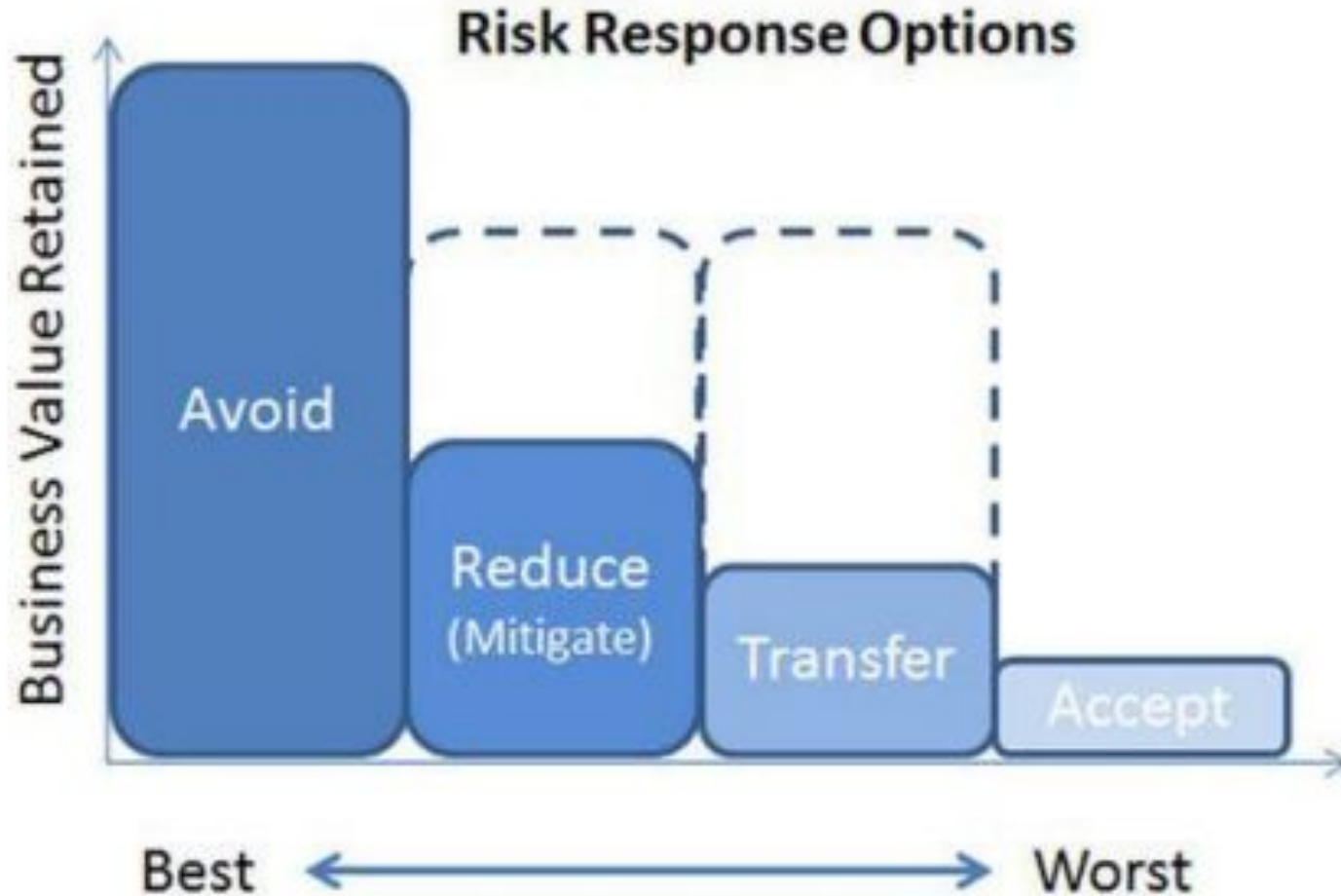
(Try different risk categories; Do not repeat the entries posted by others; instead improve those entries)

Risk mitigation - examples



Risk	Probability (High/Med/ Low)	Impact (High/Med/ Low)	Mitigation Strategy	Monitoring Metric
Changing requirements	High	High	Regular stakeholder reviews	Number of change requests per sprint
Technology obsolescence	Med	High	Enhance vendor relationship	Tech stack audit frequency
Stakeholder delays	Med	Med	Contingency buffers	Milestone completion rates

Response options



Risk Probability and Severity



		Severity		
		Low	Medium	High
Probability	High	Medium risk	High risk	High risk
	Medium	Low risk	Medium risk	High risk
	Low	Low risk	Low risk	Medium risk
		Low risk	Medium risk	High risk

Activity W07#3: Suggest how the following risks can be managed



- Scope creep
- Quality of product does not meet standards
- Project is running behind schedule

Suggest **reduce (mitigate)** strategies for managing above risks

Commonly used risk items in software development projects



Small Projects (e.g., <6 months, <5 team members):

10-12 risks, focusing on key areas like resource availability and technical feasibility.

Medium Projects (e.g., 6-12 months, 5-15 team

members): 15-20 risks, addressing scope creep, integration challenges, and schedule delays.

Large Projects (e.g., >12 months, >15 team members):

20+ risks, including strategic, operational, and external factors.

Typical risk management activities in Waterfall



Waterfall's sequential nature allows for a structured, upfront approach to risk management.

Key activities include:

- **Risk Identification:** Conduct a comprehensive risk assessment during the planning phase, identifying potential risks such as legacy system integration failures, stakeholder resistance, or project delays.
- **Risk Analysis:** Perform a detailed analysis using a probability-impact matrix to prioritize risks (e.g., infrastructure downtime, staff availability issues), documenting findings in a risk register.
- **Risk Planning:** Develop a risk mitigation plan with specific strategies, such as allocating contingency buffers for delays or conducting pilot tests on legacy systems before full integration.
- **Risk Monitoring and Control:** Establish checkpoints at the end of each phase (e.g., design, implementation) to review risks, updating the risk register and adjusting plans as needed during the system development.
- **Documentation and Reporting:** Maintain detailed documentation of all risks, mitigation actions, and outcomes, ensuring traceability and providing a reference for future projects.

Typical risk management activities in Scrum



Scrum's iterative approach integrates risk management into each sprint, allowing for continuous identification and adaptation. Key activities include:

- **Risk Identification in Sprint Planning:** Identify risks specific to each sprint (e.g., scope creep or technology obsolescence) during planning sessions.
- **Risk-Adjusted Backlog Prioritization:** Adjust the product backlog to address high-priority risks early, such as testing integration with external systems in the initial sprints to mitigate connectivity issues.
- **Mitigation During Sprints:** Implement risk mitigation strategies incrementally, such as prototyping uncertain components within a sprint and reviewing outcomes in daily stand-ups.
- **Risk Review in Sprint Retrospectives:** Assess the effectiveness of risk management strategies post-sprint, identifying new risks (e.g., staff burnout) and planning improvements for the next sprint.
- **Visual Risk Tracking:** Use tools like risk burndown charts or Kanban boards to monitor risks visually, updated in real-time fostering team awareness.

Some risk management tools and platforms



- Microsoft Project: Offers comprehensive risk tracking and reporting for large-scale projects.
- Primavera Risk Analysis: Specializes in quantitative risk analysis for complex projects with many variables.
- Celoxis: A platform for detailed risk analysis, monitoring, and integration with other project functions.
- Asana and Trello: Used for tracking project progress and mitigating risks.
- ProofHub: A centralized platform to identify, assign, and track risks, converting them into actionable tasks.



BITS Pilani presentation

BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Prof. Narasimha Bolloju
CSIS Department, Hyderabad



SS ZG622

Software Project Management

Lecture No. 8

Project monitoring and control



- Overview and common techniques
- Status report content and examples
- Earned value management
- Monitoring agile projects (some example reports)

Project monitoring and control

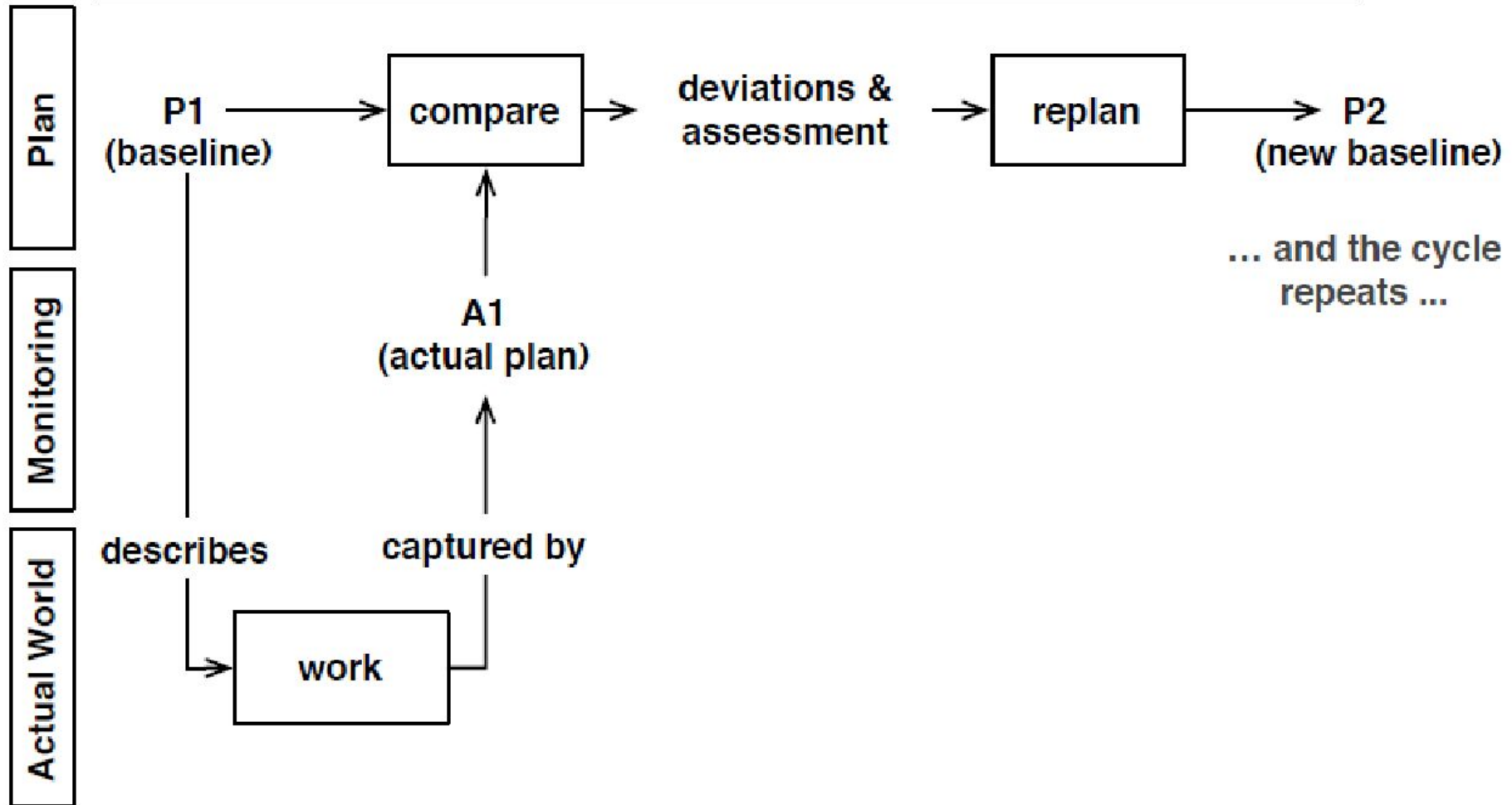


Monitoring involves collecting data on project performance;
Control uses that data to make adjustments and keep the project aligned with goals.

Key Benefits

- Early detection of issues (e.g., scope creep or delays in maintenance releases).
- Ensures alignment with time, cost, quality, and scope.
- Improves stakeholder communication and team accountability.

Monitoring and control cycle



Activity W08#1



What can be the probable challenges associated with monitoring the Mux Core project?
(assuming a plan-driven process model)

Project monitoring and control



Aspect	Plan-Driven (e.g., Waterfall)	Agile (e.g., Scrum)
Monitoring Focus	Baseline vs. actual (e.g., Gantt charts, milestones).	Daily stand-ups, burndown charts, velocity tracking.
Control Mechanisms	Change requests, variance analysis (e.g., earned value management - EVM).	Retrospectives, backlog adjustments, sprint reviews.
Frequency	Periodic (weekly/monthly reports).	Continuous (daily/iteration-end).
Tools	MS Project, Primavera.	Jira, Trello, Azure DevOps.

Techniques for monitoring and control



- **Earned Value Management (EVM):** Measures progress with metrics like Schedule Variance (SV) and Cost Variance (CV).
- **Risk Registers:** Track risks with probability/impact matrix; update weekly.
- **Change Control:** Formal process to approve scope changes—prevent "feature creep" in software.
- **Performance Reviews:** KPIs like defect rates, code coverage for development projects.

Example status report contents



- Project summary (progress %).
- Achievements, issues, risks.
- Next steps and decisions needed.
- Tailored for audience: High-level for execs, detailed for teams.

Examples of project status reports

Project Budapest

Project Status



Ongoing Activities: 5

Activity	% done
System Build	23%
Interface Build	32%
Project Review	64%
Quality Standards Prep.	12%
Interface Testing	15%

Top 5 Issues

- Find test manager
- Desclope Risky Items
- Start System Testing
- Find more money
- Demo to user group

Completed



23%

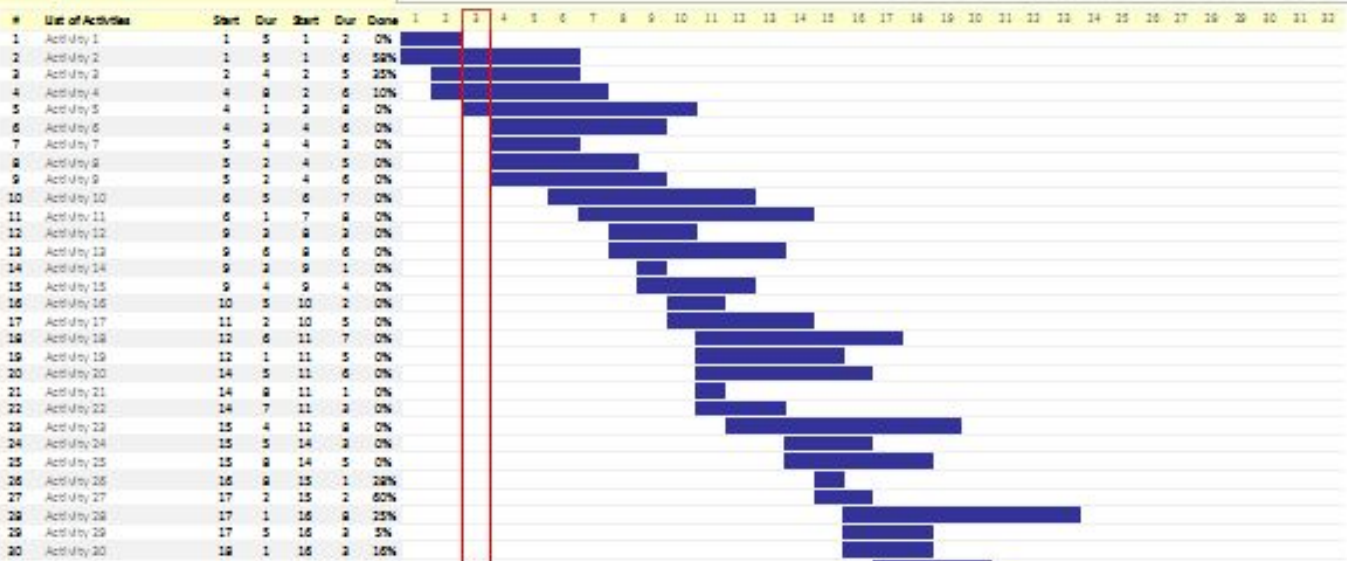
Outstanding Issues

3

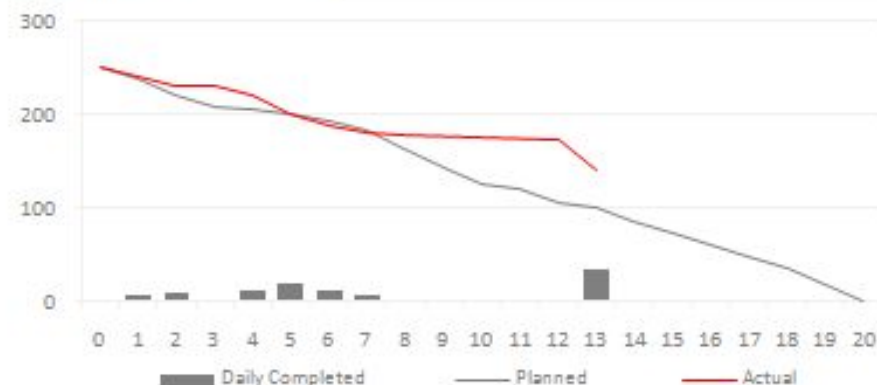
6

4

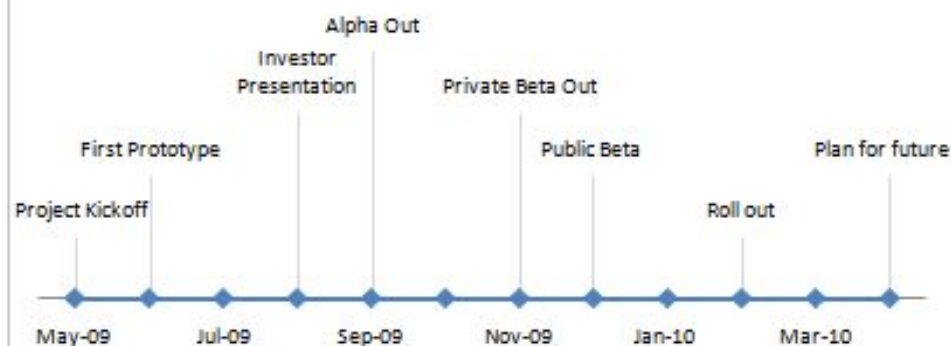
Project Plan



Burndown Chart



Project Timeline





Project Name	BRMS Tech	Project Manager	Swapnil Wale	Date	10-Jan-15	
Project Health Card				Green		
Schedule	Green	The project tasks are on track.				
Budget	Green	The budget is on track but some items are a concern.				
Resources	Amber	The project is well resourced. Some resources have taken unplanned leaves.				
Quality	Red	There number of unresolved defects has crossed 50.				
Scope	Amber	The project scope is on track. There are 2 change requests pending.				
Risks	Amber	Project Risks are under control.				
Issues	Red	There are 4 high issues open.				
Change Mgt	Green	The change management plan for the project is being drafted.				
Project Schedule						
No	Task	Start	Finish	Progress	Projected Date	Concerns
1	Analysis	14-Jan-15	30-Jan-15	100	30-Jan-15	No
2	Build	30-Jan-15	20-Mar-15	60	31-Jan-15	No
3	Quality Assurance	25-Mar-15	30-Apr-15	50	01-Feb-15	No
4	User Acceptance Testing	30-Apr-15	15-May-15	10	02-Feb-15	No
5	Production	20-May-15	20-May-15	0	03-Feb-15	No
6	Support	20-May-15	15-Jun-15	0	04-Feb-15	No
Project Financials						
No	Item	Planned Spend	Actual Spend	Variance		
1	Resourcing Costs	\$450,000	\$500,000	-\$50,000		
2	Software Costs	\$75,000	\$34,000	\$41,000		
3	Hardware Costs	\$50,000	\$23,000	\$27,000		
4	Vendor Costs	\$35,000	\$36,000	-\$1,000		
5	Other Expenses	\$5,000	\$3,500	\$1,500		
Totals		\$615,000	\$596,500	\$18,500		

Techno-PM

Project Management Templates

<https://www.techno-pm.com/2015/08/weekly-status-report-format-excel.html?m=1>

(contd.)



Top 5					
No	Type	Description	Status	Owner	Due
1	Risk	There is a technical risk with the project which has not been addressed.	Green	Neel	12-Jan
2	Issue	Some of the team members have taken unplanned leaves.	Red	Wale	14-Jan
3	Decision	A key decision regarding technical architecture is pending.	Amber	Sam	23-Feb
4	Dependency	The project has a dependency on other project for hardware which has not been resolved yet.	Green	Ram	28-Apr
5	Change Request	The change requests need be approved at the earliest so that the impact on the project is less.	Green	Jam	21-Jan
Risks			Issues		Pending Items



Executive Project Status Report with Budget Information

This slide is 100% editable. Adapt it to your needs and capture your audience's attention.



[executive_project_status_report_with_budget_information_slide01.jpg](#)

Earned value management

Earned value management



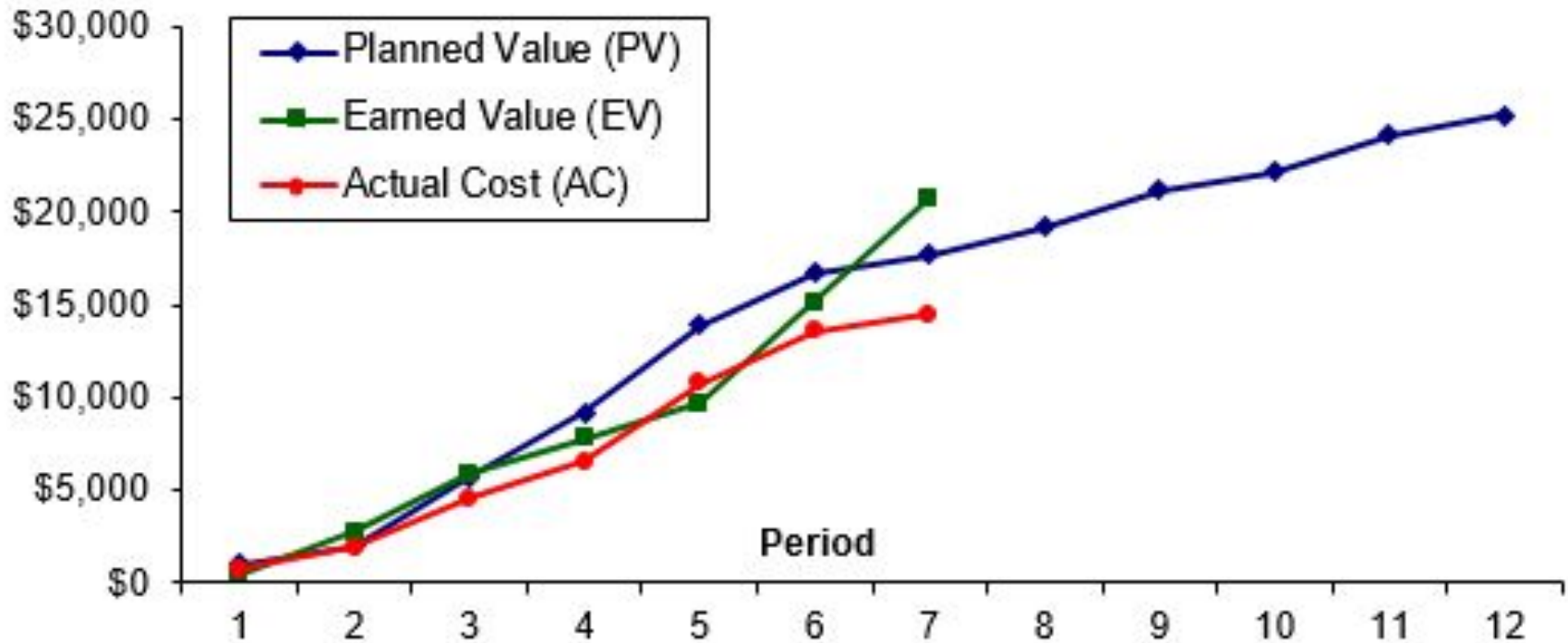
Work Package	BCWS	ACWP	%Progress	BCWP
	Planned Value	Actual Cost		Earned Value
1	\$1,00,000.00	\$1,20,000.00	100%	\$1,00,000.00
2	\$1,00,000.00	\$1,10,000.00	100%	\$1,00,000.00
3	\$1,00,000.00	\$80,000.00	90%	\$90,000.00
4	\$1,00,000.00	\$1,25,000.00	80%	\$80,000.00
5	\$1,00,000.00	\$75,000.00	50%	\$50,000.00
6	\$1,00,000.00	\$0.00	0%	\$0.00
7	\$1,00,000.00	\$0.00	0%	\$0.00
8	\$1,00,000.00	\$0.00	0%	\$0.00
9	\$1,00,000.00	\$0.00	0%	\$0.00
10	\$1,00,000.00	\$0.00	0%	\$0.00
BAC	\$10,00,000.00	\$5,10,000.00		\$4,20,000.00

BCWS – budgeted cost of work scheduled; ACWP – actual cost of work performed;
BCWP – budgeted cost of work performed; BAC – budget at completion

Activity W08#2 -



Interpreting earned value reports



What does the above chart indicate?

Activity W08#3 -

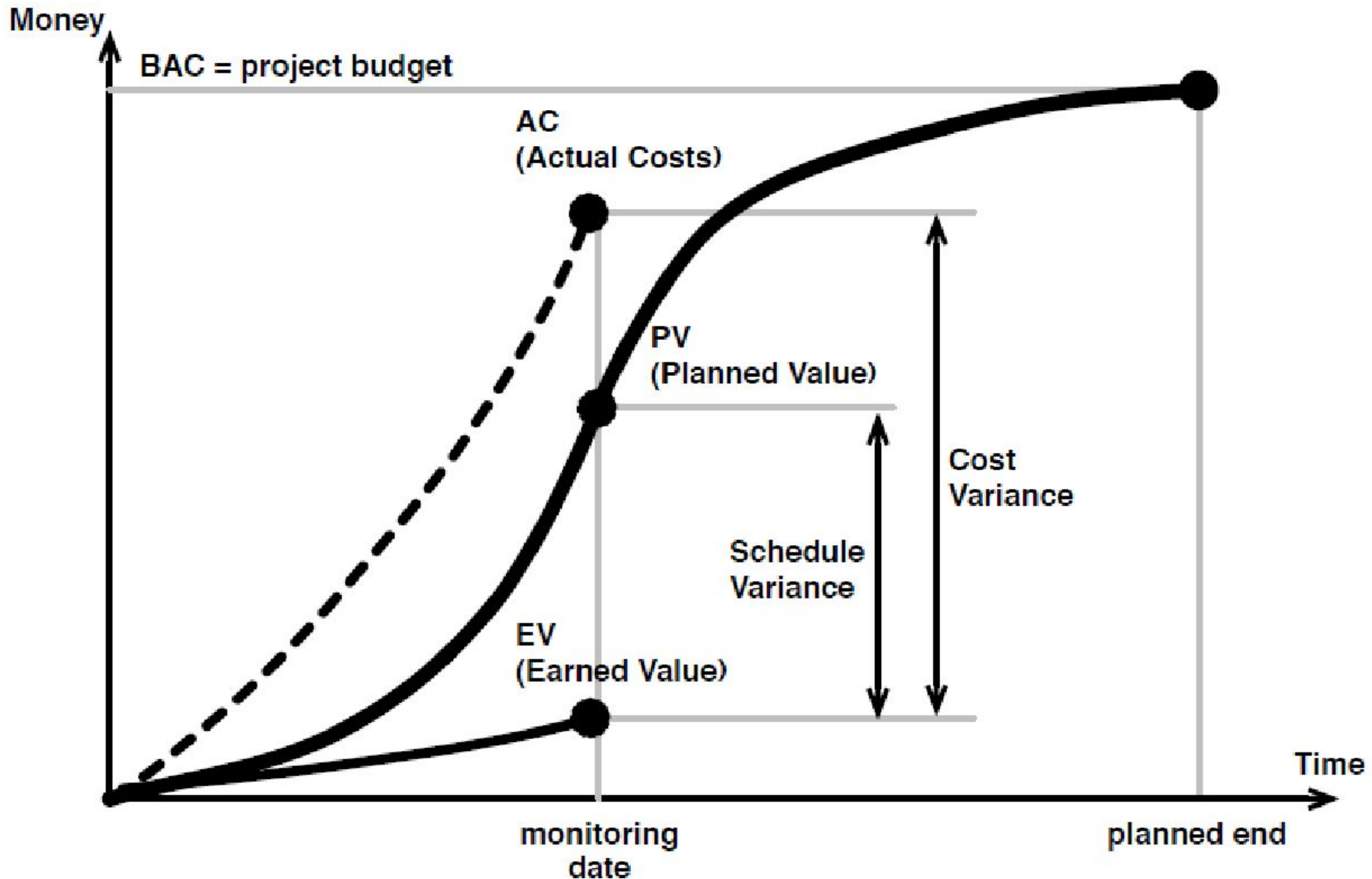


Interpreting earned value reports

How will you interpret and use these variances and indices?

PV	\$5,00,000.00
AC	\$5,10,000.00
EV	\$4,20,000.00

Cost Variance ($CV = EV - AC$)	-\$90,000.00
Cost Performance Index ($CPI = EV/AC$)	0.82
Schedule Variance ($SV = EV - PV$)	-\$80,000.00
Schedule Performance Index ($SPI = EV/PV$)	0.84



<https://www.spmbook.com/downloads/slides/pdf/C03.08-09-ExecutionMonitoringControl.key.pdf>

Interpreting CPI and SPI



Performance Measures		Schedule		
		$SV > 0 \text{ \& } SPI > 1.0$	$SV = 0 \text{ \& } SPI = 1.0$	$SV < 0 \text{ \& } SPI < 1.0$
Cost	$CV > 0 \text{ \& } CPI > 1.0$	Ahead of Schedule Under Budget	On Schedule Under Budget	Behind Schedule Under Budget
	$CV = 0 \text{ \& } CPI = 1.0$	Ahead of Schedule On Budget	On Schedule On Budget	Behind Schedule On Budget
	$CV < 0 \text{ \& } CPI < 1.0$	Ahead of Schedule Over Budget	On Schedule Over Budget	Behind Schedule Over Budget

https://mosaicprojects.com.au/WhitePapers/WP1081_Earned_Value.pdf

Activity W08#4 -



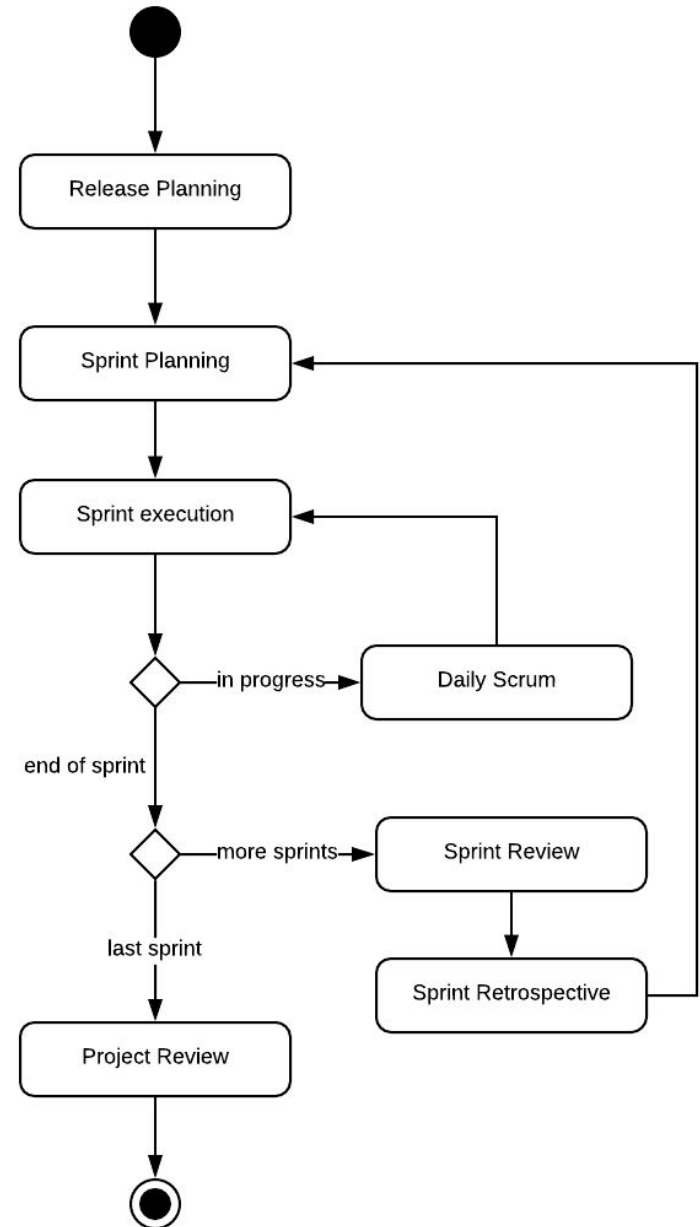
Earned value reporting exercise

Month #	Planned Value	Actual Cost	%Progress	Earned Value
1	Rs. 2,00,000	Rs. 2,20,000	100%	
2	Rs. 1,50,000	Rs. 1,10,000	100%	
3	Rs. 1,00,000	Rs. 80,000	90%	
4	Rs. 1,50,000	Rs. 1,20,000	80%	
5	Rs. 1,50,000	Rs. 75,000	50%	
6	Rs. 1,50,000	Rs. 0	0%	
7	Rs. 1,50,000	Rs. 0	0%	
8	Rs. 1,00,000	Rs. 0	0%	
9	Rs. 1,00,000	Rs. 0	0%	
10	Rs. 1,00,000	Rs. 0	0%	
Budget at completion				

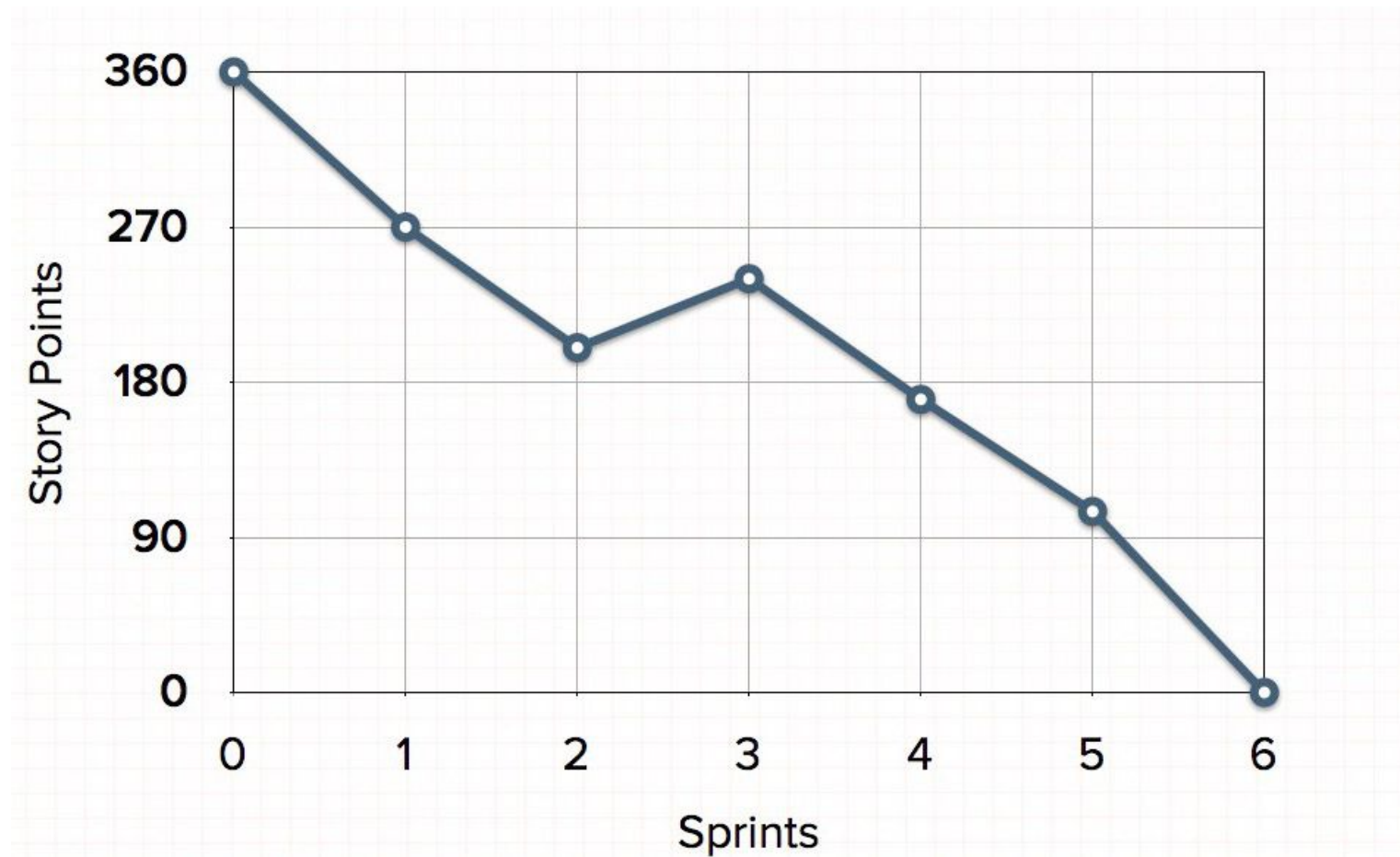
- Compute the earned values for the first five months of this project.
- Calculate the cost and schedule variances and performance indices.
- Explain your interpretation of project status based on the indices calculated in (b).
- Based on the schedule performance indices, by when the project is expected to be completed.

Monitoring scrum project progress

- Epic completion
- Velocity
- Burndown charts
- Sprint review and retrospective to adapt or improve
- ...

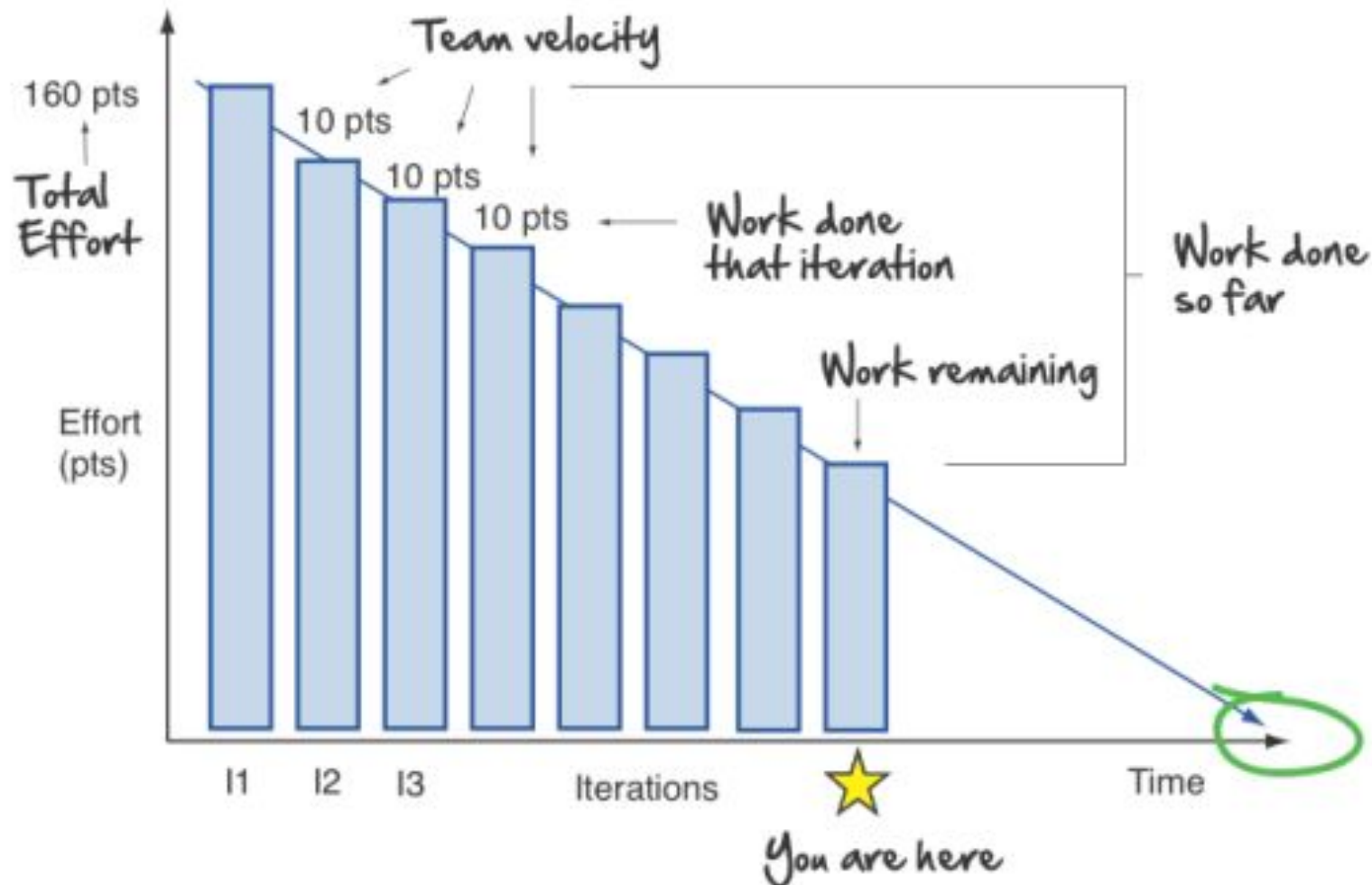


Burndown Charts

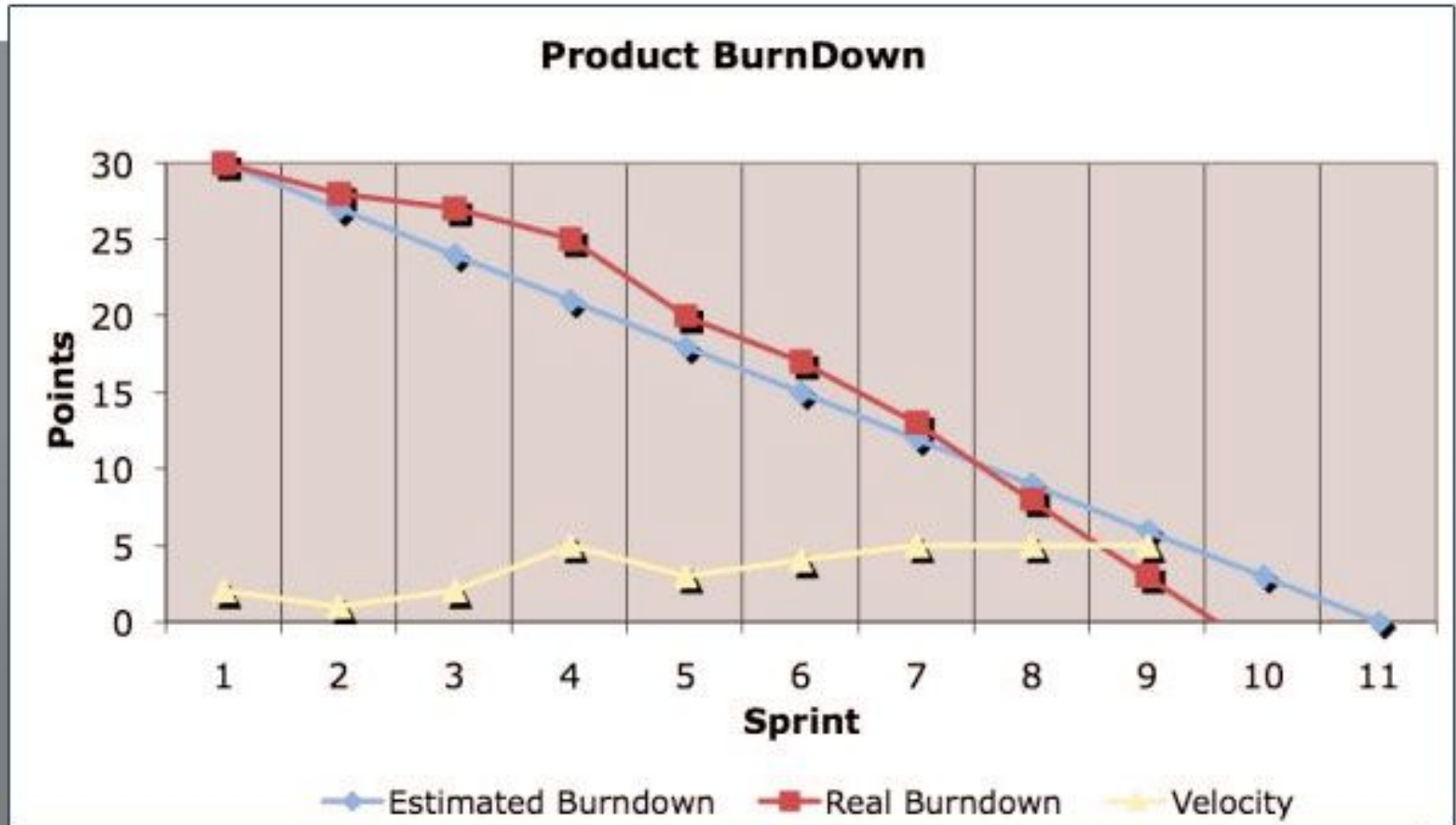


*

Burndown Charts



Burndown Charts



<https://www.scrum.as/academy.php?show=0&chapter=13>

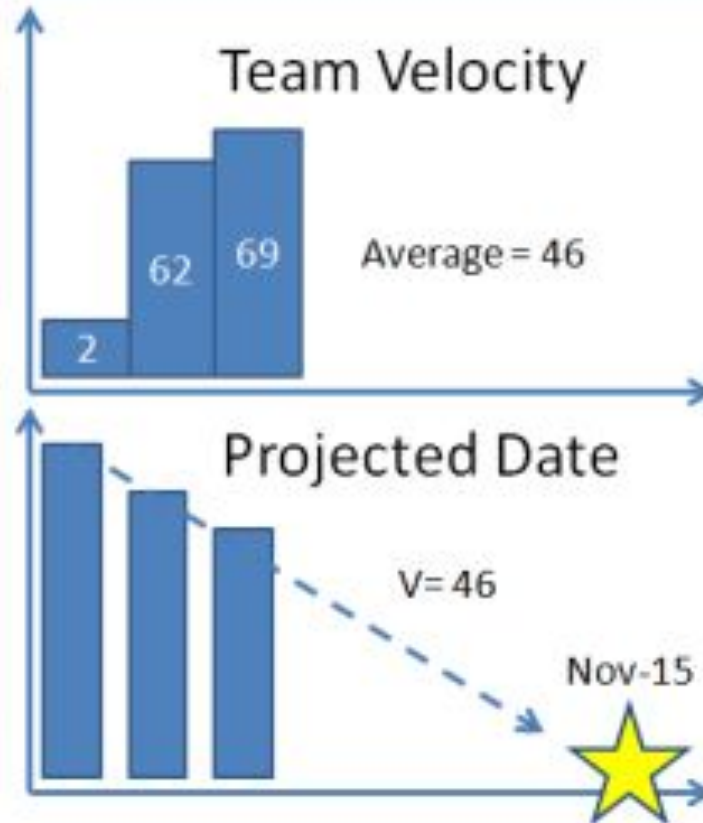


		Status
Budget spent	20%	
Scope delivered	30%	
Estimated date	Nov-1	

Summary
-Project is trending late -Budget is on track -QA bottle neck

Work done
<Story Epic #1> <Story Epic #2> <Story Epic #3>

Risk	Mitigation
- Bottleneck in QA	-Alex pulled in from Prod Support -New hire: Neil due to start Tuesday -Geddy to perform UAT part time
<Thing going bad #2>	<Plan to fix if possible>



Pivotal Tracker reports



Tracker KB Content Overview

Project trends

Velocity and points accepted

21 points

● 16 pts velocity



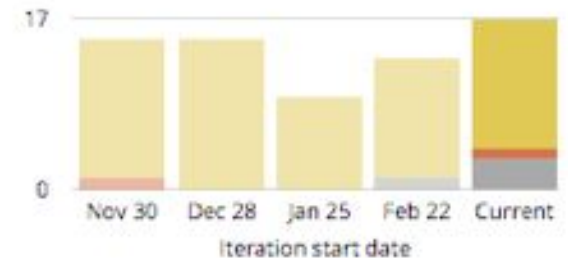
[view report](#)

Burnup

Stories accepted

17 stories

■ 13 features
■ 1 bug
■ 3 chores



[view report](#)

Story cycle time

Typical time between Start and Accept

23 hrs per story

■ 5-iteration avg: 844 hrs



[view report](#)

Rejection rate

Percentage of rejections vs. acceptances and rejections

5.7%

■ 5-iteration avg: 22%



[view report](#)

Pivotal Tracker reports

					view releases report	
Recent Releases					Total points	Points remaining
 All links in KB view burndown 					179	49
					Scheduled	Completion
						Estimated May 15, 2016

					view epics report	
Recent Epics					Point progress	
					Total	Accepted
					In-progress	Unstarted
					Iceboxed	Completion
 Convert to KB framework view report 					3	3
 New images for articles view report 					29	1
					24	4
 Knowledge base content v1.0 view report 					124	90
					16	11
					7	
						Estimated Jun 12, 2016

Pivotal Tracker reports

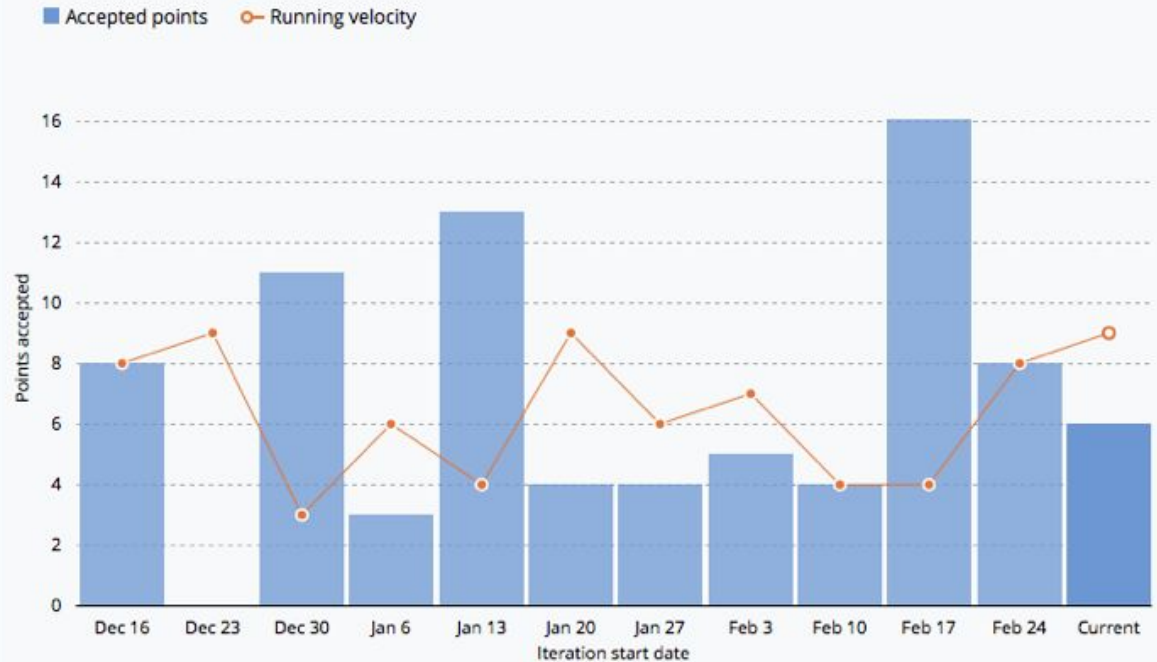
Velocity Chart



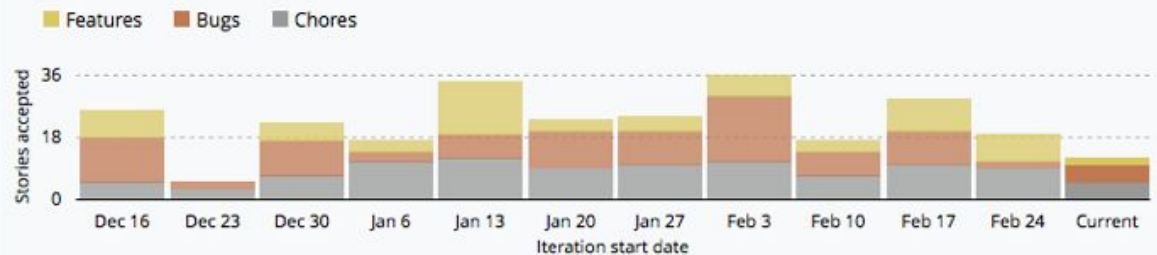
+ filter by epic/label

Points Accepted

Volatility: 66% - Standard Deviation: 4.6pts (based on completed iterations shown)



Accepted Story Type Composition



Can we use gantt charts for monitoring in agile projects?



Summary

	Plan-Driven	Agile
Focus	Scope, cost, schedule adherence	Iterative value delivery, team velocity
Metrics	EVM (CPI, SPI), variance reports	Burn-down charts, velocity, completed stories
Frequency of Control	Periodic (e.g., monthly, quarterly)	Continuous (every sprint or iteration)
Adaptability	Low (scope change is challenging)	High (changes can be incorporated in subsequent sprints)
Challenges	Managing scope creep, budget overruns	Predicting timelines, ensuring consistent velocity

Can we use earned value management technique for agile projects?



BITS Pilani presentation

BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Prof. Narasimha Bolloju
CSIS Department, Hyderabad



SS ZG622

Software Project Management

Lecture No. 9

Organizing and managing teams



- Team organization
- Motivating teams
- Stress management
- Managing attrition

(Hughes et al, Chap 11 of Textbook T2)

Team organization

- Effective structures align with project goals, reduce silos, and improve delivery speed in software projects.
- Challenges: Balancing expertise, scalability, and collaboration in distributed teams.
- Real-world impact: Poor organization leads to delays or scope creep.
- Popular options: Hierarchical, Matrix and Cross-functional

Team organization in plan-driven projects



- Role-based structure with clear hierarchies
- Specialized functional teams
- Sequential workflow and handoffs
- Centralized decision-making
- Formal communication channels
- Focus on documentation and knowledge transfer
- Resource allocation based on project phases

Team organization in agile projects



- Cross-functional teams
- Dedicated and stable teams
- Small, self-organized teams (5-9 members)
- Dynamic roles based on project needs
- Clear role definitions but fluid responsibilities
- Encourage T-shaped skills
- Foster a collaborative and open culture

Activity 9#1: MuxCore Project

Team Organization



Assuming the role of Suresh (the project manager), which one of the following options for structuring your team would you choose? And, why?

- Option 1 – one Scrum team
- Option 2 – one front end team, and one backend & support team)
- Option 3 – one team for Admin subsystem and one team for Sales subsystem
- Option 4 - one feature team (E2E) and one components team

Activity 9#2: MuxCore Project

Team Organization



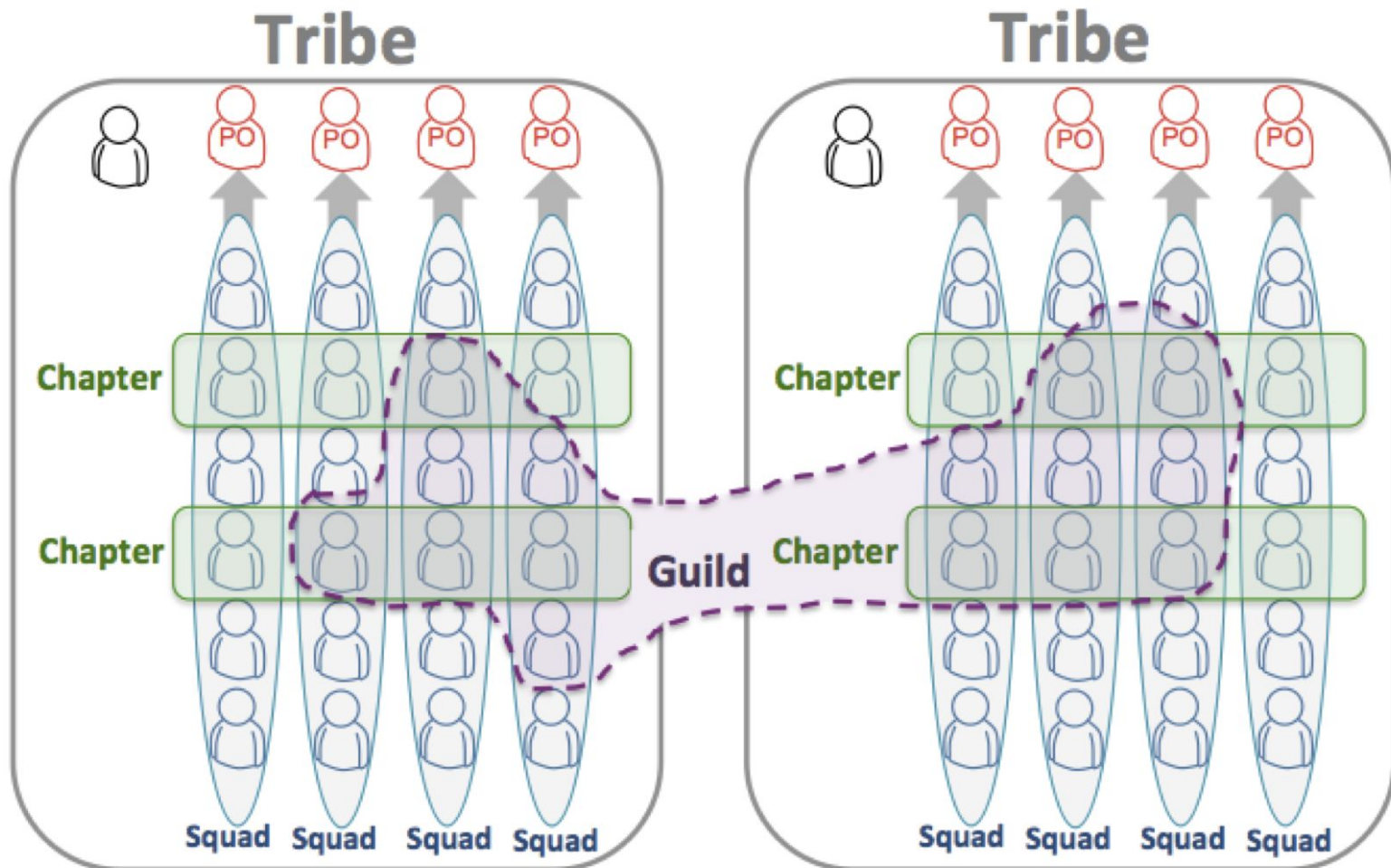
Assuming that you have selected option #3 with you as the PO and Madhu as the SM for both the teams, assign the remaining nine members to these teams:

- Mux-Admin team
- Mux-Sales team

Scaling Agile @ Spotify

with Tribes, Squads, Chapters & Guilds

Henrik Kniberg & Anders Ivarsson
Oct 2012



<https://blog.crisp.se/wp-content/uploads/2012/11/SpotifyScaling.pdf>

Managing motivation

Motivation Theories

- Maslow's Hierarchy of Needs
- Herzberg's Two-Factor Theory

Practical Techniques

- Set achievable goals
- Provide regular feedback
- Recognize and reward accomplishments



[https://www.damienfremont.com/2017/11/11/maslo-
ws-hierarchy-of-needs-for-software-developers](https://www.damienfremont.com/2017/11/11/maslo-ws-hierarchy-of-needs-for-software-developers)



Activity 9#3: Managing Motivation



After the third sprint, you noticed that motivation in both the teams is below expectation (resulting in reduced sprint velocities for fourth and fifth sprints). How will you address this issue?

Managing stress

Sources: tight deadlines, tech debt and constant learning.

Recognizing Burnout: Signs: Fatigue, reduced efficiency

Preventive Measures:

- Encourage work-life balance
- Set realistic deadlines
- Promote time off and mental health support (e.g., relaxation, meditation, mindfulness)

Activity 9#4: Managing Stress



After six sprints, you noticed that Madhu, Rekha and Priya are stressed out. How will you handle this issue?

Managing attrition

- Enhance employee engagement and satisfaction
- Competitive compensation and benefits
- Address workload and burnout
- Develop strong onboarding and training programs
- Promote internal mobility and career growth
- Build a strong employer brand

Activity 9#5: Managing Attrition



As the project manager, you were successful in addressing the motivation problem (resulting in very good sprint velocities) and the stress situation. However, during the 9th sprint you started to feel that Vishal and Priya are quite likely to leave the company.

How will you address this issue?



BITS Pilani presentation

BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Prof. Narasimha Bolloju
CSIS Department, Hyderabad



SS ZG622

Software Project Management

Lecture No. 10

Stakeholder Management

(W10-1)

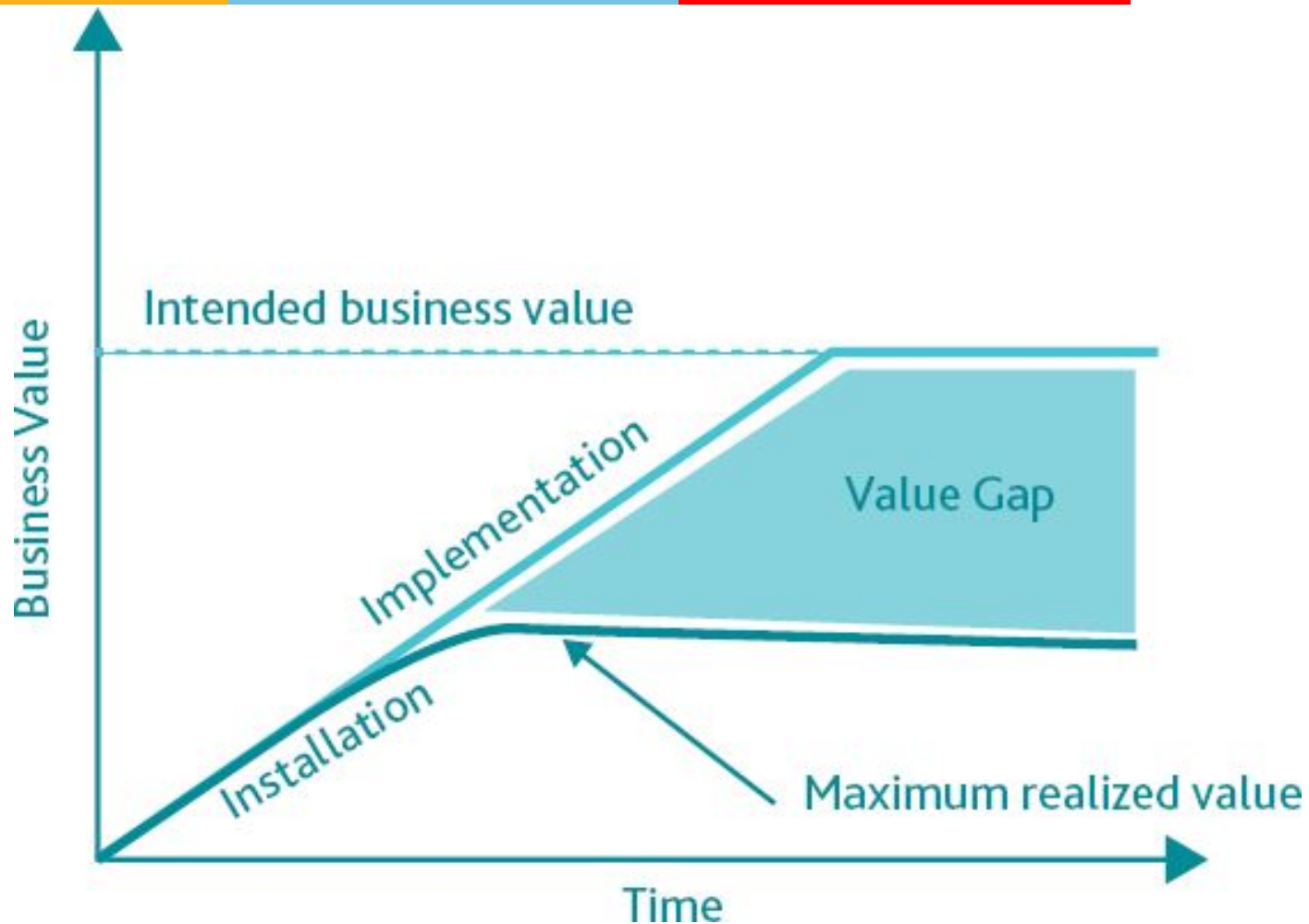
Additional reading:

<https://www.projectmanager.com/guides/stakeholder-management>

<https://www.pmi.org/learning/library/engaging-stakeholders-project-success-11199>

<https://mutomorro.com/tools/mendelow-power-interest-matrix/>

Installation vs. Implementation



<https://www.pmi.org/learning/library/engaging-stakeholders-project-success-11199>

Stakeholder management



- The Project team has a professional responsibility to its stakeholders including customers, the performing organization and the public.
- Stakeholder Management includes
 - the processes required to **identify the people, groups, or organizations** that could impact or be impacted by the project,
 - to **analyze stakeholder expectations and their impact** on the project, and
 - to **develop appropriate management strategies** for effectively engaging stakeholders in project decisions and execution.

Identifying Stakeholders

- Typical stakeholders
 - Owner: who pays for, or “accepts” the system?
 - Customer: who will actually use the system?
 - Project team: who are all responsible for executing the project?
 - Others whose interests are affected by the success/failure of the project, who can affect the outcome of the project, and who may thus become an ally or obstacle to project success.

Note: these roles may not be obvious, and they may shift during the course of the project

Identifying Stakeholders

- **Project Sponsors:** The individuals or groups funding the project, typically interested in the project's ROI and overall alignment with business goals.
- **Customers/Clients:** The primary beneficiaries of the project's outcomes, whose requirements shape the project's deliverables.
- **End-Users:** The people who will directly use the software system, and whose usability concerns are critical.
- **Development Team:** Internal stakeholders responsible for executing the technical aspects of the project.
- **Regulatory Bodies:** Any government or industry entities that impose compliance requirements on the project.

Activity W10#1: Identify ACL Mux Core project stakeholders

Analyzing Stakeholder Needs



- **Understanding Expectations:** Engage with stakeholders to gather insights on what they expect from the project.
- **Requirements Gathering:** Employ techniques like interviews, surveys, and use case analysis to document specific needs.
- **Prioritizing Needs:** Use frameworks like MoSCoW (Must have, Should have, Could have, Won't have) to prioritize requirements.
- **Documenting Requirements:** Ensure clarity by translating stakeholder needs into clear, actionable requirements documented in a formal requirements specification document.

Power-interest matrix



<https://mutomorro.com/tools/mendelow-power-interest-matrix/>



Power-interest matrix

Quadrant	Description	Strategies & Tips
High Power, High Interest (Manage Closely)	Influential stakeholders deeply invested; they drive or block success.	Engage actively—invite to sprint reviews for input; use dashboards for real-time KPI updates. In dev roles, co-create user stories to align on priorities like API integrations.
High Power, Low Interest (Keep Satisfied)	Powerful but disengaged; satisfy to avoid interference.	Meet needs efficiently—provide executive summaries; persuade with benefits like cost savings. For maintenance pros, escalate via quick demos to highlight scalability wins.
Low Power, High Interest (Keep Informed)	Enthusiastic but limited sway; inform to leverage support.	Communicate regularly—share progress via Slack/stand-ups; encourage feedback in retros. In software contexts, use polls for sprint input to boost motivation without overload.
Low Power, Low Interest (Monitor)	Minimal impact/concern; watch for changes.	Minimal effort—periodic emails or newsletters; monitor via surveys. For agile teams, flag if interest spikes (e.g., during rollouts) to prevent passive resistance.

Activity W10#2:

Place the stakeholders of ACL Mux Core project on Power-Interest matrix

Communication with Stakeholders



- **Choosing Communication Channels:** Determine the best methods (email, meetings, reports, etc.) for communicating with different stakeholders based on their preferences and importance.
- **Regular Updates and Meetings:** Set up periodic reviews to ensure stakeholders are aware of project progress and can give input.
- **Sharing Progress Reports:** Provide stakeholders with key metrics like project status, milestones, risks, and issues.
- **Feedback Mechanisms:** Create channels for stakeholders to provide feedback, ensuring they feel heard and engaged throughout the project.

Managing Expectations of Stakeholders



- **Aligning Goals:** Ensure project objectives are clearly communicated to stakeholders to align their expectations with realistic deliverables.
- **Scope Management:** Manage changes in project scope effectively by maintaining open communication channels and involving stakeholders in decision-making.
- **Clear Success Criteria:** Define and agree upon specific, measurable success criteria that will be used to evaluate the project.
- **Handling Conflicts:** Be prepared to address conflicts between stakeholders through negotiation, mediation, and compromise.

Managing Stakeholder Conflicts



- **Identify Conflicting Interests:** Recognize when stakeholders have diverging priorities, such as differing expectations for deadlines or project features.
- **Resolve Conflicts Through Communication:** Openly discuss concerns and mediate solutions that balance different stakeholder needs.
- **Escalation Procedures:** If conflicts can't be resolved at the project level, escalate them to higher management for arbitration.
- **Prioritization of Stakeholder Influence:** Use a stakeholder analysis matrix to assess and prioritize the influence and interest of stakeholders, for helping to resolve conflicts effectively.

Activity W10#3: How do you plan to manage expectations and communicate with ACL Mux Core project stakeholders?

Change Management

(W10-2)

Types of change management

- Managing process changes in organization (*out of scope*)
- Managing change from existing to new software system (*discussed as part of stakeholder management*)
- Managing changes to the software system as it is being developed
- Managing changes to an existing software system
- Managing transition to agile software development (*process model change; to be covered later*)

Topics



- Configuration management
 - **Change management**
 - *Version management*
 - *System building*
 - *Release management*
- Change management
 - Change management process
 - Change request form (example)
 - Change control board
 - Continuous integration

Change management in plan-driven process models

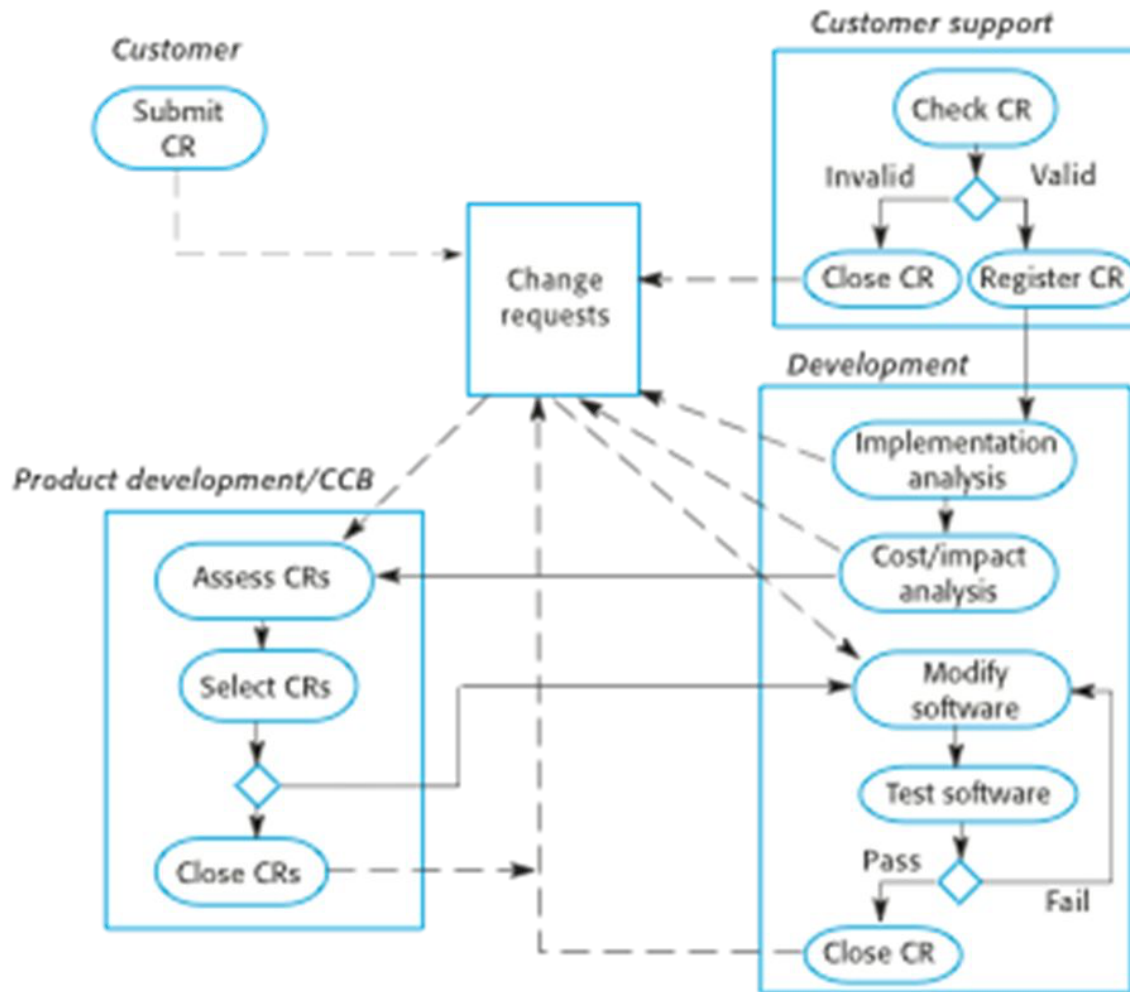


- Change management process
- Change request form (template)
- Change control board (CCB)

Change management process - an overview



Change management process in detail



Chapter 25, Somerville, I. (2007). *Software engineering*. Addison-Wesley.

Activity W10#4: Analyze CRs and Approve/Deny



- CR1: Sales report columns (change) – The daily sales report includes only the total sales of each movies on each day of screening. This report should also contain a spit of the total sales across screens. Such a report will help the manager in deciding on the number of shows for a given movie.
- CR2: Counter staff scheduling (bug) – The draft schedules suggested by the system is infeasible because some staff are assigned to more that one counter for certain durations and it does not take into account the maximum hours per day and/or week. This must be fixed asap.
- CR3: Loyalty card for customers (new) – We need to implement loyalty functionality (registration, issue of electronic card, point scheme, discounted prices, priority seating, etc.) to improve our customer service.

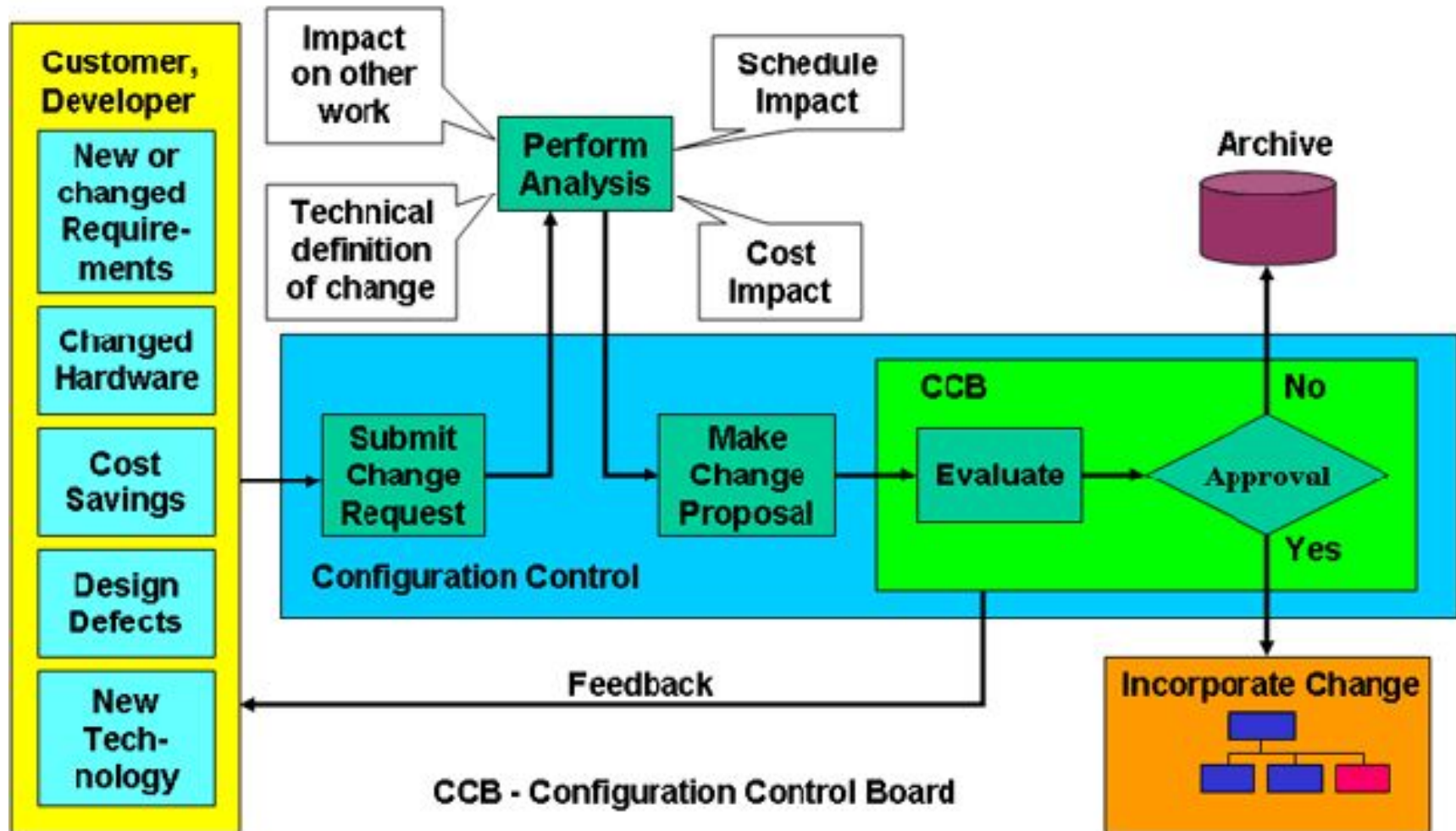
Estimate the effort required, possible impact(s) of the change on the system (specific parts), and whether you approve or deny the CR.

Typical Change Request Form

Change Request Form Template

Project Name	<i>Name Of Project</i>		
Requested By	<i>Name Of Requestor</i>	Date	
Request No	<i>Request Number</i>	Name Of Request	
Change Description	<i>This slide is 100% editable. Adapt it to your needs and capture your audience's attention.</i>		
Change Reason	<i>This slide is 100% editable. Adapt it to your needs and capture your audience's attention.</i>		
Impact Of Change	<i>This slide is 100% editable. Adapt it to your needs and capture your audience's attention.</i>		
Proposed Action	<i>This slide is 100% editable. Adapt it to your needs and capture your audience's attention.</i>		
Status	In Review	Approved	Rejected
Approval Date	<i>This slide is 100% editable. Adapt it to your needs and capture your audience's attention.</i>		
Approved By	<i>This slide is 100% editable. Adapt it to your needs and capture your audience's attention.</i>		

Configuration or Change Control Board



http://www.chambers.com.au/glossary/configuration_control_board.php

Communication with Stakeholders



- Responsible for
 - Evaluating and approving or disapproving proposed changes to a system
 - Prioritizing the incorporation of approved changes
 - Scheduling the changes for forthcoming releases.
- In some projects the CCB may also be responsible for verifying that approved changes are implemented.

Change Control Board

- Medium to large projects may have more than one CCB.
 - an external CCB comprising users, developers and marketing people is formed to deal with changes that will impact the customer.
 - An internal CCB comprising developers and technical managers is formed to deal with changes to the software system, or impact costs and delivery dates.

Change management in agile process models



- Changes are welcome and opportunities for improvement based on value
- No formal approval is required for the project team (escalation required for changes that result in enterprise-wide impact)
- The PO, Scrum team, and key stakeholders review, refine, and prioritize backlog items to ensure they're clear, estimated, and aligned with project goals



BITS Pilani presentation

BITS Pilani

Pilani | Dubai | Goa | Hyderabad

Prof. Narasimha Bolloju
CSIS Department, Hyderabad



SS ZG622

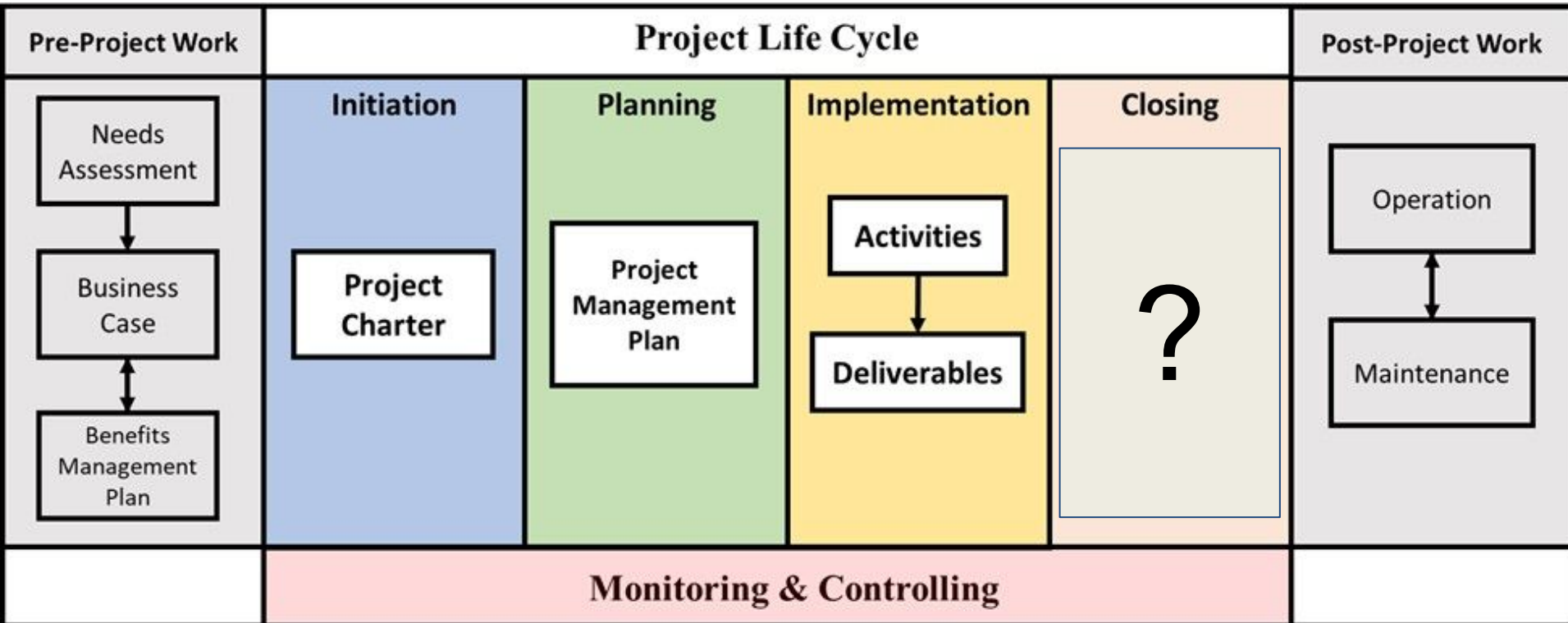
Software Project Management

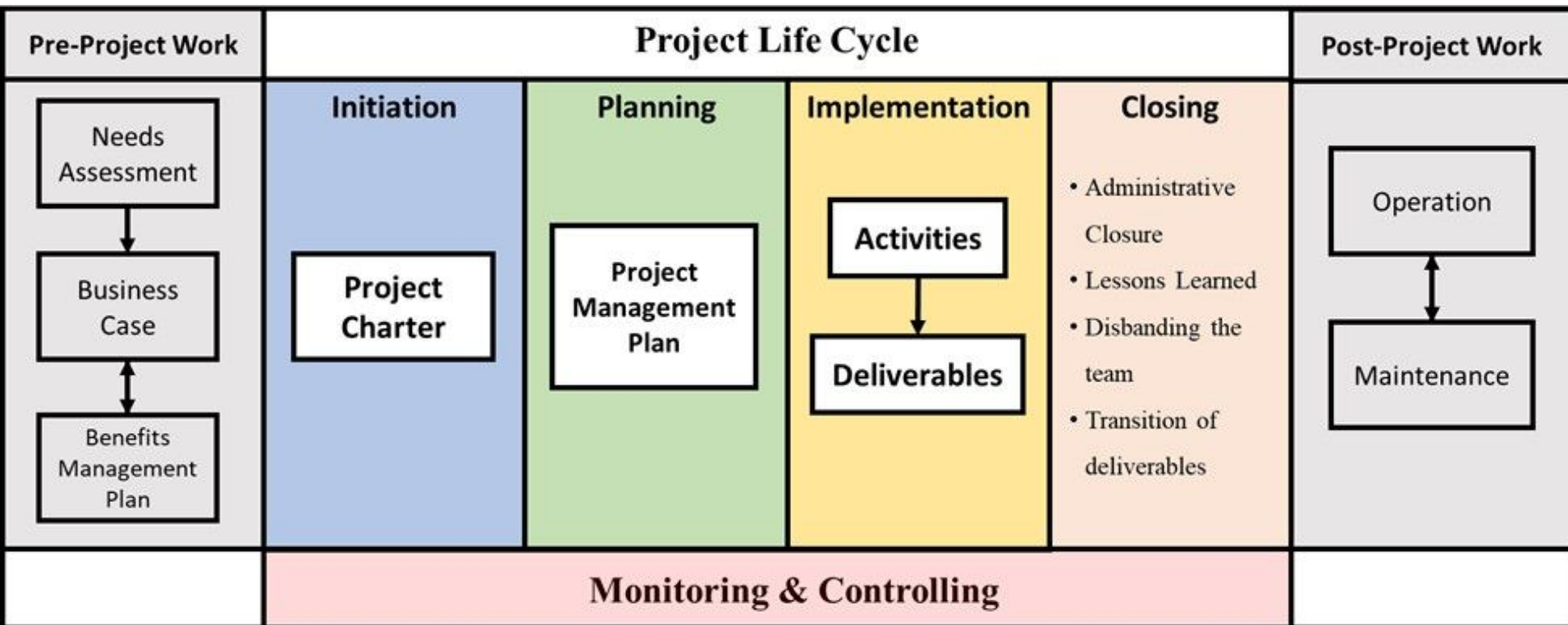
Lecture No. 11

Project closure



Activity W11#1





Project closure - outline

- Project closure checklist
- Reasons for project closure
- Why projects are not properly closed
- Project closure process
- Some related issues

Reference: Chapter 14 of T2 (textbook by Hughes, Cotterell & Mall)

Checklist to Close a Project



Deliverables are handed off and signed off by stakeholders



All documents are organized and signed by the appropriate person



Financials: pay off all invoices and close out all project-related contracts



Finalize all project reports and archive them for future reference



Assign a transition support person to shepherd the project after completion



Release or reassign project resources, which includes people and equipment



Don't neglect to celebrate the a successful project, everyone deserves it



With project management software, project closure is more efficient

Reasons for project closure



- Successful completion of project OR
- Prematurely terminating a project
 - Lack of resources
 - Changed business need
 - Perceived benefits no longer valid
 - Changes to regulatory policies
 - Technology became obsolete
 - Risk is unacceptable

https://www.smartsheet.com/sites/default/files/2021-04/IC-Software-Project-Closure-Report-11064_PDF.pdf

Why projects are not properly closed?



- Lack of interest by the project team
- Underestimation of know-how and implicit knowledge
- Emotional factors
- Indecision regarding project closure

Project closure process



- Getting client acceptance of the final product
- Completing and archiving project deliverables
- Preserving project know-how
- Performing a financial closure
- Performing post-implementation project review
- Preparing post-implementation review report
- Transferring ownership
- Celebrating success
- Releasing staff

Project closure - Other related issues



- Final integration checks
- Customer satisfaction
- Handing over to maintenance team
- Handling transition risks
- Ensuring operational continuity
- ...

Activity W11#2



What are the differences between waterfall project closure and agile project closure?

Maintenance



Software maintenance



- Ongoing maintenance is critical to post-closure and long-term success of the software system.
- Lack of planning can lead to increased downtime, loss of customers, and higher costs.



Elements of a maintenance plan



- **Transition management:** Involves handing over to a maintenance team or ensuring the delivery team provides long-term support.
- **Stakeholder involvement:** Engage users and technical teams early to understand what post-deployment support is necessary.
- Incorporating **maintenance planning into the project closure** ensures a smoother handover.

Maintenance as part of project closure



- **Documentation:** Comprehensive system and user documentation.
- **Resource Planning:** Skilled personnel and tools for future updates.
- **Cost Estimation:** Setting aside budget for future enhancements or fixes.
- **Support Infrastructure:** Setting up help desks, support teams, and knowledge bases.

Benefits of maintenance planning

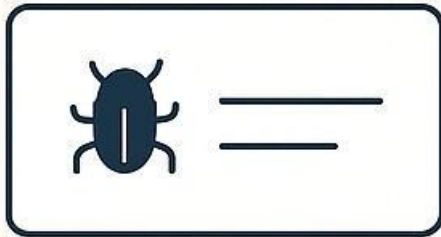


- Increases system lifespan and reduces failure risk.
- Lowers long-term costs by preventing major breakdowns.
- Integrated long-term maintenance planning reduces operational disruptions.

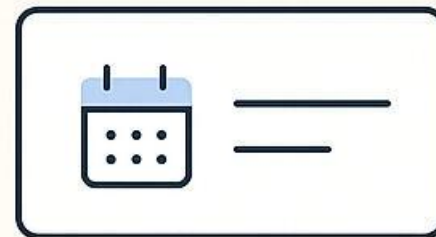
Prioritization of maintenance tasks



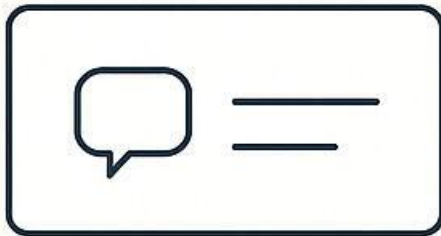
Urgent and important



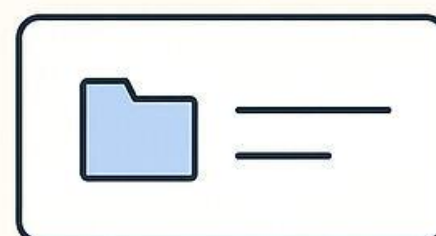
important but not urgent



Urgent but not important



Neither urgent nor important



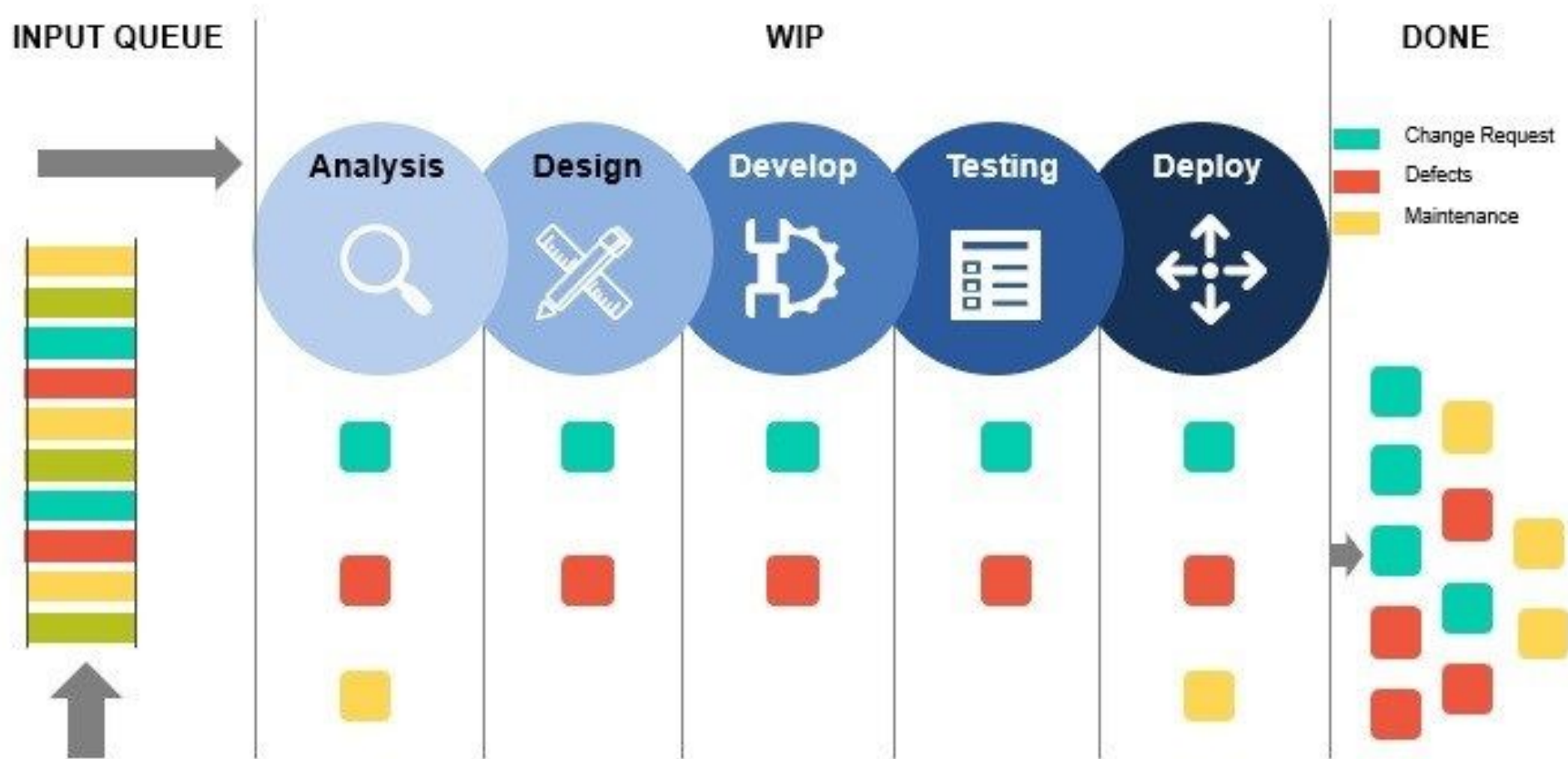
<https://www.breeze.pm/blog/eisenhower-matrix>

Activity W11#3



Can we use Scrum for
software maintenance?
Why or why not?

Kanban for software maintenance



<https://hangoutagile.com/what-is-agile-kanban/>

Agile methods for software maintenance



- Scrum or Kanban can be used for maintenance work, regardless of whether the original system was developed using waterfall, spiral, or even another agile approach
- Focus on iterative delivery and adaptability, which aligns perfectly with the unpredictable, ongoing nature of maintenance tasks like defect resolution, enhancements, and compliance updates

Scrum for structured maintenance



- Ideal for semi-predictable cycles (e.g., monthly compliance releases).
- Use 1-2 week sprints to plan a backlog of tickets, hold daily stand-ups for blockers, and retrospectives to refine processes.
- Works best with a dedicated team; unplanned emergencies might disrupt sprints, so allocate "buffer" capacity.
- Teams maintaining enterprise apps often adapt Scrum by shortening sprints to handle urgent support

Kanban for flow-based maintenance



- Perfect for high-variability work, like ad-hoc bug fixes or user-reported issues.
- Visualize tasks on a board (e.g., "To Do," "In Progress," "Done") with WIP limits to prevent overload—no fixed iterations, just continuous flow.
- Pull tickets as capacity allows, making it easier to integrate with tools like Jira for legacy system monitoring.



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

SS ZG622:

Software Project Management

Session # 12

Prof. Narasimha Bolloju, BITS-Pilani, Hyderabad Campus

Quality management system



Quality management system (QMS)



- QMS is a structured framework that defines processes, policies, and procedures to ensure that products or services meet predefined quality standards and customer expectations.
- QMS focuses on delivering high-quality software through systematic processes, from requirement gathering to maintenance.
- The goal of a QMS is to meet customer requirements, to continuously improve processes to enhance efficiency, reduce defects, and improve the product life cycle.

Role of the project manager in QMS



- Ensuring compliance with standards and procedures
- Coordinating with quality teams
- Managing stakeholder expectations

Key components of a QMS



- Quality planning
- Quality assurance
- Quality control
- Continuous improvement

Project manager's role in quality planning



- Setting quality objectives aligned with project scope
- Developing a quality management plan
- Defining roles and responsibilities for quality assurance

Project manager's role in quality assurance



- Establishing QA processes and metrics
- Regular audits and reviews
- Communicating quality standards to the team

Project manager's role in quality control



- Monitoring project deliverables for quality
- Inspecting deliverables and identifying defects
- Ensuring that changes are made to meet quality requirements

Risk management and quality



- Identifying risks that impact quality
- Mitigating risks through proper quality controls
- Using risk-based thinking in quality planning

Common Challenges in Quality Management



- Balancing time, cost, and quality
- Resistance to process adherence from teams
- Managing changing customer requirements

Continuous improvement by implementing Kaizen



- Kaizen is a Japanese term that translates to "continuous improvement."
- The core idea of Kaizen is that small, incremental changes, when applied consistently, lead to significant improvements over time.
- Kaizen encourages all team members to contribute to the enhancement of processes, systems, and products.

Kaizen in software development (example)



- **Improve Code Quality:** By regularly reviewing code and suggesting improvements, technical debt can be reduced over time.
- **Streamline Testing:** Kaizen can optimize the QA process by introducing more automated testing, reducing the need for manual interventions.
- **Reduce Development Time:** Continuous process improvements can help reduce inefficiencies in the software development lifecycle (SDLC), such as delays in hand-offs between teams or bottlenecks in the testing phase.

Tools for Kaizen in projects



PDCA Cycle (Plan-Do-Check-Act):

- Plan: Identifying an issue and developing a solution.
- Do: Implementing the solution.
- Check: Analyzing the results.
- Act: Standardizing the improvement if successful or revising the approach if necessary.

Kanban Boards: Kanban boards help teams visualize workflows and spot bottlenecks or areas for improvement.

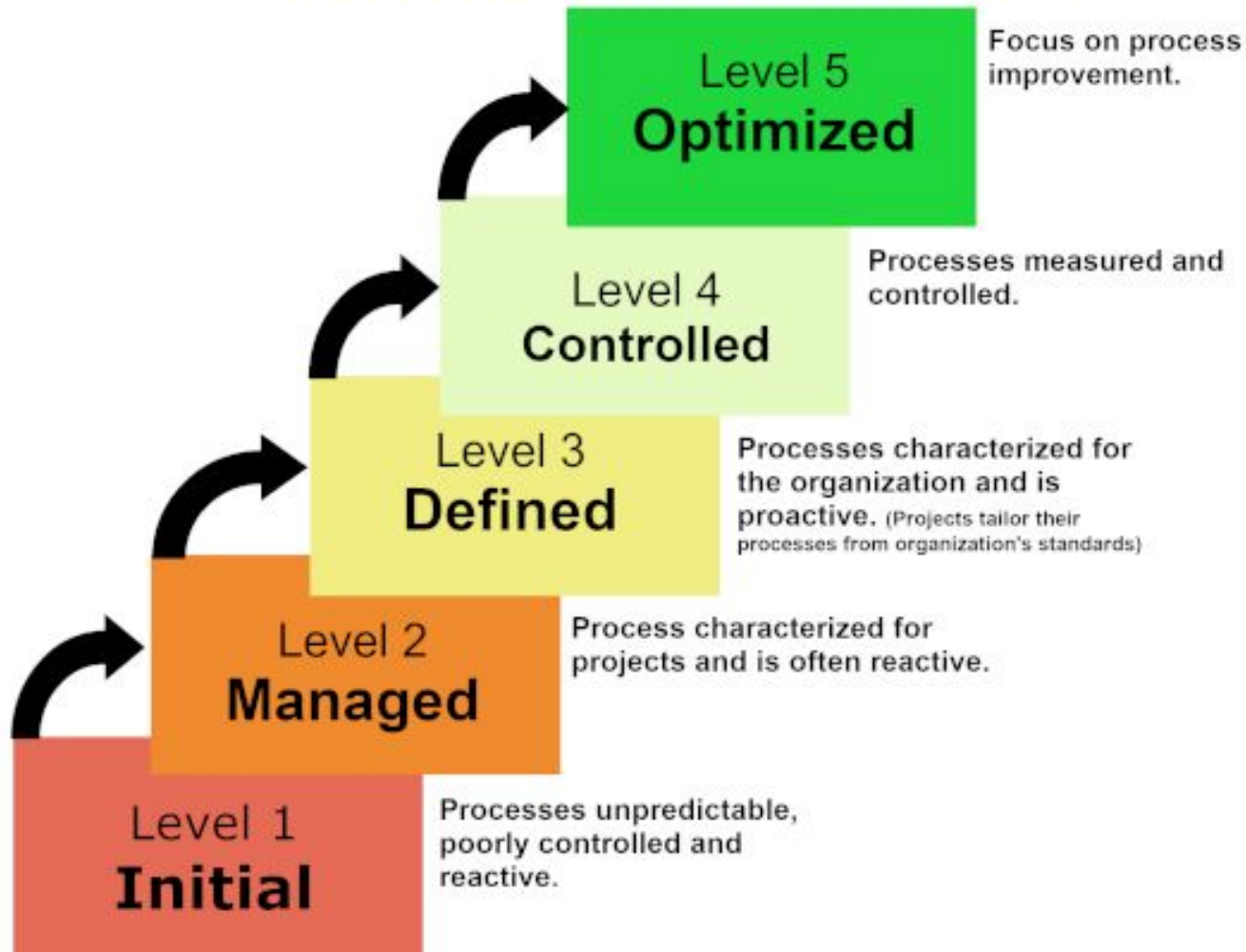
Root Cause Analysis (e.g., 5 Whys): When a problem arises, Kaizen encourages deep analysis by asking "why" five times, aiming to identify the root cause of inefficiencies or issues.

Challenges in implementing Kaizen



- Resistance to Change: Team members may resist continuous change, especially if improvements require new ways of working.
- Time and Resources: Kaizen demands ongoing attention and can sometimes slow down immediate project work in favor of long-term improvements.

Capability maturity model integration

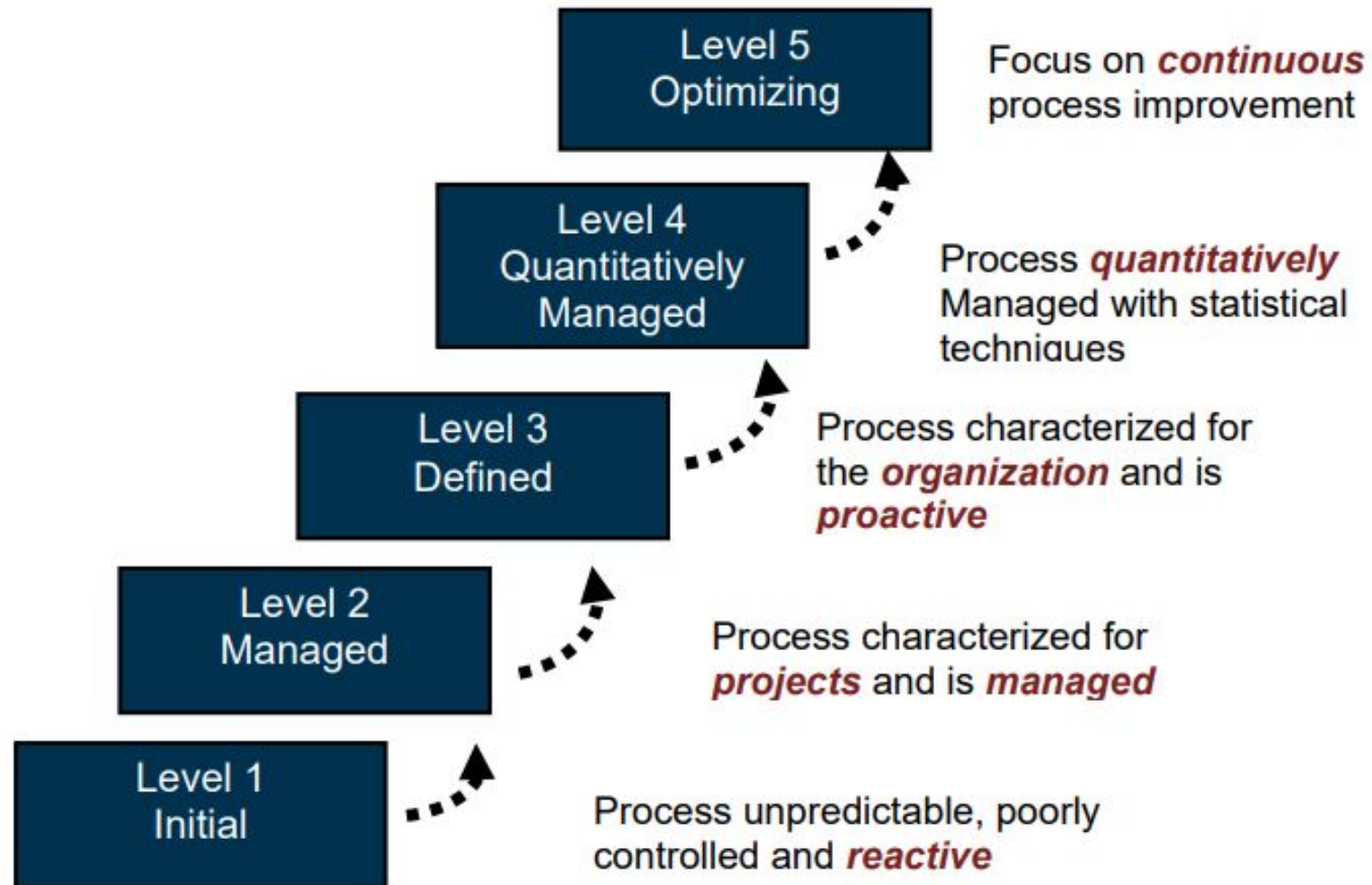


Capability Maturity Model Integration



- CMMI is a process improvement framework that provides organizations with a structured approach to improve their development processes over time
- CMMI helps organizations mature these processes through different levels, ensuring they are repeatable, measurable, and optimized

CMMI Levels



CMMI Levels for Software Testing (an example)



CMMI Level	Testing Focus	Typical Activities
Level 1: Initial	Ad hoc, reactive testing	Testing lacks structure, primarily conducted when issues arise. Testing activities are usually last-minute, focused on detecting defects post-development.
Level 2: Managed	Basic test planning and execution	Testing is introduced as a defined stage in the development process. Test plans are created, basic metrics collected, and some test cases designed. However, testing is not fully integrated with development processes.
Level 3: Defined	Standardized, repeatable testing processes	Comprehensive test planning occurs, with standardized processes across projects. Test cases are created early, and there is integration between testing and development teams. Processes ensure consistency, and defect tracking begins to improve.
Level 4: Quantitatively Managed	Data-driven testing practices	Testing processes are monitored and controlled using quantitative metrics. Focus shifts to defect prevention through root cause analysis and statistical process control, resulting in reduced defect injection.
Level 5: Optimizing	Continuous improvement and automation	Testing is highly optimized, automated wherever feasible, and aimed at continuous improvement. Teams use metrics to refine processes, aiming for defect prevention and process efficiency. Test processes are proactive, anticipating issues rather than merely reacting.

Role of project manager



- Ensuring adherence to quality standards and practices
- Driving process improvements through periodic reviews and planning
- Overseeing the metrics used to measure success



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

SS ZG622:

Software Project Management Contact Session # 13

Prof. Narasimha Bolloju, BITS-Pilani, Hyderabad Campus

Contract management



- Outsourcing/contracting software projects
- Project manager's role
- Challenges and keys to success
- Steps for outsourcing
- Contacting options
- Project manager's responsibilities (client side vs vendor side)

Outsourcing/Contracting



- Software project contracting is a business arrangement where a company or individual hires a third party to design, develop, and maintain software applications.
- Software development contracts define project scope and cost, intellectual property rights, confidentiality, success metrics, timeframes, and warranties.

Project manager's role (Client-side)



- **Responsibility** – Assessing project requirements, risks, and constraints to select the best contract type.
- **Decision Factors** - Complexity of requirements, budget constraints, timeline, and vendor capabilities.

Challenges



- **Scope Creep:** Risk of undefined requirements expanding the project scope.
- **Budget Control:** Risk of overspending, especially in T&M and hybrid models.
- **Compliance and Regulatory Requirements:** Ensuring the project meets industry standards and regulations.



Keys to success

- Understand company goals and objectives; is outsourcing the right solution?
- Create an outsourcing plan
- Define the software requirements in detail
- Create a Request for Proposal (RFP)
- Estimate your project's cost and schedule before finalizing your RFP
- Obtain sufficient budget and management resources to assure success

<http://www.andrew.cmu.edu/course/67-325/OutsourcingHB.pdf>



Keys to success (contd.)

- Select a qualified vendor
- If no vendor appears qualified, switch to a two-phase acquisition model or bring the project in-house
- Create the outsourcing contract with care, and have it reviewed
- Actively manage the outsourced project
- Use outside experts when needed

<http://www.andrew.cmu.edu/course/67-325/OutsourcingHB.pdf>

Steps for successfully managing outsourcing projects



1. Specify requirements in detail
2. Make an explicit decision to outsource
3. Obtain adequate resources
- 4. Select a qualified vendor**
- 5. Write a contract**
6. Build an in-house management team to monitor the project to completion

<http://www.andrew.cmu.edu/course/67-325/OutsourcingHB.pdf>

Proposal evaluation criteria - example

Criteria	Max Points	Vendors			
		V1	V2	V3	V4
Project management capability	10	3	8	5	3
Application-area experience	10	1	5	3	10
Technical-area experience	10	8	10	3	10
General technical capability	10	5	10	10	7
Organizational strength	10	5	9	10	3
Technical design approach	10	5	8	8	10
Technical methodologies	10	5	8	7	8
Requirements management approach	10	4	10	7	9
Technical documentation	5	3	5	4	5
Engineering data management	5	1	5	4	5
Configuration management approach	5	2	5	3	5
Quality assurance approach	5	1	5	4	4
Total points	100	43	88	68	79
Overall ranking	-	4	1	3	2

W13 - Activity #1: Vendor evaluation criteria



Assuming that ACL management has decided to outsource the Mux-Core project instead of developing in-house as planned earlier, identify FOUR or FIVE elements or components of the evaluation criteria to be used for ranking the proposals received from different vendors for Mux-Core software development.

Contracting options

- **Time and materials contracts** – hourly billing rate; risk of cost overrun; used when requirements cannot be precisely defined and/or client needs flexibility
- **Cost plus (or hybrid) contracts** – a fixed amount + hourly billing rate (or function point rate); incentives/penalties can be included
- **Fixed price contracts** – used when requirements are clear (based on function point estimate); less risk for client

<http://www.andrew.cmu.edu/course/67-325/OutsourcingHB.pdf>

Contracting options for Agile projects



- **Time and materials contracts** – Flexibility, control over budget and client involvement
- **Dedicated team model** – Team continuity, long-term collaboration, and cost efficiency

<http://www.andrew.cmu.edu/course/67-325/OutsourcingHB.pdf>

Two-phase acquisition

- Phase 1: requirements development and planning work (10-20% resources; 15-30% time)
- Phase 2: detailed development work
- Benefit: more control over vendor selection
- Used when
 - Requirements are not well-defined/expected to change
 - Client wants a formal checkpoint
 - Client wishes to share risk with the vendor

Dedicated development teams



- A dedicated development team is, usually, an offshore software development strategy hired to work exclusively on one or more projects.
- Unlike outsourcing, such a team becomes an extension of the company and collaborates closely with the in-house team.
- Cost includes monthly salaries + administrative expenses
- Such teams are used for long term projects where scope is not clearly defined.

W13 - Activity #2:

Contracting option



Assuming that ACL's management has decided to outsource the Mux-Core project instead of developing in-house as planned earlier, and then selected a vendor, suggest a suitable contracting option.

Client side project manager's responsibilities



- Role in contract selection (based on assessment of requirements, risks, and constraints)
- Responsibilities in fixed-price contracts (scope management, risk mitigation, monitoring & control)
- Responsibilities in time and material contracts (flexibility management, performance monitoring, cost control)
- Responsibilities in hybrid contracts (balancing fixed and variable elements, ensure flexibility and control, stakeholder communication)
- Common responsibilities: requirements clarity, quality assurance, vendor relationship management)

Client's role in managing agile projects



- Define clear roles and responsibilities (dedicated PO, stakeholder engagement)
- Emphasize strong communication and collaboration
- Set clear and flexible requirements
- Commit to regular reviews and testing
- Manage expectations and prepare for scope adjustments
- Ensure proper resource allocation
- Continuous improvement through retrospectives

Vendor side project manager's responsibilities



- Client relationship management (building trust, understanding requirements, regular communication)
- Scope and requirements management (scope definition, change management, requirements validation)
- Resource and team management
- Quality assurance
- Team and budget management
- Risk management and issue resolution
- Contract compliance
- Post-delivery support and maintenance

W13 - Activity #3: Outsourcing project risk management



Suggest some risk management options for outsourced projects.

Additional references



Examples of Contract Management Systems

<https://www.icertis.com/demo/>

<https://www.agiloft.com>

See <https://www.softwareadvice.com/contract-management/>
for a list of top contract management systems from different
vendors

[Software Development Contracts: All You Need to
Know + Templates](#)

[Top 10 Mistakes and Risks to Avoid When Outsourcing
Software Development - Waverley](#)



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

SS ZG622:

Software Project Management Contact Session # 14

Prof. Narasimha Bolloju, BITS-Pilani, Hyderabad Campus

Project Evaluation

- Portfolio management
- Business case
- Financial analysis for project evaluation

Portfolio management

Portfolio management



- **Portfolio:** Collection of projects and programs managed to achieve strategic objectives.
- **Program:** Group of related projects managed in a coordinated way.
- **Project:** A specific, temporary effort to create a unique product or service.

Portfolio management



- **Top-Down style** - starts with the high-level strategic initiatives, often referred to as strategic buckets, and then works to drive project idea generation for the portfolio until the buckets are appropriately full
- **Bottom-Up style** - starts with the project ideas, allowing them to come from anywhere in the firm and be analyzed to determine the cream of the crop; the strategy is used to measure and analyze the project ideas, determining if they should become part of the project portfolio
- **Blended or mixed style** – combination of the above two styles

<https://www.pmi.org/learning/library/project-portfolio-management-tools-techniques-1050>

Project portfolio management

1. Project portfolio definition

- Create a central record of all projects within an organization
- Must decide whether to have ALL projects in the repository or, say, only ICT projects
- Note difference between new product development (NPD) projects and renewal projects e.g. for process improvement



Project portfolio management

2. Project portfolio management

Actual costing and performance of projects can be recorded and assessed

3. Project portfolio optimization

Information gathered above can be used achieve better balance of projects e.g. some that are risky but potentially very valuable balanced by less risky but less valuable projects

Portfolio management - activities



- Inventory and assessment of applications and IT systems in the organization
- Categorization and prioritization of applications
- Alignment of portfolio with business strategy
- Life cycle management of applications from inception to retirement
- Risk management of the portfolio
- Governance and decision making regarding changes to portfolio
- Performance measurement and reporting about the portfolio

Contribution of various projects to business objectives



	Projects															
Objectives:	Project 1	Project 2	Project 3	Project 4	Project 5	Project 6	Project 7	Project 8	Project 9	Project 10	Project 11	Project 12	Project 13	Project 14	Project 15	Row Scores
Establish Competitive Product Lines	1			2						2			2			7
Maintain Product Competitiveness	1	2			2		1					2				8
Reduce Operating Costs			1	1	1			2		1		1	1		2	10
Increase Capacity/Headcount Ratio				1	1			2					1		2	7
Establish Infrastructure for Growth	1			1			2						2			6
Maintain Infrastructure			2					1	1							4
Growth Through Acquisition	2													2		4
Column Scores	5	2	3	5	4	0	3	5	1	3	0	3	6	2	4	
			= No Contribution (0)													
		1	= Supports (1)													
		2	= Fulfills (2)													

<https://www.pmi.org/learning/library/project-portfolio-management-tools-techniques-1050>

Business and IT Strategy of ACL



- **Business strategy:** achieving increased operational efficiency, effective decision making, and increased revenues through better customer service and operations through standardized business processes
- **IT strategy:** establishing ICT with a set of standardized and integrated application systems for supporting the standardized business processes

W14 - Activity #1: Strategic alignment of the following proposed projects



1. **Intelligent control of air-conditioning:** to save energy by controlling AC in theatre based on the occupancy during a given show; uses seat maps for directing air flow and adjusting temperature using IoT sensors located at various places inside the theatre; ROI in 5 years
2. **Mobile app for food and drink ordering and delivery:** to take food and drinks orders from customers from a given menu, and to deliver the ordered items; ordered items are arranged for efficient delivery; customers can order/pre-order and pay for their favourite items; ROI cannot be quantified

Discuss alignment of these projects with business and IT strategy of ACL

Program management

- One definition:

‘a group of projects that are managed in a coordinated way to gain benefits that would not be possible were the projects to be managed independently’ - Ferns

- Some example programs

- Business cycle programs
- Infrastructure programs
- Research and development programs
- Innovative partnerships

Business case

Business case



A successful project business case should address **a specific business need, assess all alternatives** to lead towards a strong recommendation, **describe what success would be and how it would be measured**, anticipate concerns and questions from senior leadership and those who are asked to provide resources, and **persuade the decision maker** to take the recommended action.

https://uwaterloo.ca/it-portfolio-management/sites/ca.it-portfolio-management/files/uploads/files/project_business_case_template_0.docx

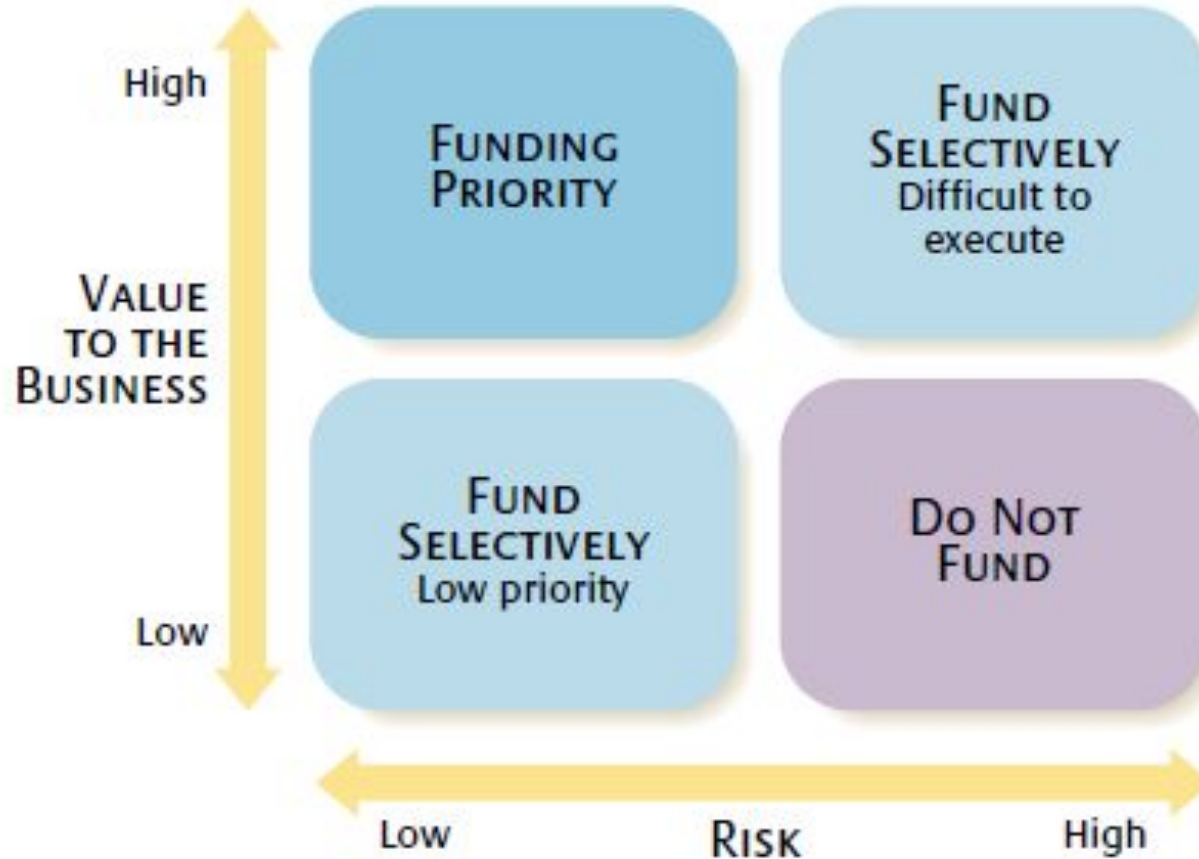
Business case



The purpose of a business case is to clearly outline the **business benefits and strategic alignment** of the proposed project, in order to **justify the expenditures** (budget and people/time) and **seek approval** to proceed

https://uwaterloo.ca/it-portfolio-management/sites/ca.it-portfolio-management/files/uploads/files/project_business_case_template_0.docx

IT project funding decision



<http://www.ucipfg.com/Repositorio/MATI/MATI-08/BLOQUE-ACADEMICO/Unidad-2/4.pdf>

Business case – Typical Contents



- Executive Summary
- Purpose
- Methodology
- Assumptions
- Business Need
 - Problem/Opportunity Description
 - Impact of Problem/Opportunity
 - Benefits
- Alternatives Analysis
- Recommendation
 - Project Description
 - Goals and Objectives
 - Project Performance
 - Organizational Impact
 - Constraints
- Cost Benefit Analysis
- **Strategic Alignment**
- Key Issues and Risks
- Acknowledgements
- Approvals
- Revision History

Financial analysis for project evaluation

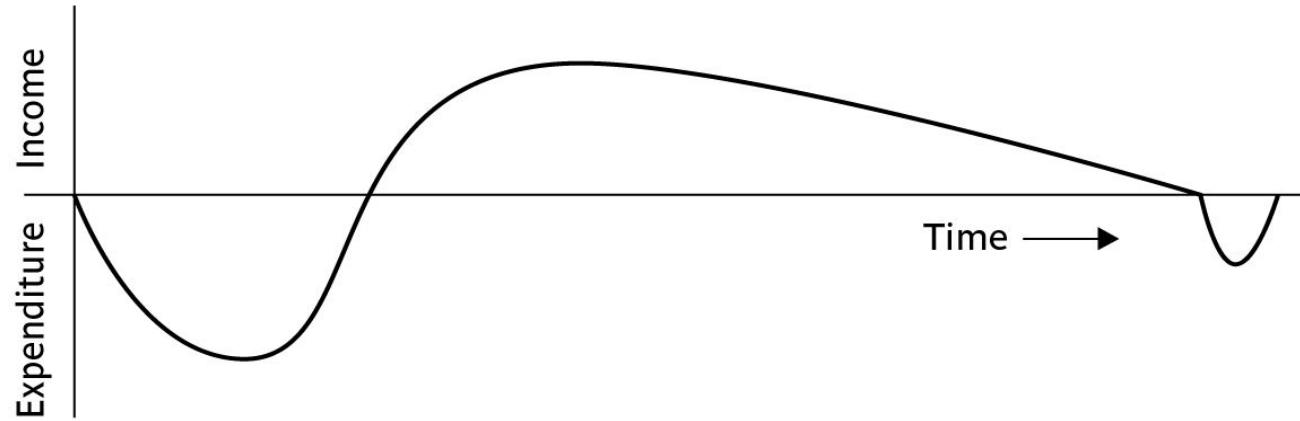
Cost benefit analysis (CBA)

- Types of costs
 - Direct costs: labour, materials and licensing fees, ...
 - Indirect costs: training, support, infrastructure, ...
- Types of benefits
 - Quantifiable benefits: increased revenue, cost savings, ...
 - Qualitative benefits: improved customer satisfaction, brand reputation, loyalty, ..

Cost benefit analysis (CBA)

- Types of costs
 - Direct costs: labour, materials and licensing fees, ...
 - Indirect costs: training, support, infrastructure, ...
- Types of benefits
 - Quantifiable benefits: increased revenue, cost savings, ...
 - Qualitative benefits: improved customer satisfaction, brand reputation, loyalty, ..

Product/system life cycle vs cash flows



- The timing of costs and income for a product or system needs to be estimated.
- The development of the project will incur costs.
- When the system or product is released it will generate income that gradually pays off costs

Net profit

Year	Cash-flow
0	-100,000
1	10,000
2	10,000
3	10,000
4	20,000
5	100,000
Net profit	50,000

- 'Year 0' represents all the costs before system is operation
- 'Cash-flow' is value of income less outgoing
- Net profit value of all the cash-flows for the lifetime of the application

Payback period

Year	Cash-flow	Accumulated
0	-100,000	-100,000
1	10,000	-90,000
2	10,000	-80,000
3	10,000	-70,000
4	20,000	-50,000
5	100,000	50,000



Return on investment (ROI)

$$\text{ROI} = \frac{\text{Total Net Cash Inflows} - \text{Initial Investment}}{\text{Initial Investment}} \times 100$$

In the example given on previous slides:

$$\text{ROI} = (150000 - 100000) / 100000 \times 100 = 50\%$$

Over the five years



Net present value (NPV)

Would you rather I gave you Rs. 100 today or in 12 months (or one year) time?

If I gave you Rs. 100 now you *could* put it in savings account and get interest on it.

If the interest rate was 10% how much would I have to invest now to get Rs.100 in a year's time?

$$\text{NPV} = 100 / (1 + 0.10) = 90.909$$

the **net present value** of Rs.100

Discount factor

$$\text{Discount factor} = 1/(1+r)^t$$

r is the interest rate (e.g. 10% is 0.10)

t is the number of years

In the case of 10% rate and one year

$$\text{Discount factor} = 1/(1+0.10) = 0.9091 \text{ (year 1)}$$

$$\text{Discount factor} = 1/(1.10 \times 1.10) = 0.8294$$

Net Present Value (NPV)

$$\text{NPV} = \sum \frac{\text{Cash Flow}_t}{(1 + r)^t} - \text{Initial Investment}$$

Where:

- Cash Flow_t is the cash inflow in year t .
- r is the discount rate (10% in this example).
- t is the year number.

Applying discount factors

Year	Cash-flow	Discount factor	Discounted cash flow
0	-100,000	1.0000	-100,000
1	10,000	0.9091	9,091
2	10,000	0.8264	8,264
3	10,000	0.7513	7,513
4	20,000	0.6830	13,660
5	100,000	0.6209	62,090
		NPV	618

W14 - Activity #2: ROI and NPV calculation for ACL Mux Core Sales App



Project Cost:

- **Development Cost:** ₹35,00,000; **Marketing Cost:** ₹10,00,000 (first year)
- **Maintenance Cost (Annual):** ₹5,00,000 (next three years)

Expected Benefits (from second year onwards):

- **Cost Savings:** The app is expected to reduce counter service costs, saving an estimated ₹8,00,000 per year.
- **Revenue Increase:** By attracting more customers, the app is expected to generate an additional ₹15,00,000 per year in net revenue.

Calculate ROI and NPV (assume a discount rate of 10% per year)

W14 - Activity #2: ROI and NPV calculation for ACL Mux Core Sales App



Total net cash inflows over second to fourth year =

$$\text{₹}18,00,000 \times 3 = \text{₹}54,00,000$$

$$\text{ROI} = ((\text{₹}54,00,000 - \text{₹}45,00,000) / \text{₹}45,00,000) \times 100 = 20\%$$

Total of Discounted Cash Flows =

$$\text{₹}14,87,603 + \text{₹}13,52,276 + \text{₹}12,29,301 = \text{₹}40,69,180$$

$$\text{NPV} = \text{₹}40,69,180 - \text{₹}45,00,000 = -\text{₹}4,30,820$$

Project evaluation summary

- A project may fail not through poor management but because it should never have been started
- A project may make a profit, but it may be possible to do something else that makes even more profit
- A real problem is that it is often not possible to express benefits in accurate financial terms
- Projects with the highest potential returns are often the most risky

Program and Portfolio Management Systems



Introduction Oracle Primavera EPPM

<https://www.youtube.com/watch?v=FEINyZI-PIY>

References:

<https://www.pmi.org/learning/library/project-portfolio-management-tools-techniques-1050>



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

SS ZG622: Software Project Management Contact Session # 16a

Prof. Narasimha Bolloju, BITS-Pilani, Hyderabad Campus

Transitioning to agile methodologies



- Challenges associated with transition
- Managing and/or dealing with the challenges
- Phased transition strategy

Activity W15#1:



What are the possible challenges in transitioning to agile methodologies in large organizations with distributed teams?

Transitioning to agile methodologies



- Other functions unwilling to change (mentioned by 31% of the reported cases)
- Lack of guidance from the literature (21%)
- Reverting to the old way of working (19%)
- Misunderstanding agile concepts (19%)

Dikert K, Paasivaara M, Lassenius C (2016) Challenges and success factors in large-scale agile transformations: A systematic literature review. J Sys Softw

Activity 15#2:



How to overcome the challenges identified above?

Tips for large scale agile transformation



- Consistent process and practices (mentioned by 43% of respondents)
- Implementation of a common tool across teams (40%)
- Agile consultants or trainers (40%)
- Executive sponsorship (37%)
- Internal agile support team (35%)

VersionOne Inc (2016) 10th annual “state of agile development” survey.

<https://versionone.com/pdf/VersionOne-10th-Annual-State-of-Agile-Report.pdf>

Phased transition strategy

1. Initial assessment and agile training
2. Pilot project implementation
3. Scale agile practices across teams
4. Optimization and continuous improvement
5. Full agile transformation and integration with business strategy

Phased transition strategy

- Communication - Transparent communication is essential throughout each phase to manage expectations and reduce resistance.
- Change Management - Address resistance through change management practices, making the case for agile clear to all stakeholders.
- Leadership Support - Sustained support from leadership helps reinforce agile practices and promote an agile-friendly culture.
- Incremental Learning - Allow time for incremental learning and adjust training programs as teams grow more familiar with agile practices.
- Performance Metrics - Use metrics judiciously to measure success but avoid excessive focus on metrics that may stifle innovation.

Cross Cultural Issues in Global Project Management

Overview



- Frameworks for comparing cultural differences based on values, attitudes, and behaviors
- Hofstede's work as the foundational study in cultural dimensions
- Alternative frameworks (e.g., Trompenaars' dimensions, Hall's High- vs Low-context communication) to show the diversity of approaches to cultural analysis

Hofstede's Cultural Dimensions

innovate

achieve

lead



IEduNote.com

https://www.iedunote.com/hofstedes-cultural-dimensions-theory#google_vignette

Hofstede's Cultural Dimensions



Dimension	Description
Power Distance (PDI)	Degree to which less powerful members accept unequal power distribution.
Individualism vs. Collectivism (IDV)	Preference for individual achievement vs. group cohesion.
Uncertainty Avoidance (UAI)	Comfort level with ambiguity and risk.
Masculinity vs. Femininity (MAS)	Preference for competition and achievement (masculine) vs. caring for others and quality of life (feminine).
Long-Term vs. Short-Term Orientation (LTO)	Focus on future rewards vs. immediate outcomes.
Indulgence vs. Restraint (IVR)	Degree of freedom in fulfilling desires vs. strict societal norms.

Hofstede's Cultural Dimensions

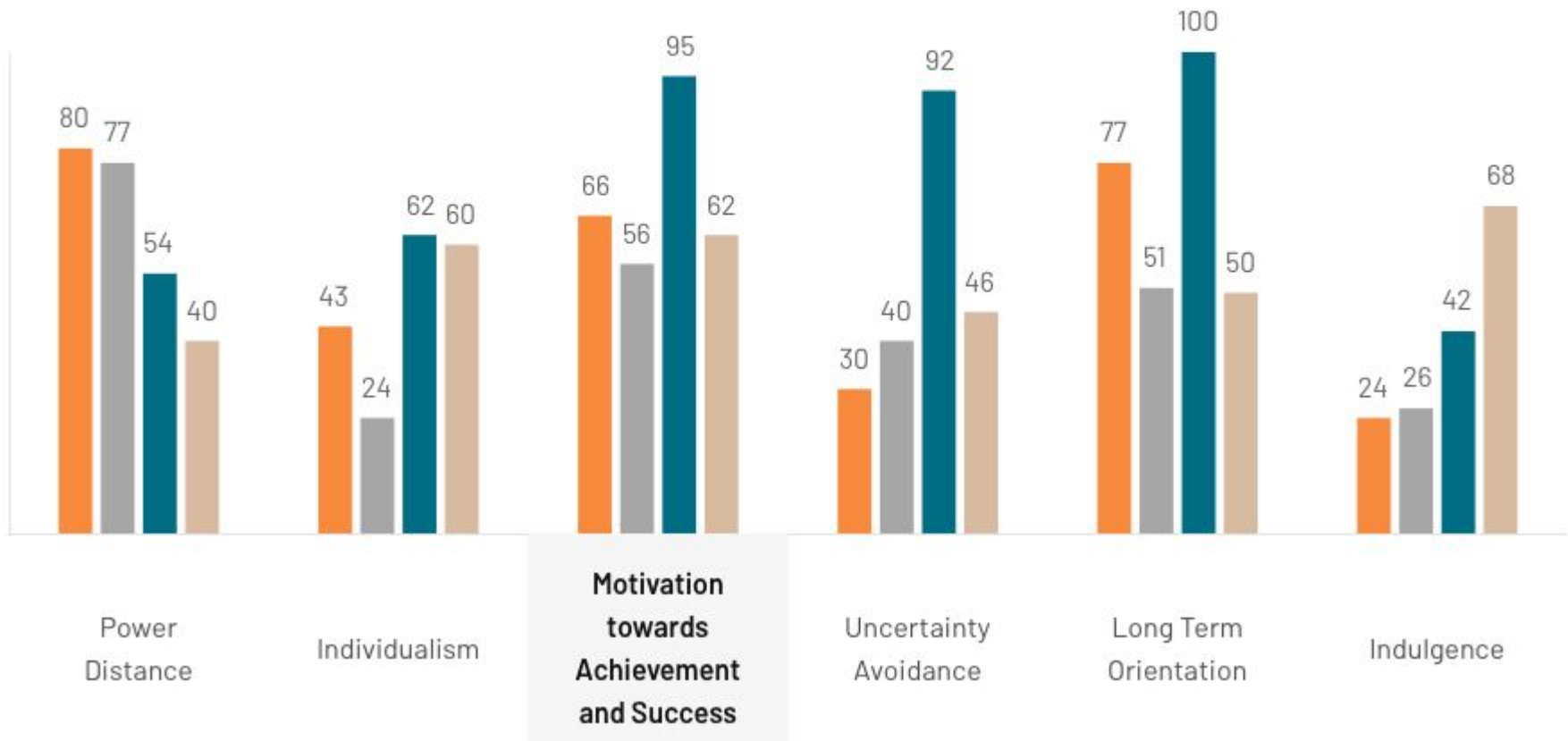
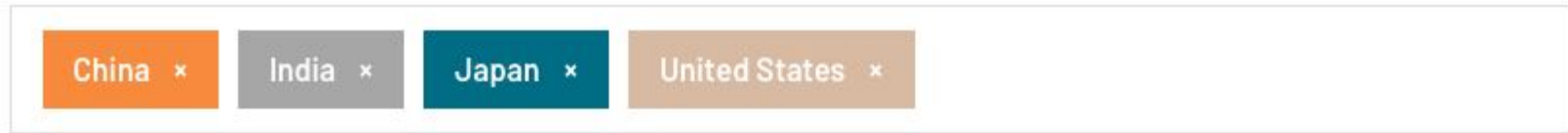


Dimension	Application in Software Project Management
Power Distance (PDI)	High PDI cultures may require clear hierarchical roles, while low PDI cultures promote more equality in communication and decision-making.
Individualism vs. Collectivism (IDV)	Individualist cultures may prefer working independently, while collectivist cultures favor collaboration and team-oriented goals.
Uncertainty Avoidance (UAI)	High UAI cultures may prefer detailed planning and risk mitigation, while low UAI cultures are more open to flexible approaches and innovation.
Masculinity vs. Femininity (MAS)	Masculine cultures may focus on deadlines and results, while feminine cultures emphasize work-life balance and team well-being.
Long-Term vs. Short-Term Orientation (LTO)	Long-term-oriented cultures may emphasize strategic project goals, while short-term cultures prioritize quick wins and immediate deliverables.
Indulgence vs. Restraint (IVR)	Indulgent cultures may encourage innovation and experimentation, while restrained cultures adhere to established norms and structured

Hofstede's Cultural Dimensions



Dimension	India	USA	UK	Germany	China	Japan
Power Distance (PDI)	77	40	35	35	80	54
Individualism (IDV)	48	91	89	67	20	46
Masculinity (MAS)	56	62	66	66	66	95
Uncertainty Avoidance (UAI)	40	46	35	65	30	92
Long-Term Orientation (LTO)	51	26	51	83	87	88
Indulgence vs. Restraint (IVR)	26	68	69	40	24	42



<https://www.theculturefactor.com/country-comparison-tool?countries=china%2Cindia%2Cjapan%2Cunited+states>

HIGH X LOW CONTEXT CULTURES

innovate

achieve

lead

JAPAN			CHINA
VIETNAM			SOUTH KOREA
RUSSIA			INDIA
ROMANIA			TURKEY
ARAB			ARMENIA
GREECE			PAN AFRICA
BRAZIL			MEXICO
SPAIN			IRELAND
FRANCE			ITALY
			BELGIUM
U.K.			CANADA
AUSTRALIA			U.S.A.
SWITZERLAND			SWEDEN
GERMANY			NORWAY
DENMARK			FINLAND

High Context

- Indirect & Implicit
- Needs context
- Relationship-oriented
- Collectivist
- Polychronic
- Attention to non-verbal



- Direct & Explicit
- No need for context
- Task-oriented
- Individualistic
- Monochronic
- Attention to verbal

Low Context

<https://www.coachingexpatriates.com/communicating-in-high-and-low-context-cultures/>

HIGH X LOW CONTEXT CULTURES

PERSONAL DIMENSIONS



DIMENSION	LOW CONTEXT CULTURE	HIGH CONTEXT CULTURE
Relationships	• Short-term, looser, broad • Subordinate to task	• Long-term, tighter, fewer • Subordinate to context
Communication	• Verbal, explicit, written, direct • Disagreements are depersonalized	• Non-verbal, implicit, oral, indirect • Conflict avoidance
Sense of time	• Monochronic, rigid • Present/Future-oriented • Punctuality is valued	• Polychronic, flexible • Tradition-oriented • Bonds & processes are valued
View of change	• Changes are fast, viewed as progress	• Traditions over any changes
Knowledge	• Explicit, conscious • Data-driven	• Implicit, not fully conscious • Community-driven
Learning	• Knowledge is transferable • Acquired from data/information • Product more important than process • Follow explanation, written or oral • Inductive reasoning	• Knowledge is situational • Acquired from people • Process more important than product • Observing and modeling others • Deductive reasoning
Locus of Control	• Greater external locus of control and blame on others for failures	• Greater internal locus of control and acceptance of personal failure
Affiliation	• Low need for affiliation • Fragile bonds • Little sense of loyalty in general	• Great need for affiliation • Strong bonds • Great sense of loyalty to affiliation
Commitment to relationships	• Lower commitment to relationships • Tasks are more important than relationships	• High commitment to relationships • Relationships are more important than tasks
Iceberg Metaphor	• Things are and happen above the water line	• Things are and happen below the water line

<https://www.coachingexpatriates.com/communicating-in-high-and-low-context-cultures/>

Model

Focus

Project Management

Hofstede's Cultural Dimensions	Analyzes cultural differences through six key dimensions like power distance, individualism, and uncertainty avoidance.	Useful for understanding and managing team dynamics, communication styles, and preferences for structure and risk tolerance.
Trompenaars' Seven Dimensions of Culture	Explores how cultures vary in relationship orientation, communication, and time management.	Helps in adjusting project processes, especially in team alignment, client expectations, and balancing task- and relationship-oriented goals.
Hall's Contextual Theory	Focuses on communication style (high vs. low context) and time perception (monochronic vs. polychronic).	Guides communication training, helping team members interpret implicit versus explicit communication styles and adapt to time management preferences.

Best practices for project managers



- Manage diverse teams considering cultural dimensions (e.g., respecting hierarchy, adapting communication styles)
- Place greater emphasis on the need to flexibility, empathy and continuous learning



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

SS ZG622:

Software Project Management

Contact Session # 16a

Prof. Narasimha Bolloju, BITS-Pilani, Hyderabad Campus

Ethical issues in software project management



- Examples of ethical failures
- Some common ethical dilemmas
- Ethical decision-making process
- Engineering ethics
- ACM Code of ethics



Uber Employees Use Secret Tools To Target Drivers And Undercut Competition

Ryan Felton (March 2017, Jalopnik.com)



Three years ago, ride-hailing giant Uber came under investigation for a tool called “God View” that allowed employees to track drivers and customers in real-time. Uber has since scaled that ability back, but Jalopnik has learned the company still deploys a revamped version of it, along with other secret internal tools, to target its drivers who also work for competitors.

Killer software: 4 lessons from the deadly 737 MAX crashes

By **Matt Hamblen** • Mar 2, 2020 01:23pm

Acceleration/Vibration

Aerospace/Military

Sensor Applications

Software



Volkswagen hit with \$14.7 billion in fines, compensation over emissions scandal

Volkswagen, prosecuted in the United States under the Clean Air Act for **falsifying diesel emissions results in several of its car models**, must pay \$10 billion to compensate drivers. The company must pay an additional \$2.7 billion fine to the US government, and spend \$2 billion on developing zero-emission vehicles.

Some common ethical dilemmas

1. Client confidentiality vs team transparency
2. Profit maximization vs ethical software dev
3. Developing surveillance software
4. Disclosure of software vulnerabilities
5. AI and recruitment bias
6. Legal compliance vs ethical standards in international projects
7. Client requests for backdoor in software

Ethical decision-making process

1. Identify Ethical Issues: What dilemmas exist?
2. Evaluate Options: What are the potential consequences of each option?
3. Consult Stakeholders: Who will be affected by the decision?
4. Make and Communicate the Decision:
Ensure transparency.

Activity W16a#1:



When you, an ethically responsible project manager, are approached by your team member to perform unethical practices (e.g., falsifying project metrics, skipping critical testing phases, or bypassing compliance protocols) how you should handle the situation?

Activity W16a#2:



When you, an ethically responsible project manager, are asked by a client to implement a functionality that is unethical (e.g., violates laws, infringes on user privacy, or misleads stakeholders) how you should handle the situation?

Activity W16a#3:



When you, an ethically responsible project manager, are asked by your boss to perform unethical practice how you should respond?

Engineering ethics



- **Engineering ethics** is the field of applied ethics and system of **moral principles** that apply to the **practice of engineering**.
- The field examines and sets the **obligations by engineers** to society, to their clients, and to the profession.
- Engineer's most basic ethical duty is to “**hold paramount the safety, health, and welfare of the public**”
- https://en.wikipedia.org/wiki/Engineering_ethics

ACM Code of Ethics - General principles



- Contribute to society and to human well-being, acknowledging that all people are stakeholders in computing
- Avoid harm - minimize the possibility of indirectly or unintentionally harming others
- Be honest and trustworthy
- Be fair and take action not to discriminate
- Respect the work required to produce new ideas, inventions, creative works, and computing artifacts
- Respect privacy
- Honor confidentiality

ACM Code of Ethics - Professional responsibilities



- Strive to achieve high quality in both the processes and products of professional work
- Foster public awareness and understanding of computing, related technologies, and their consequences
- Access computing and communication resources only when authorized or when compelled by the public good
- Design and implement systems that are robustly and useably secure

ACM Code of Ethics - Professional responsibilities



- Perform work only in areas of competence
- Maintain high standards of professional competence, conduct, and ethical practice
- Know and respect existing rules pertaining to professional work
- Accept and provide appropriate professional review
- Give comprehensive and thorough evaluations of computer systems and their impacts, including analysis of possible risks.